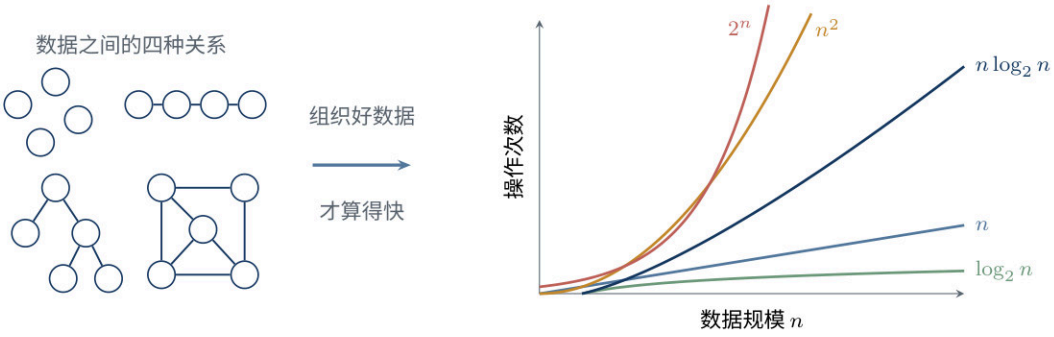
绪　论

数据结构的基本概念与算法复杂度分析

从“数据怎么组织”到“算法有多快”：一份可以独立读懂的入门讲义



所属课程　《数据结构与算法》（C++ 版），第 1 章 绪论

本讲义含　12 个知识模块 · 逐点配图 · 生活应用 · 相似知识对比 · 38 道分层练习与图解解析

适用场景　课前预习、课后复习、作业前梳理、期中期末基础巩固

XeLaTeX 排版 · 全部图形为 TikZ / PGFPlots 矢量绘制

阅读说明

这份讲义是依据你的课堂笔记和老师的第 1 章课件重新编写的，目标不是把课件抄一遍，而是让一位**第一次学、基础不牢**的同学只读这一份材料就能看懂。课件里为了节省篇幅而省掉的定义条件、推导步骤、证明、正例与反例，这里都按教材的标准内容补上了，并用“教学补全”字样标出，方便你区分哪些是老师讲的、哪些是讲义补的。

蓝：核心知识	米：例子与应用	红：易错与纠错	绿：总结与自检	灰蓝：联系与对比
定义 · 定理 · 公式	例题 · 生活 · 历史	易错点 · 原图纠错	方法 · 小结 · 自检	相似知识 · 迁移

图 1　全篇色块的固定含义：看到颜色就知道这一块在干什么

怎么读　每个知识模块都按同一条路线展开：为什么需要 → 先看图 → 正式定义 → 公式与推导 → 最简单的例子 → 生活中的它 → 易错点 → 和相似知识比一比 → 通向下一节。第一次读可以跳过带“拓展联想”字样的内容。

三个记号　在“迁移判断表”里，✓ 表示旧知识的结论可以**直接搬过来用**；△ 表示**要加条件**才能用；✗ 表示**一搬就错**。

练习怎么用　38 道正式练习分三层：基础 25 道、中等偏下提升 10 道、跨学科迁移 3 道。题目区的图只画题目给出的信息；答案和带标注的解析图统一放在后面，建议先独立做完再对照。

一点技术说明　讲义用 XeLaTeX 编译。行内代码与程序清单用等宽字体，如 `for (i=0; i<n; i++)`；数学量用斜体，如问题规模 n、时间频度 $T(n)$。全文统一约定：$\log_2 n$ **表示以 2 为底的对数**；课件中出现的 $\mathrm{lb}\, n$ 是它的另一种写法。

目录

阅读说明 1

导读　本专题学什么、从哪里来、有什么用 3

0　导读与资料整理 3
- 0.1　本专题学什么 . . . 3
- 0.2　它在课程体系中的位置 . . . 3
- 0.3　学科小史：这些概念是怎样出现的 . . . 4
- 0.4　从课本到现实：这一章到底有什么用 . . . 4
- 0.5　原始资料重新编码 . . . 5
- 0.6　原图纠错 . . . 5
- 0.7　知识点总览 . . . 7

第一部分　数据怎么组织：从现实问题到内存 7

1　为什么要学数据结构 7
- 1.1　它解决什么问题 . . . 7
- 1.2　先看图：用计算机解决问题的五个步骤 . . . 7
- 1.3　三个例子，三种模型 . . . 8
- 1.4　最简单的例子 . . . 8

2　数据、数据元素与数据项 10
- 2.1　先看图：一张表里的三个层次 . . . 10
- 2.2　正式定义 . . . 10
- 2.3　最简单的例子 . . . 11

3　数据结构与逻辑结构 12
- 3.1　它解决什么问题 . . . 12
- 3.2　先看图：四种基本关系 . . . 13
- 3.3　正式定义 . . . 13
- 3.4　怎样把“关系”写成数学：有序对与二元组 13
- 3.5　四种逻辑结构的精确区别 . . . 14
- 3.6　最简单的例子 . . . 15
- 3.7　数据结构这门课研究哪四件事 . . . 15

4　存储结构：顺序存储与链式存储 16
- 4.1　它解决什么问题 . . . 16
- 4.2　先看图：同一个线性结构的两种存法 . . . 17
- 4.3　顺序存储的寻址公式 . . . 17
- 4.4　逻辑结构、存储结构与操作的三句话 . . . 18

5　基本操作与抽象数据类型 20
- 5.1　基本操作的五大类 . . . 20
- 5.2　抽象数据类型（ADT） . . . 20
- 5.3　ADT 在 C++ 里怎么落地 . . . 21

第二部分　算法好不好：从“数次数”到“看增长” 24

6 算法及其特性 24
6.1 它解决什么问题 . . . 24
6.2 先看图：算法的五个特性 . . . 24
6.3 好算法的五条基本要求 . . . 25
6.4 设计算法要用“计算思维” . . . 25

7 时间频度：先学会“数次数” 27
7.1 为什么不直接用秒表 . . . 27
7.2 正式定义 . . . 27
7.3 先看图：逐句数一遍 . . . 27
7.4 度量原则 . . . 28
7.5 一个省力的简化：只数循环体 . . . 28

8 大 O 表示法：只看增长的阶 30
8.1 它解决什么问题 . . . 30
8.2 先看图：什么叫“被 $C \cdot f(n)$ 压住” . . . 30
8.3 正式定义 . . . 30
8.4 最高次项、去系数：这条规则从哪里来 . . . 31
8.5 课件里“极限”的那句话 . . . 32

9 常见的时间复杂度 34
9.1 先看图：增长曲线 . . . 34
9.2 每一个阶长什么样 . . . 35
9.3 差距到底有多大 . . . 36

10 求和定理、求积定理与循环分析 38
10.1 两个定理 . . . 38
10.2 五种循环形状 . . . 39

11 空间复杂度 42
11.1 算法占用的空间分三块 . . . 42
11.2 先看图：同一件事的两种做法 . . . 42

第三部分　写代码要用的四个 C++ 工具 45

12 面向对象、模板、const 与异常处理 45
12.1 面向过程与面向对象 . . . 45
12.2 泛型机制：函数模板与类模板 . . . 45
12.3 const 机制与引用 . . . 46
12.4 异常处理 . . . 46
12.5 把四样东西合在一起：一个完整程序 . . . 47
本章知识地图 . . . 49

第四部分　分层练习：38 道 50

13 练习说明与知识点—题目对应表 50

14 A 层：基础巩固训练（25 道） 50

15 B 层：中等偏下提升训练（10 道） 54

16 C 层：跨学科迁移训练（3 道） 57

第五部分　答案速查与逐题图解 59

17 答案速查 59

18 A 层详细解析 59

19 B 层详细解析 69

20 C 层详细解析 75

第六部分　总表：复习时看这几页 77

21 易错点、解题步骤与概念对比总表 77
21.1 高频易错点总表 . . . 77
21.2 基础解题步骤总结 . . . 77
21.3 相近概念总对比 . . . 78
21.4 跨学科 / 相似知识总对比表 . . . 79

22 后续课程连接与参考资料 79
22.1 这一章的内容以后在哪里再出现 . . . 79
22.2 参考资料 . . . 80

导读　本专题学什么、从哪里来、有什么用

第 0 节　导读与资料整理

0.1　本专题学什么

第 1 章“绪论”不讲任何一种具体的数据结构，它做两件打地基的事：第一，把后面每一章都要用到的**词汇**讲清楚（数据元素、逻辑结构、存储结构、抽象数据类型）；第二，给出一把**尺子**（时间复杂度与空间复杂度），以后每学一个算法都要用它量一量“快不快、省不省”。

学完本专题，你应该能够

(1) 说出数据、数据元素、数据项三者的包含关系，并在一张表里指认它们；
(2) 画出集合、线性、树形、图四种逻辑结构，并用二元组 (D, R) 描述一个具体结构；
(3) 区分“逻辑结构”和“存储结构”，说明顺序存储与链式存储各自怎样保存元素之间的关系；
(4) 解释抽象数据类型（ADT）的“抽象性”和“封装性”，并把一组操作归入五大类；
(5) 复述算法的 5 个特性和 5 条基本要求，并给每个特性举一个反例；
(6) 对一段含循环的程序，数出基本语句的执行次数 $T(n)$，并写出它的大 O 表示；
(7) 按从快到慢的顺序排出 $O(1)$、$O(\log_2 n)$、$O(n)$、$O(n\log_2 n)$、$O(n^2)$、$O(2^n)$、$O(n!)$；
(8) 用求和定理、求积定理分析顺序、嵌套、内外关联、倍增四类循环；
(9) 判断一段程序使用的辅助空间是 $O(1)$ 还是 $O(n)$。

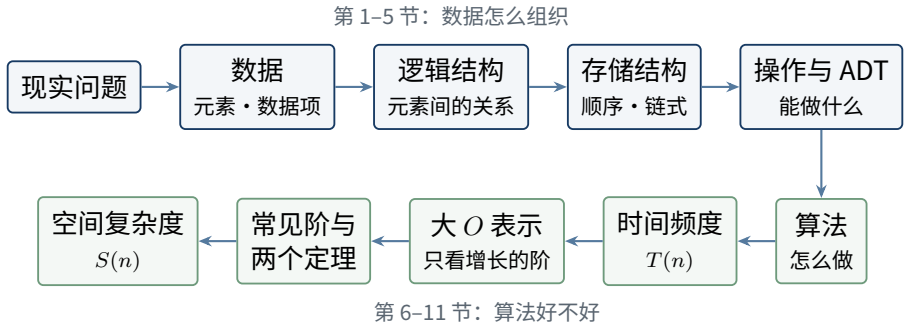


图 2　本专题的知识路线：上一行回答“数据怎么组织”，下一行回答“算法好不好”

读图：蓝色方框是“数据结构”这一半，绿色方框是“算法分析”这一半；箭头表示“学完前一个才好理解后一个”。第 12 节另外复习课件末尾要求的四个 C++ 工具（面向对象、模板、const、异常处理），它们是后面各章写代码时的通用工具。

0.2　它在课程体系中的位置

课件第 9 页指出：数据结构是一门**介于数学、计算机硬件和计算机软件之间**的核心课程。下面左图按课件原意重画；右图说明本章依赖什么、通向哪里。

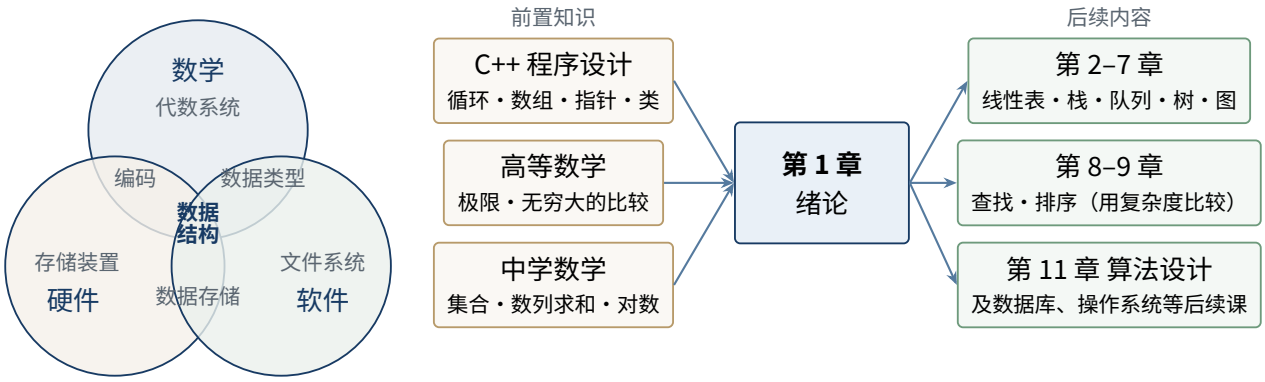


图 3　左：数据结构处在数学、硬件、软件的交汇处（按课件第 9 页重画）；右：本章的前置知识与后续去向

读图：左图三个圆分别是数学、硬件、软件，三者重叠的中心区域就是“数据结构”。右图中米色是你已经学过的内容，绿色是学完本章后要去的地方。本章真正用到的前置知识很少：会读 `for` / `while` 循环，会用等差数列求和公式 $1+2+\cdots+n=\dfrac{n(n+1)}{2}$，知道对数 $\log_a n$ 的含义，就足够了。

0.3 学科小史：这些概念是怎样出现的

学科小史　从“会算”到“会比较哪种算法更好”

“算法”一词的西文 algorithm，来源于 9 世纪数学家花拉子米（al-Khwārizmī）名字的拉丁化写法；但把算法当成一门**可以定量比较**的学问，是电子计算机出现以后的事。早期计算机主要做数值计算，数据就是一串数；到 20 世纪 60 年代，人们开始用计算机处理表格、文本、层次和网络这类“带关系”的数据，于是出现了两个新问题：**这些关系在内存里怎么放？同一个问题的两种做法怎么比较优劣？**



大 O 记号最早是纯数学工具：1894 年德国数学家巴赫曼（P. Bachmann）在数论著作中引入它，随后被兰道（E. Landau）沿用和推广，所以也叫“兰道记号”。几十年后，高德纳（D. E. Knuth）等人把它用于描述算法的运行时间，并在 1976 年补充了 Ω、Θ 等配套记号——这就是今天课本里“时间复杂度 $O(n^2)$”这种写法的来历。另一条线是数据的组织：1974 年提出的抽象数据类型思想，主张“先规定能做哪些操作，再决定内部怎么存”，这与 C++ 中“类”把数据和操作封装在一起的做法一脉相通，也是本讲义第 5 节的主角。

0.4 从课本到现实：这一章到底有什么用

从课本到现实　三个每天都在发生的例子

① **查找：为什么十亿条记录也能秒查。**数据库和搜索系统不会从第一条记录看到最后一条，而是先把数据组织成有序的或分层的结构。若 $n=10^9$ 条记录排好了序，每次比较排除一半，最多只需约 $\log_2 10^9 \approx 30$ 次比较；而逐条比较最坏要 10^9 次。差别完全来自“数据怎么组织”和“算法是什么阶”，这正是本章两条主线。



② **文件夹：树形结构。**电脑里“磁盘—文件夹—子文件夹—文件”的层层包含关系，就是课件第 12 页 UNIX 文件系统那张图所示的**树形结构**：一个文件夹里可以有多个子项，但每个子项只属于一个上级文件夹。

③ **导航与管网：图结构。**地图上的路口和道路、课件第 13 页的通信网、第 17 页的煤气管道，都是“多对多”的**图结构**。在图上找最短路径、找最省钱的连通方案，是第 7 章要学的算法；而判断这些算法“在几百万个路口上跑不跑得动”，用的就是本章的时间复杂度。

0.5 原始资料重新编码

这一小节交代讲义依据了什么材料，以及这些材料被整理到了哪一节。

原始资料	可靠识别出的内容	对应本讲义
手写笔记（1 页）	数据结构分类；算法的含义；算法复杂度的含义（时间和空间的**增长速度**）；7 种常见时间复杂度及各自的代表；复杂度大小关系与“推荐 / 不推荐”分界线	第 3、6、8、9 节
课件第 1–6 页	课程信息、章节目录、实验与成绩安排	不属于知识内容，未展开
课件第 7–17 页	为什么学数据结构；课程地位；解题流程；学生表、文件系统、通信网三种结构；数值与非数值计算；电话查询、人机对弈、煤气管道三例	第 0、1 节
课件第 18–32 页	数据、数据元素、数据项；数据结构定义；逻辑结构及二元组；五类基本操作；存储结构；3 道单选题	第 2–5 节、练习
课件第 33–35 页	抽象数据类型及两个特性；顺序表类模板 `SqList`	第 5 节
课件第 36–54 页	算法的 5 个特性与 5 条要求；时间频度；大 O 定义；常见复杂度；求和 / 求积定理；空间复杂度与逆置示例	第 6–11 节
课件第 55–63 页	小结；6 道复杂度填空题；3 道作业	练习与解析
课件第 64–83 页	C++ 复习：面向对象、泛型机制、`const` 机制、异常处理	第 12 节

无法可靠辨认或不完整之处　（1）笔记照片上能看到纸张背面透过来的字迹，这些内容不属于本页，未作识别。（2）课件第 15 页“记录了 □ 个人的名字”一句中数量处为空白，按上下文应为“n 个人”——**此为根据上下文的推测**。（3）课件第 15、16 页“算法”“模型”标注上叠有问号动画，文字部分被遮挡，本讲义按可辨认部分转述。

0.6 原图纠错

以下是在笔记和课件中发现的笔误或不够严谨之处。它们大多是课堂上为求简洁造成的，这里逐条列出规范写法，后文一律按规范写法讲解。

原图纠错　手写笔记

(1) **原始写法：**数据结构“分类：线性结构、散列结构、树形结构、图形结构等”。**问题：**线性、树形、图是按**逻辑关系**分的；“散列”是一种**存储 / 查找技术**（第 8 章哈希表），不是逻辑结构，把它并列进来混用了两套标准。**规范写法：**逻辑结构分为**集合、线性、树形、图**四种（课件第 20、23 页）。**影响：**第 3、4 节按规范分类讲解。

(2) **原始写法：**“$O(\log n)$（对数指数上涨）”。**问题：**对数增长和指数增长正好是一慢一快两个极端，不能连用。**规范写法：**$O(\log_2 n)$ 称**对数阶**，增长非常缓慢。

(3) **原始写法：**“$O(1)$：哈希表”“$O(n\log n)$：快速排序”。**问题：**缺少条件。**规范写法：**哈希表查找是**平均** $O(1)$，最坏可退化到 $O(n)$；快速排序是**平均** $O(n\log_2 n)$，最坏为 $O(n^2)$。**影响：**第 9 节的总表中已补上条件，这两种结构分别在第 8、9 章详细学习。

(4) **原始写法：**“$O(n)$（随着数据量线性搜索）”。**规范写法：**$O(n)$ 称**线性阶**，含义是“操作次数随数据量成正比增长”；线性搜索只是它的一个例子。

原图纠错　课件

(1) **第 19 页：**“数据元素是数据处理的最小单位”，同页又说“数据项是不可分割的最小单位”，两句话冲突。**规范写法：**数据元素是数据的**基本单位**（处理时作为一个整体）；数据项才是**不可分割的最小单位**。

(2) **第 34 页：**标题“抽象数据库类型”多了一个“库”字，应为**抽象数据类型**（Abstract Data Type，ADT）。

(3) **第 37 页：**“有输出：有零个或者若干个输出”。**问题：**没有任何输出的算法毫无意义。**规范写法：**一个算法有**零个或多个输入**，有**一个或多个输出**（高德纳及国内通行教材均如此表述）。

(4) **第 43 页：**同一例中先写“$n>0$ 时为 $3n+2$ 次”，后写“时间频度为 $3n+3$”。**原因：**前者没有把 `if (n>0)` 这次判断算进去，后者算了。**规范写法：**连同判断共 $3n+3$ 次（推导见第 7 节）；按同样口径，$n\le 0$ 时应为 2 次而不是 1 次。两种口径只差常数，不影响复杂度 $O(n)$。

(5) **第 44 页：**“$T(n)/f(n)$ 的极限为非零常数，称 $O(f(n))$……即 $\exists C, N_0$，$\forall n>N_0$ 有 $T(n)\le C\cdot f(n)$”。**问题：**“即”字前后两句并不等价：后一句才是大 O 的标准定义（上界）；前一句是更强的条件（同阶），能推出后一句，反过来不行。**影响：**第 8 节给出标准定义，并证明“极限为非零常数 $\Rightarrow$ 大 O 成立”。

(6) **第 47 页：**代码内层循环写的是 `for (j=0; j<i; j++)`，求和式却写成 $\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}1=n(n+1)/2$。**问题：**求和式对应的是 `for (j=i; j<n; j++)`，与代码不一致。**规范写法：**对代码 `j<i`，循环体执行 $\sum_{i=0}^{n-1} i=\dfrac{n(n-1)}{2}$ 次。两者都是 $O(n^2)$，结论不受影响（两种写法的对比见第 10 节例题）。

(7) **第 49 页：**把 $O(2^n)$ 称为“幂阶”。**规范写法：**2^n 是指数函数，通行名称是**指数阶**；n^2、n^3 这类“底数是 n”的才是幂函数。

(8) **代码笔误：**第 59 页 `for i=0; i<n; i++)` 缺左括号；第 62 页 `while(x⩾(y+1)*(y+1))` 中“⩾”在 C++ 里应写作 `>=`；第 71 页 `count<<max(3,5)` 应为 `cout`，且 `int main` 后缺 `()`；第 35 页类模板参数名为 `ElemType`，后面却写成 `elemType`（C++ 区分大小写）。本讲义引用时均已改正，并且第 12 节的完整程序经过实际编译运行。

0.7 知识点总览

节	知识模块	一句话	来源
1	为什么学数据结构	非数值问题的“数学模型”不是方程，而是表、树、图	课件
2	数据·数据元素·数据项	一张表是数据对象，一行是数据元素，一格是数据项	课件＋补全
3	逻辑结构	元素之间的关系只有四种：没有、一对一、一对多、多对多	笔记＋课件
4	存储结构	关系进了内存，要么靠“挨着放”，要么靠“记地址”	课件＋补全
5	基本操作与 ADT	先约定能做什么，再决定怎么做	课件＋补全
6	算法及其特性	有限步、无歧义、做得到、有结果	笔记＋课件
7	时间频度 $T(n)$	数基本语句执行了多少次	课件＋补全
8	大 O 表示法	只留最高次项，去掉系数	笔记＋课件＋补全
9	常见的时间复杂度	对数最慢、指数和阶乘爆炸	笔记＋课件
10	两个定理与循环分析	顺序相加取大，嵌套相乘	课件＋补全
11	空间复杂度	只数“额外借用”的空间	课件
12	四个 C++ 工具	类、模板、const、异常	课件

第一部分　数据怎么组织：从现实问题到内存

第 1 节　为什么要学数据结构

1.1 它解决什么问题

课件第 7 页的开场白是：计算机科学研究的是**信息的表示**与**信息的处理**，两者密不可分。早期计算机主要做**数值计算**——解方程、算积分，数据就是一些数，彼此之间没有什么“结构”可言。后来计算机要处理的东西变成了学生名单、文件目录、通信网络、棋局……这些数据的共同点是：**单个数据本身很简单，难的是它们之间有关系**。这类问题叫**非数值计算**问题。

对数值计算问题，建立数学模型就是列方程（课件举的例子：天气预报对应偏微分方程，人口增长预测对应常微分方程）。对非数值计算问题，方程帮不上忙，这时的“数学模型”是**数据结构**：用表、树、图这样的结构把对象和对象之间的关系描述出来。

1.2 先看图：用计算机解决问题的五个步骤

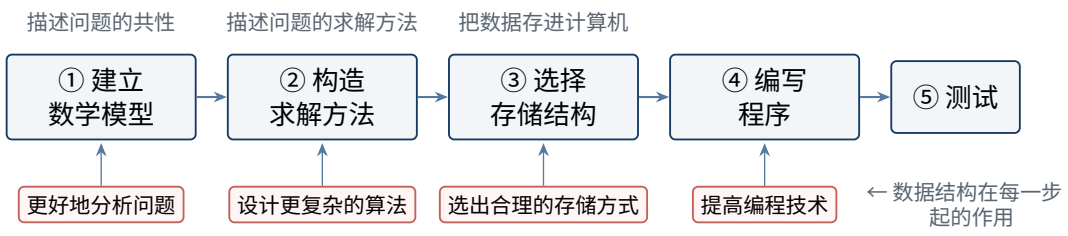


图 4　用计算机解决问题的过程，以及数据结构在其中的作用（按课件第 10 页重画）

读图：中间一行蓝框是解决问题的五个先后步骤；上方灰字说明每一步在做什么；下方红框是“学了数据结构以后，这一步能做得更好的地方”。最该看出的是：数据结构不只影响第 ③ 步，它从第 ① 步“怎么描述

问题”就开始起作用。

1.3 三个例子，三种模型

课件用三张图说明非数值问题里的数据各有各的“形状”：

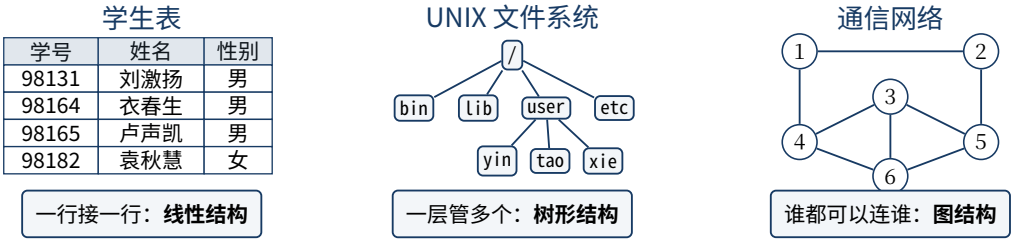


图 5　三类非数值问题对应三种数据结构模型（依据课件第 11–13 页）

读图：左图每个学生占一行，行与行之间只有“前一个、后一个”的关系；中图每个文件夹下面可以挂多个子项；右图每个站点可以同时和好几个站点相连。三张图的差别不在于“存的是什么”，而在于**元素之间的关系是什么形状**。

课件第 15–17 页的三个例题正好与此对应，整理如下（课件在这三页的标注里把“选用的结构”写在了“算法”一栏，这里统一称为“模型”）：

问题	要处理的对象及其关系	选用的模型
电话号码查询	n 个人的（姓名，号码）记录一条接一条排列	线性表
人机对弈（井字棋）	一个棋局可以走出多个后续棋局，层层展开	树（对弈树）
铺设城市煤气管道	n 个小区是点，可铺的管线是带造价的边；只需铺 $n-1$ 条并使总造价最低	图

1.4 最简单的例子

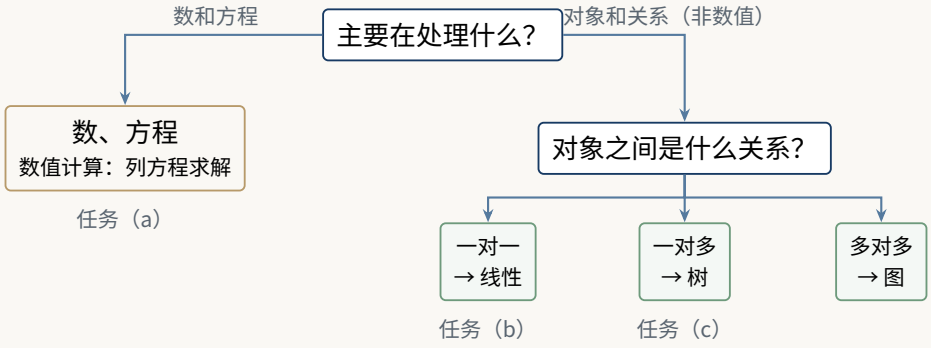
例题　1.1　判断问题类型并选模型

题目　下面三个任务，哪些是数值计算问题，哪些是非数值计算问题？非数值问题应选什么模型？

（a）根据一个微分方程预测 10 年后的人口数；　（b）在图书馆系统里查一本书是否可借；

（c）表示“学校—学院—专业—班级”的隶属关系。

思路　先问“要处理的主要是数和方程，还是对象和关系”；若是后者，再看关系是一对一、一对多还是多对多。



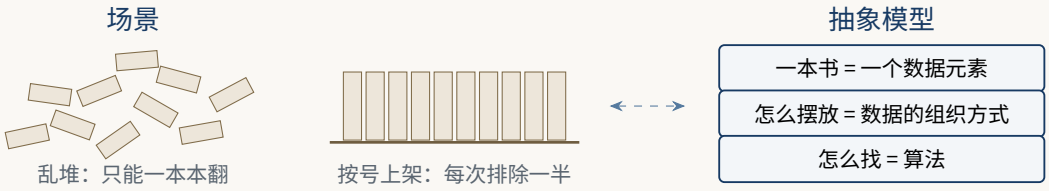
结果　（a）数值计算，模型是常微分方程；（b）非数值计算，图书记录一条接一条，

用**线性表**；（c）非数值计算，一个学院下有多个专业、一个专业下有多个班级，用**树**。

本题想说明什么　“选模型”其实就是在回答“关系是什么形状”。

生活中的它　图书馆里找一本书

生活场景　同样是 10 万册书，胡乱堆在地上和按索书号上架，找书的难度天差地别。



一次小估算　$n = 100\,000$ 册。乱堆时最坏要翻 100 000 本；排好序后每次从中间一分为二，最多只需约 $\log_2 100\,000 \approx 17$ 次。

边界说明　真实图书馆还会分楼层、分区、分架，是多级索引；这里只保留了“有序”这一个最关键的因素。

易错点

(1) **“数据结构就是数组和链表的代码。”**不对。数组、链表只是**存储**手段；数据结构包含逻辑结构、存储结构和其上的操作三件事（第 3–5 节）。

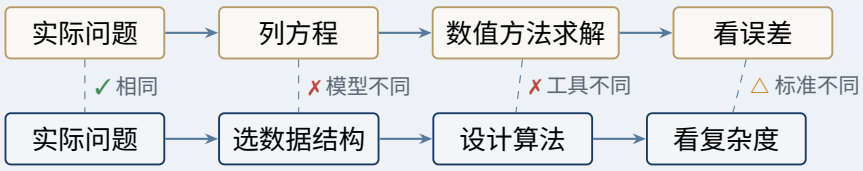
(2) **“这门课是在学 C++。”**不对。C++ 只是描述工具，换成别的语言，线性表仍然是线性表。

(3) **“非数值计算就是不算数。”**不对。它的意思是**模型不是数学方程**，处理的重点是关系；过程中照样有比较和加减。

对照　方程模型 ↔ 数据结构模型（方法互补型）

一句话定位　你在数学和物理课上熟悉的“先列方程再求解”，与本课的“先选数据结构再设计算法”，是同一套“建模—求解”思路用在两类不同问题上。

比较维度	数值计算：方程模型	非数值计算：数据结构模型
研究对象	连续变化的量（温度、人口、电压）	离散的对象及其关系（记录、结点、连线）
模型形式	代数方程、微分方程	线性表、树、图
求解工具	差分法、有限元法等数值方法	查找、排序、遍历等算法
结果好坏看什么	误差有多大	时间和空间用了多少



迁移判断

从数学 / 物理课带来的习惯	迁移结果	理由或反例
解题要先建立模型，再求解	✓ 直接成立	课件第 10 页的流程第 ① 步就是“建立数学模型”
模型一定是方程	✗ 不成立	井字棋的对弈过程列不出方程，它的模型是一棵树
解得好不好，看误差大小	△ 加条件	只对数值问题适用；查找、排序的结果没有“误差”，好坏看时间复杂度和空间复杂度

到这里，我们已经解决了什么？　知道了非数值问题的模型是数据结构。下一步还需要解决什么？“数据结构”四个字里，“数据”究竟指什么、最小能分到多细？这是第 2 节的内容。

第 2 节　数据、数据元素与数据项

2.1　先看图：一张表里的三个层次

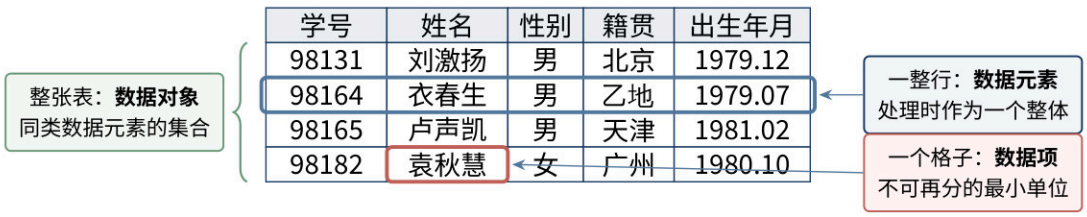


图 6　学生表中的数据对象、数据元素与数据项（表格内容取自课件第 11 页）

读图：蓝框圈住的一整行是一个**数据元素**（一个学生的全部信息）；红框圈住的一个格子是一个**数据项**（这个学生的姓名）；左侧花括号括住的所有行合在一起是一个**数据对象**。浅色表头不是数据，它说明每个数据项叫什么名字。

2.2　正式定义

定义　数据、数据元素、数据项

数据（data）：信息的载体，是描述客观事物的数、字符，以及所有能输入到计算机中、被计算机程序识别和处理的**符号的集合**。数据分为数值性数据和非数值性数据。

数据元素（data element）：数据的**基本单位**，在程序中通常作为一个**整体**来考虑和处理。例如员工信息表中，每个员工的一条记录是一个数据元素。

数据项（data item）：数据元素中**不可分割的最小单位**。一个数据元素可以由若干个数据项组成，例如姓名、性别、地址、电话。

教学补全　课件作业 1.1 还问到了“数据类型”，课件正文没有给出定义，这里按通行教材补上两个相关概念：

数据对象：性质相同的数据元素的集合，是数据的一个子集。例如“全体学生记录”是一个数据对象。

数据类型：一个**值的集合**以及定义在这个值集上的**一组操作**的总称。例如 C++ 的 `int`：值

集是某个范围内的整数，操作有加、减、乘、除、取余、比较等。

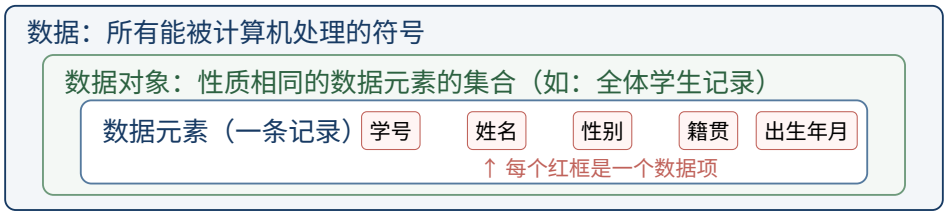


图 7　四个概念的包含关系：数据 ⊃ 数据对象 ⊃ 数据元素 ⊃ 数据项

读图：由外到内范围越来越小。记住一条线索即可：数据项组成数据元素，数据元素组成数据对象，数据对象是数据的一部分。

2.3 最简单的例子

例题　2.1　在一张小表里数一数

题目　某图书管理程序保存了 3 本书，每本书记录（书号，书名，价格）。问：有几个数据元素？每个数据元素含几个数据项？一共保存了多少个数据项的值？

书号	书名	价格	
B001	数据结构	45	← 第 1 个数据元素
B002	高等数学	52	← 第 2 个数据元素
B003	大学物理	48	← 第 3 个数据元素
数据项 1	数据项 2	数据项 3	

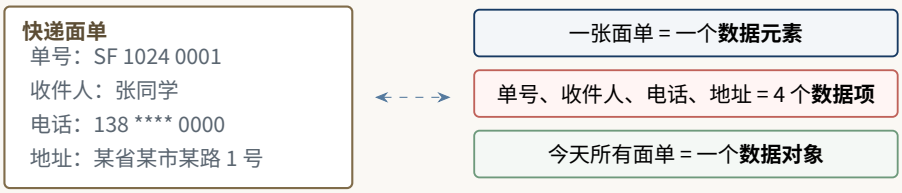
解　一行是一个数据元素，共 **3 个**；每个数据元素有书号、书名、价格 **3 个数据项**；所以共保存 $3 \times 3 = 9$ 个数据项的值。

检查　表头那一行不是数据元素，它只是数据项的名字。

本题想说明什么　“行”对应数据元素，“列”对应数据项。

生活中的它　一张快递面单

生活场景　快递站每天登记几百个包裹，每个包裹贴着一张面单。



一次小估算　一天 200 个包裹：200 个数据元素，每个 4 个数据项，共登记 $200 \times 4 = 800$ 个数据项的值。

边界说明　“地址”其实还能拆成省、市、街道。数据项“不可分割”是**相对于当前问题**而言的：只要程序不需要单独处理省份，地址就可以当作一个不再拆的数据项。

易错点

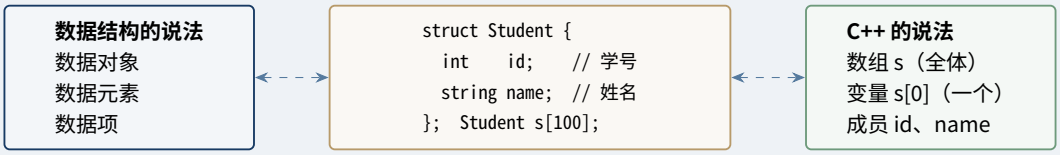
(1) **把数据元素和数据项弄反**。数据元素是“一条记录”，数据项是“记录里的一栏”。口诀：**元素是行，数据项是格**。

(2) **“数据元素是最小单位。”**这是课件第 19 页的不严谨说法（见原图纠错）。数据元素是**基本单位**，最小单位是数据项。

(3) **认为同一个数据元素里各数据项的类型必须相同**。不需要。学号是整数、姓名是字符串完全可以。要求的是：**不同数据元素之间**，数据项的个数相同、对应数据项的类型一致（课件第 32 页单选题第 3 题的考点）。

对照　数据元素 · 数据项 ↔ C++ 结构体变量 · 成员（思想重现型）

一句话定位　你在 C++ 里用 `struct` 或 `class` 定义的一个变量，就是一个数据元素在程序里的样子；它的每个成员对应一个数据项。



从 C++ 课带来的认识	迁移结果	理由或反例
一个结构体变量可以看成一个数据元素	✓ 直接成立	它把一个对象的各项信息打包成整体来处理
结构体的每个成员就是一个数据项	△ 加条件	成员本身若又是结构体或数组，它还可再分，此时它不是“最小单位”
数组的元素必须同类型，所以数据项也必须同类型	✗ 不成立	同类型的要求针对的是**元素与元素之间**；一个元素内部，学号（整数）和姓名（字符串）类型不同

到这里，我们已经解决了什么？　弄清了“数据”可以分到多细。**下一步：**一堆数据元素放在一起，如果彼此毫无关系，那只是一袋零件；**关系**才让它们成为“结构”。

第 3 节　数据结构与逻辑结构

3.1　它解决什么问题

食堂窗口前排着的一队人，与操场上随意站着的一群人，人数可以一样，但前者多了“谁先谁后”的关系——正是这层关系决定了“下一个该谁打饭”。数据结构研究的重点就是这层**关系**。

3.2 先看图：四种基本关系

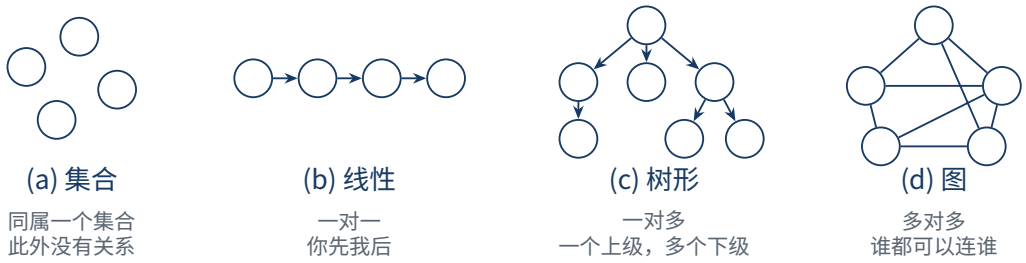


图 8　四种基本逻辑结构（依据课件第 20、23 页重画）

读图：小圆圈代表数据元素，连线或箭头代表两个元素之间有关系。从左到右，“一个元素最多能和几个元素有关系”逐步放宽：(a) 没有，(b) 前后各一个，(c) 上面一个、下面多个，(d) 任意多个。

3.3 正式定义

定义　数据结构

数据结构是相互之间存在着一定关系的数据元素的集合。更具体地说，是**有限个**、**类型相同**、相互之间具有**一定制约关系**的数据元素组成的集合。

例：某窗口前的队列——人数有限；都是“人”这一类型；彼此有“你先我后”的关系。

定义里的三个关键词缺一不可，可以逐个用反例检验：

条件	含义	去掉它会怎样（反例）
有限个	元素个数是一个确定的有限数	“全体自然数”有无穷多个，无法全部存入计算机
类型相同	每个元素的数据项个数相同、对应类型一致	一条学生记录和一条图书记录混在同一张表里，就没法统一处理
有关系	元素之间存在线性、树形或图等关系；集合结构是“只同属一个集合”这一最弱的关系	不说明关系，就不知道“下一个”是谁，也就无从设计操作

3.4 怎样把“关系”写成数学：有序对与二元组

课件第 21 页约定：逻辑结构图中，小圆圈表示数据元素，带箭头的线段表示一个**有序对**。

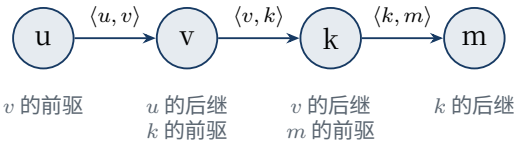


图 9　有序对 $\langle u, v\rangle$：箭头从前驱指向后继

读图：箭头 $u \to v$ 就是有序对 $\langle u, v\rangle$：u 称为 v 的**直接前驱**（简称前驱），v 称为 u 的**直接后继**（简称后继）。“有序”的意思是顺序不能颠倒，$\langle u, v\rangle \neq \langle v, u\rangle$。全部有序对放在一起，就完整描述了元素之间的关系。

核心公式　逻辑结构的二元组描述

$$\text{Data_Struct} = (D,\ R)$$

D：数据元素的有限集合
“有哪些元素”

R：D 上关系的集合
由有序对组成，“谁和谁有关系”

符号说明　D 取自 Data，是元素集合；R 取自 Relation，是关系集合，它的每个成员是一个有序对 $\langle x, y\rangle$，其中 $x, y \in D$。**使用条件：**D 必须是有限集；R 中出现的元素必须都属于 D。

课件的例子　整数 $1 \sim 10$ 组成的有序集是一个线性结构：

$$D = \{x \mid 1 \le x \le 10,\ x \in \mathbb{N}\}, \qquad R = \{\langle x_i, x_{i+1}\rangle \mid x_i \in D,\ x_{i+1} \in D\},$$

即 $R = \{\langle 1,2\rangle, \langle 2,3\rangle, \ldots, \langle 9,10\rangle\}$，共 9 个有序对。

> **为什么这样定义**　“逻辑结构是事物本身的性质，和计算机没有关系”（课件第 24 页）。二元组里只有“元素”和“关系”，完全没提数据放在内存哪里、用数组还是指针——这正是把它叫做“**逻辑**结构”的原因。同理，逻辑结构也与元素本身的形式、内容、相对位置、个数无关（课件第 31 页单选题第 2 题）：把队伍里的人全换一批，它仍然是线性结构。

3.5　四种逻辑结构的精确区别

〔教学补全〕课件用“一对一、一对多、多对多”概括三种关系。把它翻译成“每个元素有几个前驱、几个后继”，判断起来更可靠：

结构	关系	前驱个数	后继个数
集合	无	0（$R = \varnothing$）	0
线性	一对一	第一个元素 0 个，其余恰好 1 个	最后一个元素 0 个，其余恰好 1 个
树形	一对多	根 0 个，其余恰好 1 个	0 个、1 个或多个
图	多对多	任意多个	任意多个

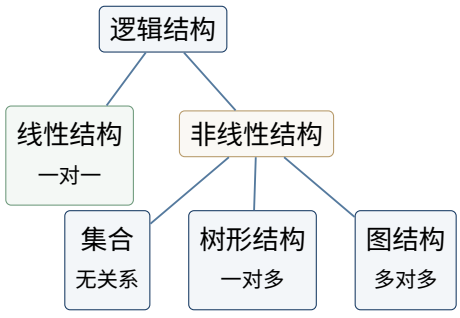


图 10　从逻辑上，数据结构分为线性结构和非线性结构两大类

读图：这是课件第 30 页单选题第 1 题的答案：从逻辑上可以把数据结构分成**线性结构和非线性结构**。除线性结构以外的三种都属于非线性结构。

3.6 最简单的例子

例题 3.1　由二元组画图并判断结构类型

题目　设 $D=\{1,2,3,4,5\}$，$R=\{\langle 1,2\rangle,\langle 1,3\rangle,\langle 3,4\rangle,\langle 3,5\rangle\}$。画出逻辑结构图并指出它属于哪种逻辑结构。若再往 R 中加入 $\langle 2,4\rangle$，结论变不变?

第一步：画图。每个有序对画一条箭头。　**第二步：数前驱**。看每个元素有几个箭头指向它。



原题：每个元素至多 1 个前驱 ⇒ **树形结构**

元素 4 有 2 个前驱（2 和 3）⇒ 不再是树，是**图结构**

前驱个数表

元素	1	2	3	4	5
原题	0	1	1	1	1
加边后	0	1	1	2	1

结果　原题是树形结构（1 是根）；加入 $\langle 2,4\rangle$ 后，4 有两个前驱，变成图结构。

本题想说明什么　判断树还是图，关键看**有没有元素拥有多个前驱**。这也是树形结构“一对多”中“一”字的含义：每个元素上面只有一个。

3.7 数据结构这门课研究哪四件事

课件第 22 页以队列为例，把每一种数据结构的研究内容分成四个方面；第 55、56 页的小结又重复强调了一遍。后面每一章都按这四步走：

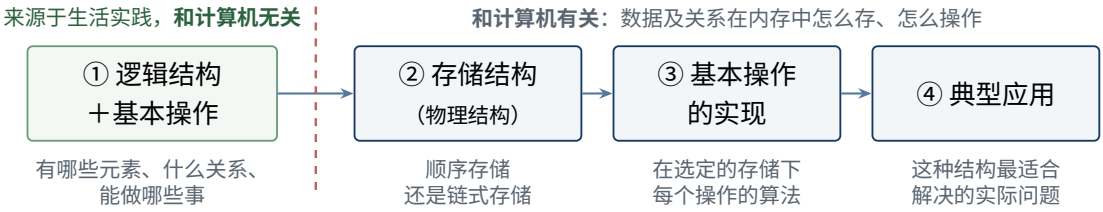
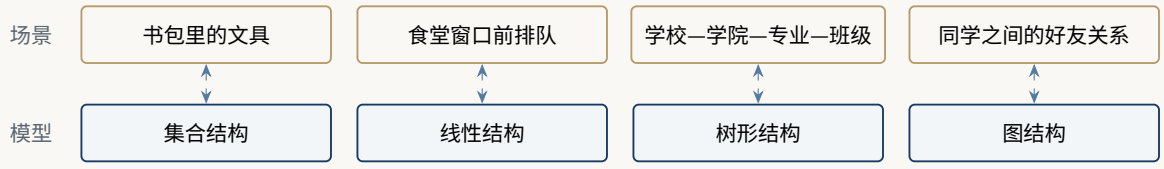


图 11　每一种数据结构都要研究的四个方面；红色虚线左边不涉及计算机，右边才进入内存

读图：红色虚线是本章最重要的一条分界线。线的左边讨论“事物本来的关系”，画的是圆圈和箭头；线的右边讨论“在内存里怎么实现”，画的是内存格子和地址。第 4 节就要跨过这条线。

生活中的它　一天里遇到的四种关系

生活场景与对应关系　四种逻辑结构在校园里都找得到原型：



一次小判断　5 个人排成一队，有 4 个“前后”关系；5 个人两两都是好友，则有 $\dfrac{5\times 4}{2}=10$ 个好友关系。元素一样多，关系的数量和形状完全不同。

边界说明　现实中的关系常常混在一起（班级是树形隶属，班里同学之间又有图状的好友关系）。建模时要先明确：**当前问题关心的是哪一种关系**。

易错点

(1) **把“顺序”“链式”说成逻辑结构。**顺序、链式是**存储结构**（第 4 节）；逻辑结构只有集合、线性、树形、图。这是本章最高频的混淆。

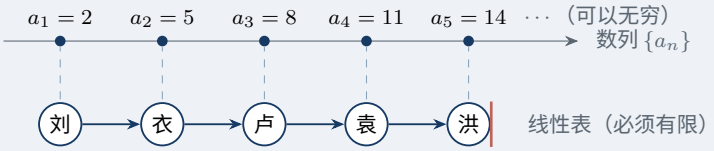
(2) **认为树是“多对多”。**树中每个元素只有一个前驱（上级），所以是“一对多”。一旦某个元素有两个上级，就成了图。

(3) **认为集合结构“没有结构”所以不算数据结构。**它是关系最弱的一种：元素只满足“同属一个集合”，此时 $R=\varnothing$。

(4) **无方向的连线怎么写成有序对？**一条没有箭头的连线 $a-b$ 表示关系是相互的，相当于同时有 $\langle a,b\rangle$ 和 $\langle b,a\rangle$。

对照　线性结构 ↔ 中学的数列（推广型）

一句话定位　数列 $a_1,a_2,\ldots$ 是你最早见到的“线性结构”：每一项有唯一的前一项和后一项。线性结构把“项”从数推广为任意类型的数据元素，同时加上了“有限”的限制。



数列中的结论	迁移结果	理由或反例
除首项外每一项都有唯一的前一项	✓ 直接成立	这正是线性结构“一对一”的定义
每一项可以是任意实数	△ 加条件	元素可以是任意类型（一条学生记录也行），但所有元素必须**类型相同**
数列可以有无穷多项	✗ 不成立	数据结构要求**有限个**元素（图中红色竖线），无穷多项存不进内存

拓展联想：以下内容只用于建立知识联系，当前阶段不要求掌握。

在离散数学或图论中，“图”被定义为 $G=(V,E)$，V 是顶点集，E 是边集——这与本节的 (D,R) 是同一种写法：D 对应 V，R 对应 E。区别在于，图论主要研究图本身的数学性质；数据结构还要继续回答“怎么存进内存”和“操作要花多少时间”。

到这里，我们已经解决了什么？　会用圆圈、箭头和二元组描述元素之间的关系。**下一步：**这些圆圈和箭头只存在于纸上，内存里只有一格一格的存储单元。元素的值好存，“箭头”怎么存？

第 4 节　存储结构：顺序存储与链式存储

4.1　它解决什么问题

定义　存储结构（物理结构）

存储结构是数据结构在内存中的表示。数据要得到处理首先必须进入内存；内存中**不仅要存储数据元素的值，还要存储元素间的关系**。

存值很简单，难点在后半句。保存“谁的后面是谁”有两种基本思路：一种是**让位置说话**——把

相邻的元素挨着放；另一种是**让元素自己说**——每个元素随身带一张纸条，写着下一个元素在哪里。

4.2 先看图：同一个线性结构的两种存法

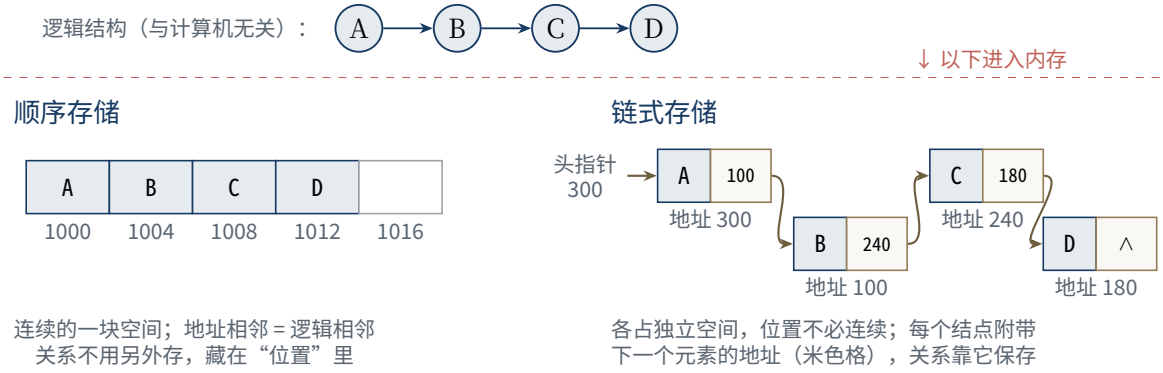


图 12　线性结构 (A, B, C, D) 的顺序存储与链式存储

读图：红色虚线上方是纸面上的逻辑结构。左下：四个元素占用地址 1000、1004、1008、1012 这一段**连续**空间（假设每个元素占 4 个字节），“B 在 A 后面”由“1004 紧跟 1000”体现。右下：四个结点散落在地址 300、100、240、180 处，每个结点分两格——蓝格存元素的值，米格存**后继的地址**；箭头只是把“米格里的地址指向哪里”画了出来；最后一个结点的米格填空标记 ∧，表示没有后继。

定义　两种基本存储方式

顺序存储：用一块**连续**的空间存储数据，借助空间地址上的有序性存储元素间的关系。高级语言中数组是地址连续的空间，有助于实现顺序结构。

链式存储：元素各自存储在**独立**的空间中，不要求不同数据的存储空间连续；元素间的关系附带存储在数据各自占据的独立空间中。

4.3 顺序存储的寻址公式

〔教学补全〕课件只说了“借助地址上的有序性”。把这句话写成公式，就能看清顺序存储最大的优点。

核心公式　顺序存储中第 i 个元素的地址

$$\mathrm{LOC}(a_i) = \mathrm{LOC}(a_0) + i \times d$$

- $\mathrm{LOC}(a_i)$：下标为 i 的元素的地址
- $\mathrm{LOC}(a_0)$：第一个元素的地址（首地址）
- i：下标，从 0 开始
- d：每个元素占用的字节数

公式从哪里来　所有元素类型相同，所以每个元素占的字节数都是 d；元素又是一个挨一个连续存放的。下标为 i 的元素前面恰好有 $a_0, a_1, \ldots, a_{i-1}$ 共 i 个元素，它们一共占 $i \times d$ 个字节，所以 a_i 的起始地址就是首地址再往后 $i \times d$。

使用条件：空间连续；每个元素大小相同；下标从 0 开始（若从 1 开始编号，公式变为 $\mathrm{LOC}(a_1) + (i-1) \times d$）。

它意味着什么：无论 i 是多少，算一次乘法和一次加法就能得到地址——这叫**随机访问**，与元素个数 n 无关。笔记里“$O(1)$：数组随机访问”说的就是这件事。

链式存储没有这样的公式：结点的地址是程序运行时临时分配的，彼此没有规律，要找第 i 个

元素，只能从头指针出发，沿着“下一个的地址”一个一个走过去。

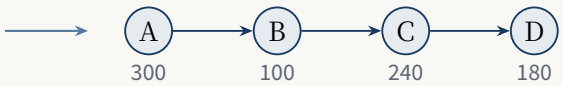
例题　4.1　两种存储各算一次

（1）顺序存储　`int a[5]` 的首地址为 2000，每个 `int` 占 4 个字节。求 `a[3]` 的地址。

解　$\mathrm{LOC}(a_3) = 2000 + 3 \times 4 = 2012$。

（2）链式存储　内存中有如下 4 个结点，头指针为 300。按逻辑顺序写出各元素。

结点地址	100	180	240	300
元素值	B	D	C	A
后继地址	240	∧	180	100



解　从头指针 300 出发：地址 300 存的是 A，后继在 100；地址 100 存的是 B，后继在 240；地址 240 存的是 C，后继在 180；地址 180 存的是 D，后继为空。所以逻辑顺序是 **A, B, C, D**。

检查　按地址从小到大读是 B, D, C, A，与逻辑顺序完全不同。

本题想说明什么　顺序存储“算”出位置；链式存储“走”到位置。链式存储中，**物理位置与逻辑顺序无关**。

4.4　逻辑结构、存储结构与操作的三句话

课件第 27、29 页用三句话说明了它们的关系：

(1) 基本操作的**设计**取决于逻辑结构——队列该有“入队”“出队”，这是由“先来先服务”的关系决定的，与怎么存无关；

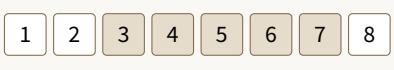
(2) 基本操作的**实现**完全依赖存储结构——同样是“取第 i 个元素”，顺序存储用公式算，链式存储要顺着指针走；

(3) 选哪种存储，原则是**有利于基本操作的实现**——如果某种存储方式下基本操作不易实现，说明这种存储方式不好，可以考虑放弃。

比较维度	顺序存储	链式存储
空间要求	一整块连续空间	零散空间即可
关系怎么保存	隐含在地址的相邻性里	每个结点额外存后继地址
额外空间开销	无	每个结点多一个地址域
按下标取元素	用公式直接算，$O(1)$	从头走，最坏 $O(n)$
在中间插入或删除	要移动后面的元素（第 2 章详述）	改几个地址即可（第 2 章详述）

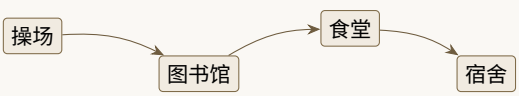
生活中的它　电影院连座与寻宝纸条

生活场景　5 个同学看电影买了连座；另一次班级活动玩“寻宝”，每个藏宝点放一张纸条，写着下一个藏宝点在哪里。



7 排 3 号起的 5 个连座

座位号 ↔ 地址　连座 ↔ 顺序存储



每处纸条写着“下一站在哪”

纸条 ↔ 后继地址　寻宝路线 ↔ 链式存储

一次小判断　连座从 3 号开始，第 4 位同学（从第 0 位数起下标为 3）坐 $3 + 3 \times 1 = 6$ 号，直

接算出，不用问别人。寻宝要找到第 4 个藏宝点，必须先依次找到前 3 个。
边界说明　座位每人占 1 个号；内存里每个元素占 d 个字节，所以公式里多了“$\times d$”。

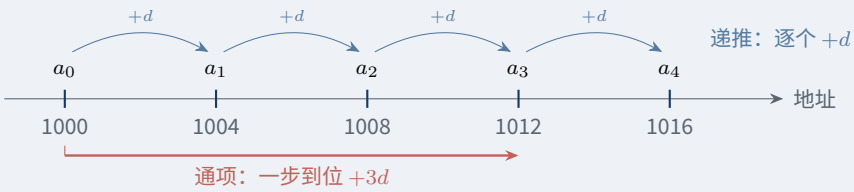
易错点

(1) **“逻辑上相邻的元素，物理上也一定相邻。”**只对顺序存储成立。链式存储中逻辑相邻的元素可以相隔很远（例 4.1）。
(2) **“顺序存储就是线性结构。”**两个词不在同一层面：线性是逻辑结构，顺序是存储结构。线性结构既可以顺序存储，也可以链式存储。
(3) **“链式存储更省空间。”**不一定。它不需要整块连续空间，但每个结点要多花一个地址域。
(4) **忘了下标从 0 开始。**a_3 前面有 3 个元素（a_0, a_1, a_2），所以偏移量是 $3d$ 而不是 $4d$。

对照　顺序 / 链式访问 ↔ 数列的通项公式 / 递推公式（类比同构型）

一句话定位　顺序存储的寻址公式就是一个**等差数列的通项公式**；链式存储“只知道下一个在哪”则像只给了**递推关系**。

比较维度	数列	存储结构
“直接算”的方式	通项公式 $a_n = a_1 + (n-1)d$	顺序存储 $\mathrm{LOC}(a_i) = \mathrm{LOC}(a_0) + i\,d$
“一步步推”的方式	递推公式 $a_{n+1} = f(a_n)$，求 a_n 要从 a_1 推起	链式存储：由当前结点得到下一个结点的地址，要从头结点走起
求第 n 项的代价	通项 1 步；递推 $n-1$ 步	顺序 $O(1)$；链式 $O(n)$
公差 / 步长	d 是相邻两项的差	d 是每个元素占的字节数



同一个小算例　首项（首地址）1000，公差（元素大小）4，求下标为 3 的那一项：通项公式一步得到 $1000 + 3 \times 4 = 1012$；递推要算 $1000 \to 1004 \to 1008 \to 1012$ 三步。

数列中的结论	迁移结果	理由或反例
有了通项公式，任意一项都能直接算	✓ 直接成立	顺序存储可以直接算出任一元素的地址
通项公式写作 $a_1 + (n-1)d$	△ 加条件	C++ 下标从 0 开始，偏移量是 $i\,d$；从 1 开始编号才是 $(i-1)d$
递推数列常常能化出通项，所以链表也能“化出”寻址公式	✗ 不成立	结点地址由系统在运行时分配，彼此没有任何规律（例 4.1 中是 300、100、240、180），无公式可化

到这里，我们已经解决了什么？　知道了关系进入内存的两种办法。**下一步：**数据存好以后要拿

来用。对一种数据结构“能做哪些事”，能不能先规定清楚，而不必关心它内部到底是数组还是链表?

第 5 节　基本操作与抽象数据类型

5.1　基本操作的五大类

课件第 25、26 页指出：基本操作来源于现实生活中关系自身的特点；无论哪种逻辑结构，基本操作都可分为五大类。下图以“队列”为例。

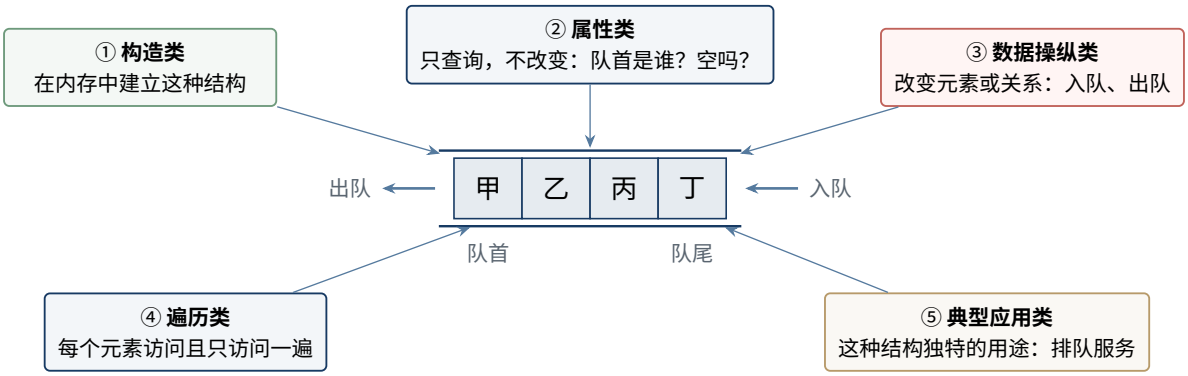


图 13　基本操作的五大类（以队列为例）

读图：中间是一个队列：左端出队，右端入队。周围五个框是五类操作。区分第 ② 类和第 ③ 类的标准只有一条：**操作之后，元素或元素之间的关系变没变**。课件对第 ② 类的形容是“东瞧瞧、西看看”。

〔教学补全〕两点补充说明：其一，“修改”可以看作在同一位置先删除一个旧元素再插入一个新元素，所以课件不再单独讨论。其二，遍历本来也“只看不改”，可归入属性类；但它很重要，又是许多其他操作的基础（尤其在树和图中并不简单），所以单列为一类。

5.2　抽象数据类型（ADT）

定义　抽象数据类型（Abstract Data Type，ADT）

抽象数据类型是指一个**数学模型**以及定义在此数学模型上的**一组操作**。它有两个特性：

抽象性：用 ADT 描述程序处理的实体时，强调的是其本质特征、所能完成的功能以及它和外部用户的接口（即外界使用它的方法）。

封装性：将实体的外部特性和其内部实现细节分离，并且对外部用户隐藏其内部实现细节。

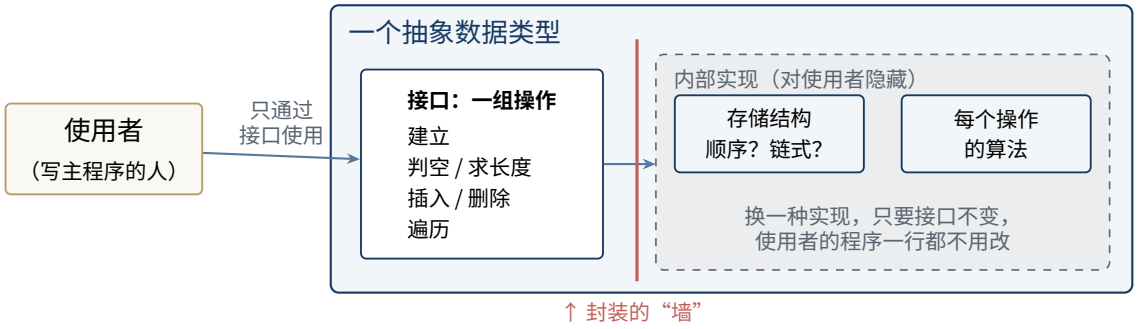


图 14　ADT 的两个特性：左半边“只说能做什么”是抽象性，红线“把怎么做藏起来”是封装性

读图：使用者只能看到白色方框里的操作名称（接口）；红线右侧灰色区域里用数组还是链表、每个操作具体怎么写，使用者看不到，也不需要看。**抽象性**回答“它是什么、能做什么”；**封装性**保证“内部怎么做”可以随时更换而不影响使用者。

〔教学补全〕通行教材常把 ADT 写成三部分：**数据对象**（有哪些元素）、**数据关系**（元素间是什么关系）、**基本操作**（能做什么）。前两部分就是第 3 节的 (D, R)，所以也可以记成：**ADT = 逻辑结构 + 基本操作**——恰好是图 11 红色虚线左边的全部内容。课件小结（第 55 页）说的“最后给出用伪代码描述的抽象数据类型”，样子如下（以队列为例）：

```
ADT Queue {
  数据对象：  D = { a1, a2, ..., an } ，各元素类型相同
  数据关系：  R = { <a1,a2>, <a2,a3>, ..., <a(n-1),an> } ，  a1 为队首，  an 为队尾
  基本操作:
    InitQueue()     构造一个空队列                  —— 构造类
    QueueEmpty()    判断队列是否为空                —— 属性类
    GetHead()       读取队首元素（队列非空时）      —— 属性类
    EnQueue(x)      在队尾插入元素 x                —— 数据操纵类
    DeQueue()       删除队首元素（队列非空时）      —— 数据操纵类
    Traverse()      依次访问每个元素一次            —— 遍历类
}
```

5.3 ADT 在 C++ 里怎么落地

课件第 34 页说：抽象数据类型需要通过固有数据类型（高级语言中已实现的数据类型）来实现，本书由**类模板或类**来实现。第 35 页给出了顺序表的类模板 SqList，下面是其主要部分（参数名已统一为 ElemType）：

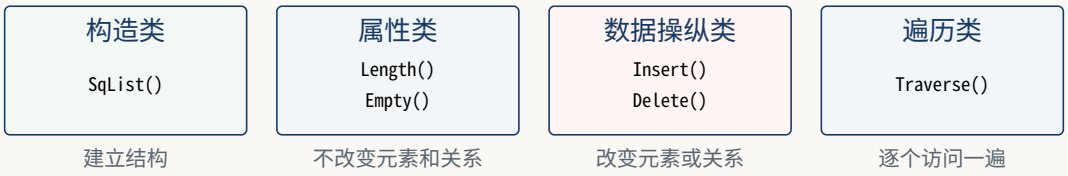
```
template <class ElemType>
class SqList {
protected:                          // —— 内部实现：对使用者隐藏 ——
    int count;                      // 元素个数
    int maxSize;                    // 顺序表最大元素个数
    ElemType *elems;                // 元素存储空间（一块连续空间：顺序存储）
public:                             // —— 接口：使用者能调用的操作 ——
    SqList(int size = DEFAULT_SIZE);                  // 构造类
    virtual ~SqList();
    int  Length() const;                              // 属性类：求长度
    bool Empty() const;                               // 属性类：判空
    bool GetElem(int position, ElemType &e) const;    // 属性类：取指定位置元素
    void Traverse(void (*visit)(const ElemType &)) const;   // 遍历类
    bool SetElem(int position, const ElemType &e);    // 数据操纵类
    bool Insert(int position, const ElemType &e);     // 数据操纵类：插入
    bool Delete(int position, ElemType &e);           // 数据操纵类：删除
    void Clear();                                     // 数据操纵类：清空
};
```

每一段在干什么　protected 部分的三个数据成员说明这个实现选用了顺序存储（elems 指向一块连续空间），它们在类外不能直接访问，这就是封装；public 部分的成员函数是接口，逐个对应 ADT 的基本操作。函数声明里出现的 template、const、& 是 C++ 的语法工具，第 12 节集中复习。

例题　5.1　给操作归类

题目　把类模板 SqList 的下列成员函数归入五大类：SqList()、Length()、Empty()、Insert()、Delete()、Traverse()。

思路　对每个操作问两个问题：它是在“建立”结构吗？它执行后元素或关系变了吗？

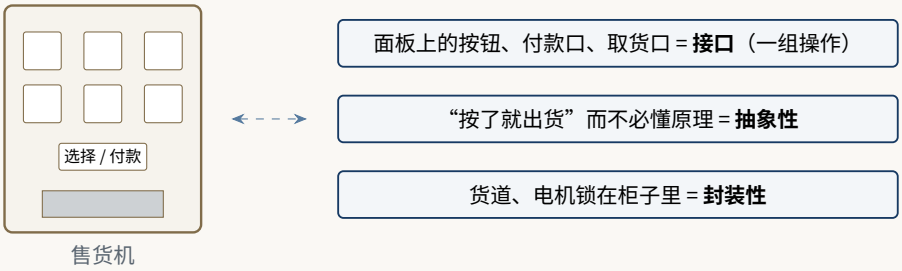


检查　属性类和遍历类函数的声明末尾都带 const——C++ 用它保证“这个函数不会修改对象”，与分类标准正好吻合。

本题想说明什么　五大类不是凭空的分类，它直接体现在代码的写法上。

生活中的它　自动售货机

生活场景　在自动售货机上买水：选商品、付款、取货，从不需要打开机柜。



一次小判断　厂家把内部的弹簧货道换成了传送带，顾客的买水步骤需要改变吗？**不需要**——接口没变。这对应“把顺序存储换成链式存储，使用者的程序不用改”。

边界说明　真机会遇到缺货、卡币。对应到程序里，就是操作的前提条件不满足时该怎么办——第 12 节的“异常处理”专门处理这类情况。

易错点

(1) **“ADT 就是 C++ 的类。”** ADT 是**规格说明**（只说有什么、能做什么），类是**实现手段**。同一个 ADT 可以用顺序存储实现一个类，再用链式存储实现另一个类。

(2) **“抽象就是说得模糊。”** 恰恰相反：抽象是**精确地**规定对外的行为，同时不规定内部做法。

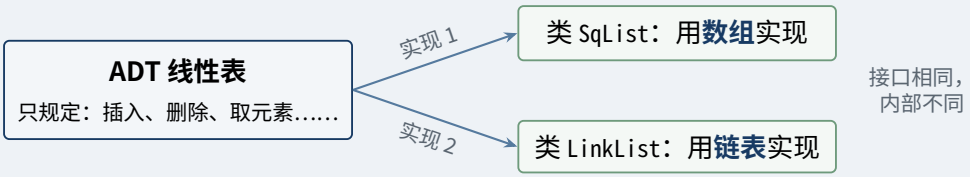
(3) **把“属性类”和“数据操纵类”分错。** 判断标准只有一条：操作之后元素或关系有没有变化。GetElem 只读，属于属性类；SetElem 改值，属于数据操纵类。

相近概念辨析　这三个词都带“数据”，层次不同：

概念	它是什么	例子
数据类型	值的集合 + 这些值上的操作；由语言**已经实现**	int、double、char
抽象数据类型	数学模型 + 一组操作；**只定义，不规定实现**	“队列”“线性表”这种说法
数据结构	有关系的数据元素的集合；完整研究要包括逻辑结构、存储结构、操作实现	顺序表、链表

对照　抽象数据类型 ↔ C++ 的类（思想重现型）

一句话定位　类是 ADT 思想在 C++ 中的落地方式：public 成员函数对应“一组操作”，private/protected 部分对应“被隐藏的实现”。



从 C++ 类带来的认识	迁移结果	理由或反例
public 成员函数就是 ADT 的基本操作	✓ 直接成立	二者都是“对外提供的使用方法”
类里的数据成员就是 ADT 的数据对象	△ 加条件	ADT 只说“有哪些元素、什么关系”，不说用 elems 数组还是指针；数据成员已经是某一种存储方案
一个类名对应一种实现，所以一个 ADT 也只有一种实现	✗ 不成立	上图：线性表这一个 ADT 至少有顺序、链式两种实现（第 2 章）

学完第一部分，你应该能够回答

(1) 学生表里，“一行”和“一格”分别叫什么？哪一个是最小单位？
(2) 四种逻辑结构各是“几对几”？怎样通过数前驱个数区分树和图？
(3) 二元组 (D, R) 中的 D 和 R 各是什么？R 的元素长什么样？
(4) “顺序”和“线性”哪个是存储结构、哪个是逻辑结构？
(5) 顺序存储靠什么保存元素间的关系？链式存储呢？
(6) 为什么顺序存储可以随机访问？写出寻址公式。
(7) ADT 的两个特性各指什么？“求长度”和“插入”分别属于哪类基本操作？

到这里，我们已经解决了什么？　数据怎么组织的问题告一段落：能描述关系、能把关系放进内存、能规定对它做哪些操作。**下一步：**每个操作都要靠一系列步骤来完成，这些步骤就是**算法**。同一个操作往往有好几种算法，怎么判断哪个更好？这是第二部分的任务。

第二部分　算法好不好：从“数次数”到“看增长”

第 6 节　算法及其特性

6.1　它解决什么问题

第一部分结束时留下的问题是：每个基本操作都要靠一系列步骤完成。这些步骤不能随便写——计算机不会“意会”，也不会“差不多就行”。所以要先说清楚：什么样的一组步骤才配叫**算法**，什么样的算法才算**好**算法。

定义　算法

算法是解决一个具体问题的方法和步骤。（笔记中的表述：求解具体问题的步骤描述；落实到代码上，是一组**有限的指令序列**。）

6.2　先看图：算法的五个特性

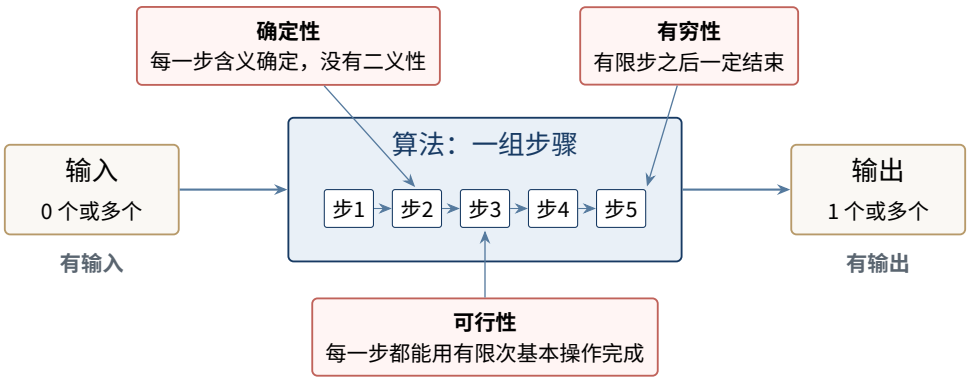


图 15　算法的五个特性：两个管“两头”（输入、输出），三个管“中间的步骤”

读图：中间蓝框是一串步骤。左右两个米色框是输入和输出；三个红框分别约束步骤本身：每一步说得清（确定性）、做得到（可行性）、整体停得下（有穷性）。

〔教学补全〕课件第 37 页只给了每个特性的一句话说明。为每个特性配一个“不满足时是什么样”的反例，才算真正理解：

特性	含义（课件）	反例：不满足时
确定性	每一步有确定的含义，没有二义性	“把 x 加上一个比较大的数”——多大算比较大?
有穷性	每一步在有限时间内完成，整个算法在有限步之后完成	`i=1; while (i>0) i++;`（不考虑溢出时）永远不停
可行性	每一步都能经过有限次基本操作完成	“写出 π 的全部小数位”——位数无穷，做不到
有输入	有零个或多个输入，作为已知条件	零个输入是允许的：如“输出前 10 个偶数”不需要输入
有输出	有**一个或多个**输出，作为运行结果	算完什么都不给出，等于没算（见原图纠错）

6.3 好算法的五条基本要求

满足五个特性只说明“它是算法”。课件第 38 页又提出了“好算法”的五条要求：



图 16　特性与要求的分工：特性是“门槛”，要求是“评分标准”

读图：左框是门槛，少一条就不叫算法；右框是评分标准，做得越好算法越好。其中**时间效率**和**空间效率**可以定量计算，是第 7–11 节的全部内容。

相近概念辨析　特性和要求中有几对很容易混：

容易混的一对	区别
有穷性 ↔ 时间效率	有穷性只要求“能停”；时间效率关心“多久停”。一个要跑 100 年才停的算法满足有穷性，但时间效率极差
确定性 ↔ 正确性	确定性说“每一步怎么做是明确的”；正确性说“最后结果对不对”。步骤明确但思路错了，仍然不正确
正确性 ↔ 健壮性	正确性针对**合法**输入给出正确结果；健壮性针对**不合法**输入（如下标越界）也能合理应对而不是崩溃

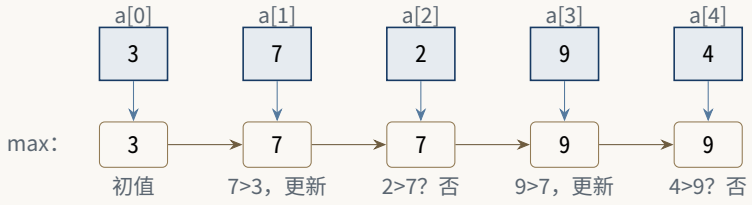
6.4 设计算法要用“计算思维”

课件第 39 页提醒了一个误区：**从以往生活经验出发找解决办法**。人从一堆桔子里挑最大的，扫一眼就行；计算机没有“扫一眼”这个动作，它每次只能做输入、输出、赋值、**两两比较**、算术和逻辑运算、循环、函数调用这些基本动作。正确的方式是从这些基本动作出发来设计——这叫计算思维。

例题　6.1　用“两两比较”找最大值

题目　数组 a = {3, 7, 2, 9, 4}，按下面的算法求最大值。跟踪变量 max 的变化，并数一数做了几次比较。

```
max = a[0];
for (i = 1; i < n; i++)
    if (a[i] > max) max = a[i];      // 两两比较：当前元素 与 目前为止的最大值
```

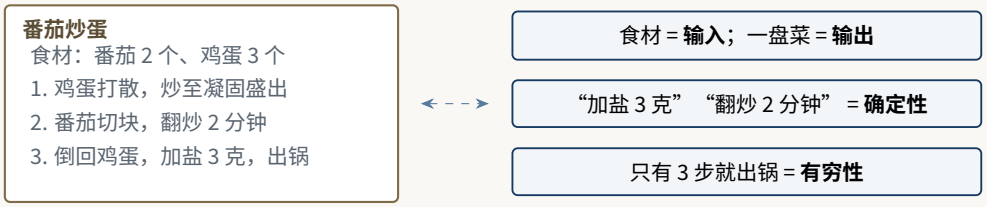


结果　最大值为 9。比较发生在 $i = 1, 2, 3, 4$，共 $n - 1 = 4$ 次。

本题想说明什么　人“一眼看出”的事，计算机要拆成 $n - 1$ 次两两比较。以后分析算法快慢，数的就是这种基本动作的次数。

生活中的它　一张菜谱

生活场景　照着菜谱做番茄炒蛋。



一次小判断　下面两句菜谱各违反了哪个特性？“加盐少许”——违反**确定性**（少许是多少）；“一直搅拌”——没有停止条件，违反**有穷性**。

边界说明　人能靠经验把“少许”补全成合适的量，计算机不能。这正是算法比菜谱要求更严格的原因。

易错点

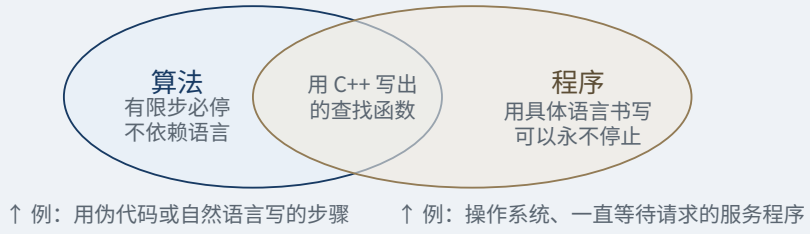
(1) **“算法必须有输入。”**不对，输入可以是零个；但输出至少一个。

(2) **“能运行的程序都是算法。”**不对，见下面的对照。

(3) **把“有穷”理解成“很快”**。有穷只是“有限步”，可以是非常大的有限数。快不快由时间复杂度回答。

对照　算法 ↔ 程序（同名异义型）

一句话定位　程序是算法用某种编程语言写出来的一种具体实现；但并不是每个程序都满足算法的全部特性。



从 C++ 程序带来的认识	迁移结果	理由或反例
每条语句的含义都是确定的	✓ 直接成立	对应算法的确定性
写程序必须选定一种语言，所以算法也必须用 C++ 写	✗ 不成立	算法可以用自然语言、流程图、伪代码描述；同一算法可以写成多种语言的程序
程序运行起来总会结束	✗ 不成立	操作系统开机后一直运行、等待用户操作，它是程序，但不满足有穷性，所以不是算法
程序写对了，算法就是好的	△ 加条件	正确只是五条要求之一，还要看可读性、健壮性、时间效率、空间效率

到这里，我们已经解决了什么？　知道了什么是算法、好算法看哪五条。**下一步：**“时间效率”怎么量？拿秒表掐时间行不行？

第 7 节　时间频度：先学会“数次数”

7.1　为什么不直接用秒表

课件第 40 页给出了两种度量方法。**运行后度量**：把不同算法写成程序，用同样的测试数据实际运行、计时、比较。**运行前分析**：算法设计好之后、程序运行之前，根据影响因素直接分析执行时间。

秒表计时的问题在于：算出来的时间里混进了太多与算法本身无关的东西。课件第 41 页列出了影响运行时间的五个因素：

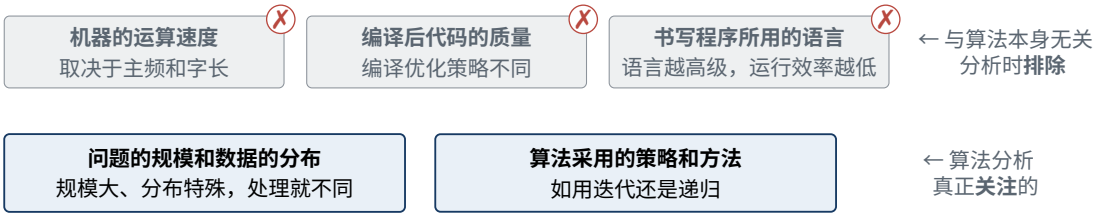


图 17　影响程序运行时间的五个因素：上面三个被排除，下面两个才是算法分析的对象

读图：灰色、打叉的三项换一台电脑、换一种语言就会变，无法反映算法的优劣。留下的两项中，“策略和方法”正是算法本身；“问题规模”用一个数 n 表示。于是目标变成：**把运行时间表示成 n 的函数。**

7.2　正式定义

定义　基本语句与时间频度

用算法中**标准操作**（即**基本语句**）的执行次数来度量运行时间。基本语句执行次数越多，时间花费越多；这个执行次数称为**时间频度**。时间频度与处理的数据规模 n 有关，可表示为 n 的函数，记作 $T(n)$。

符号说明　n：问题规模（要处理的数据个数），正整数；$T(n)$：规模为 n 时基本语句执行的总次数，是一个随 n 变化的非负整数。**隐含假设：**每条基本语句执行一次所花的时间大致相同（都当作 1 个单位）。

7.3　先看图：逐句数一遍

课件第 42 页的例子，是全章最重要的一次“数次数”：

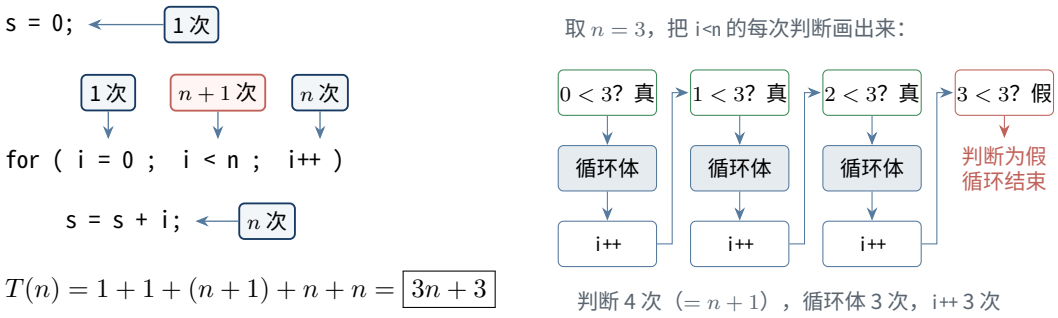


图 18　逐句统计执行次数；右侧取 $n = 3$ 说明循环条件为什么多判断一次

读图：左半边：每个方框标出它所指的那部分代码执行了多少次。红框是最容易数错的地方——循环条件 i<n 执行 $n + 1$ 次。右半边：$n = 3$ 时，条件在 $i = 0, 1, 2$ 时为真（各进入一次循环体），在 $i = 3$ 时为假（退出），所以一共判断了 4 次。

公式从哪里来　把五个部分的次数相加：`s=0` 执行 1 次；`i=0` 执行 1 次；`i<n` 执行 $n+1$ 次（n 次为真、最后 1 次为假）；`i++` 执行 n 次；`s=s+i` 执行 n 次。于是

$$T(n) = 1 + 1 + (n+1) + n + n = 3n + 3.$$

7.4 度量原则

课件第 43 页给出两条原则：

(1) **循环语句**：要计算**实际运行**的次数，不是看语句书写的次数。`s=s+i;` 只写了一行，却执行 n 次。

(2) **分支语句**：按照执行语句**多的那个分支**计算。因为分析的目的是给出时间的保证，所以取代价大的情况。

课件的例子（已按原图纠错统一口径）

```
if (n > 0)                                  // 判断 1 次
    { for (i = 0; i < n; i++)  cout << i; } // n>0 时: 1 + (n+1) + n + n = 3n+2 次
else
    cout << "n<=0!";                        // n<=0 时: 1 次
```

两个分支分别要 $3n+2$ 次和 1 次，取多的；再加上 `if` 的 1 次判断，时间频度为 $T(n) = 3n + 3$。

7.5 一个省力的简化：只数循环体

逐句数很准确，但也很繁琐。课件第 46 页给出了两条简化规则：

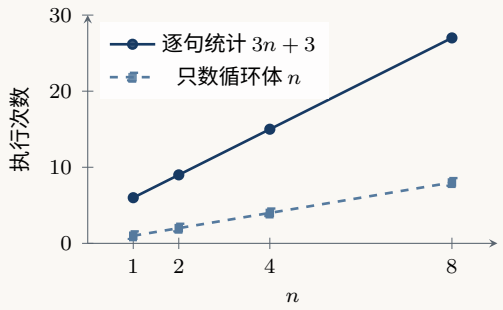
(1) 执行次数与 n 无关的语句（如 `s=0`），可以不计入；

(2) 循环控制条件、循环变量的变化次数，与循环体的执行次数**接近一致**，所以**只需要看循环体的执行次数**。

对上面的例子，只数循环体得到 n 次，逐句数得到 $3n+3$ 次。两个结果差了约 3 倍外加一个常数——第 8 节会证明：这种差别在比较算法时**可以忽略**。

例题　7.1　两种数法对照

题目　对课件第 42 页的程序段，分别取 $n = 1, 2, 4, 8$，列出“逐句统计”的 $T(n) = 3n + 3$ 和“只数循环体”的次数 n，观察 n 翻倍时各自怎样变化。



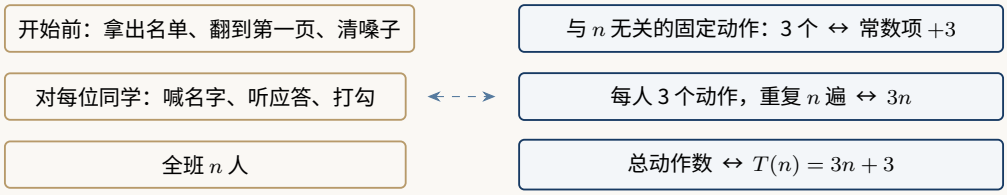
n	1	2	4	8
$3n+3$	6	9	15	27
n	1	2	4	8
比值	6	4.5	3.75	3.375

观察　两条线都是直线，n 翻倍时两者都**大约翻倍**；二者的比值随 n 增大越来越接近 3。

本题想说明什么　两种数法只差一个常数倍，增长的“形状”相同。这为下一节“只看增长的阶”做了铺垫。

生活中的它　老师点名

生活场景　上课前老师按名单点名。



一次小估算　40 人的班：$3\times 40+3=123$ 个动作；80 人的合班：243 个动作，约为原来的 2 倍。

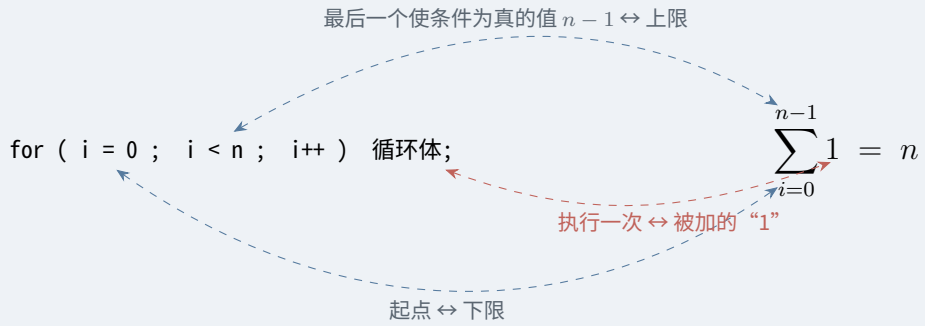
边界说明　现实中有人迟到要重点、应答有快有慢。模型假设每个基本动作耗时相同，这正是“把基本语句都当作 1 个单位”的假设。

易错点

(1) **把循环条件 i<n 数成 n 次**。它要多判断一次“假”才能退出，是 $n+1$ 次。
(2) **按“写了几行”来数**。要数的是实际执行的次数。
(3) **分支语句把两个分支的次数相加**。每次只会走一个分支，应取较多的那个。
(4) **以为“只数循环体”和“逐句数”会得到不同的复杂度**。二者只差常数倍和低次项，写成大 O 后完全相同（第 8 节）。

对照　数循环次数 ↔ 数列求和（思想重现型）

一句话定位　数一个循环的执行次数，就是在做一次数列求和：循环变量对应求和下标，循环体每执行一次对应“加 1”。



数列求和中的结论	迁移结果	理由或反例
$1+2+\cdots+n=\dfrac{n(n+1)}{2}$	✓ 直接成立	内层次数随外层变量变化的双重循环正好用它（第 10 节）
求和的上下限照着公式写就行	△ 加条件	上下限必须与循环的初值、终止条件**逐一对应**。课件第 47 页的求和式就没对上代码（见原图纠错）
求和的项数等于上限	✗ 不成立	项数 = 上限 − 下限 +1。$\sum_{i=0}^{n-1}$ 有 n 项，$\sum_{i=1}^{n-1}$ 只有 $n-1$ 项

到这里，我们已经解决了什么？　会把运行时间写成 $T(n)$。**下一步：**$3n+3$ 与 n 到底算不算“一样

快”？$3n^2+2n+10$ 与 $100n$ 谁更快？需要一个能忽略枝节、只比较“增长趋势”的记号。

第 8 节　大 O 表示法：只看增长的阶

8.1　它解决什么问题

笔记里有一句被红笔圈出的话：算法复杂度衡量的是，随着数据规模 n 的增长，算法执行所花费的时间和空间的**增长速度**。注意是“增长速度”，不是“花了多少”。$T(n)=3n+3$ 和 $T(n)=n$ 具体次数不同，但 n 翻倍时都大约翻倍；而 $T(n)=n^2$ 在 n 翻倍时会变成 4 倍。大 O 表示法要抓住的就是这种差别。

8.2　先看图：什么叫“被 $C\cdot f(n)$ 压住”

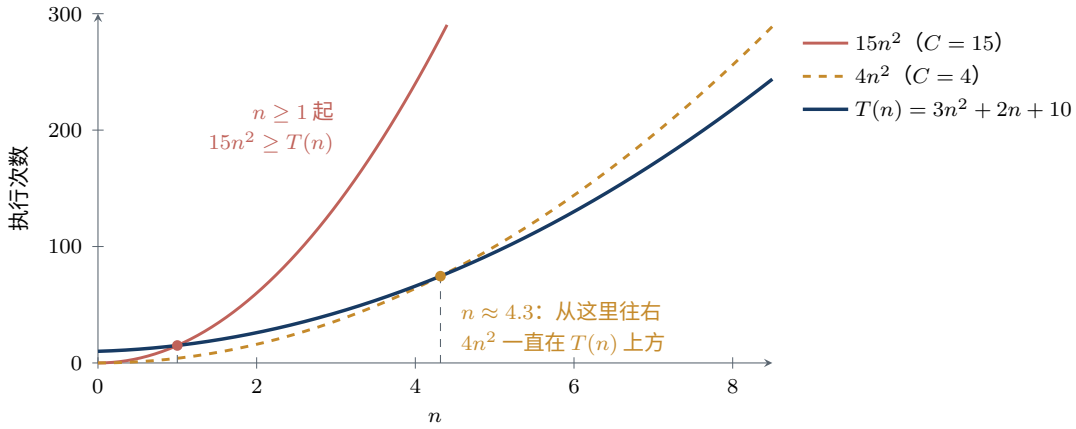


图 19　$T(n)=3n^2+2n+10$ 被 $C\cdot n^2$ 从上方“压住”：C 和 N_0 的取法不唯一

读图：横轴是规模 n，纵轴是执行次数。蓝色实线是 $T(n)$。红线 $15n^2$ 从 $n=1$ 起就一直在蓝线上方；橙色虚线 $4n^2$ 开始时在蓝线下方，过了 $n\approx 4.3$ 之后也一直在上方。两条线都说明同一件事：n 足够大以后，$T(n)$ 不会超过 n^2 的某个常数倍。这就是 $T(n)=O(n^2)$ 的含义。

8.3　正式定义

定义　大 O 表示法（渐进时间复杂度）

设 $T(n)$ 和 $f(n)$ 是定义在正整数上的非负函数。如果**存在**正常数 C 和 N_0，使得**对所有** $n>N_0$ 都有

$$T(n)\le C\cdot f(n),$$

则称 $O(f(n))$ 为算法的**渐进时间复杂度**，简称**时间复杂度**，记作 $T(n)=O(f(n))$。

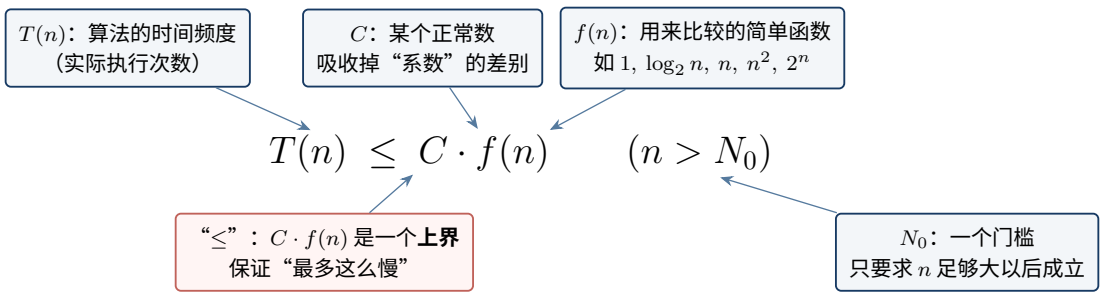


图 20　大 O 定义式中每个符号的含义

读图：定义里有两个“自由度”：常数 C 允许我们不计较系数（$3n$ 还是 n）；门槛 N_0 允许我们不计较 n 较小时的情况。把这两样都放开之后，剩下的就只有“增长的阶”。

8.4 最高次项、去系数：这条规则从哪里来

课件第 45、48 页的结论是：**时间复杂度由时间频度函数中的最高次项决定，不带系数。**课件用两个例子说明，这里把一般情形的证明补全。

课件例 1　$T(n)=3n^2+2n+10$。当 $n\ge 1$ 时，$n\le n^2$ 且 $1\le n^2$，所以

$$\begin{aligned} T(n) &= 3n^2+2n+10 \\ &\le 3n^2+2n^2+10n^2 \qquad \text{（把 } n \text{ 和 } 1 \text{ 都放大成 } n^2\text{）} \\ &= 15n^2. \end{aligned}$$

取 $C=15,\ N_0=1$，定义得到满足，故 $T(n)=O(n^2)$。

课件例 2　$T(n)=3n+3\le 3n+3n=6n$（$n\ge 1$），取 $C=6,\ N_0=1$，故 $T(n)=O(n)$。

定理　多项式的大 O（教学补全）

若 $T(n)=a_kn^k+a_{k-1}n^{k-1}+\cdots+a_1n+a_0$，其中 $k\ge 0$ 为整数、$a_k>0$，则 $T(n)=O(n^k)$。

证明　**要证什么：**存在正常数 C, N_0，使 $n>N_0$ 时 $T(n)\le C\,n^k$。
思路：像例 1 一样，把每个低次项都放大成 n^k。
证明：当 $n\ge 1$ 时，对每个 $0\le j\le k$ 都有 $n^j\le n^k$。又因为 $a_j\le |a_j|$，所以

$$T(n)=\sum_{j=0}^{k}a_jn^j\le\sum_{j=0}^{k}|a_j|\,n^j\le\Big(\sum_{j=0}^{k}|a_j|\Big)n^k.$$

取 $C=|a_k|+|a_{k-1}|+\cdots+|a_0|$，$N_0=1$ 即可。 ■
直观解释：低次项长得慢，n 大了以后都“被最高次项淹没”；系数则被常数 C 吸收。例 1 中 $C=3+2+10=15$ 正是这样来的。
特例：$k=0$ 时 $T(n)=a_0$ 是常数，得到 $O(n^0)=O(1)$——算法中没有与 n 有关的语句时，时间复杂度为 $O(1)$（课件第 48 页的“特殊地”）。

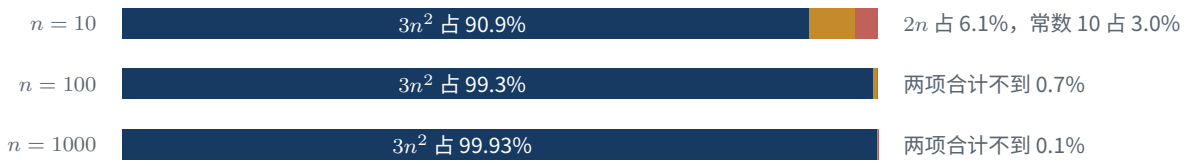


图 21　$T(n)=3n^2+2n+10$ 中三项各自的占比：n 越大，低次项越无足轻重

读图：每一条横条代表 $T(n)$ 的 100%，蓝色是最高次项 $3n^2$ 所占的份额，橙色和红色是 $2n$ 和常数 10。$n=10$ 时蓝色已占 90.9%，$n=1000$ 时占 99.93%——这就是“只看最高次项”的数值依据。

求时间复杂度的两步法（课件第 48 页）

第一步　找出算法中与数据规模 n 有关的循环语句，计算**循环体的执行次数**，得到时间频度函数 $T(n)$。
第二步　观察 $T(n)$ 中关于 n 的**最高次项**，**去掉其系数**，即得时间复杂度的大 O 表示。

特殊情况　若算法中没有执行次数与 n 相关的语句，即时间频度是常量，则时间复杂度为 $O(1)$。

8.5 课件里“极限”的那句话

课件第 44 页还有一句：“当 n 趋于无穷时，$T(n)/f(n)$ 的极限为一个非零常数，称 $O(f(n))$ 为算法的时间复杂度”。它与上面的定义是什么关系？

定理　极限判别（教学补全）

设 $f(n)>0$。若 $\lim\limits_{n\to\infty}\dfrac{T(n)}{f(n)}=c$，其中 $0<c<+\infty$，则 $T(n)=O(f(n))$。

证明　**思路：**极限为 c 说明 n 很大时比值离 c 很近，自然不会超过 $2c$。

证明：由数列极限的定义，对 $\varepsilon=c>0$，存在 N_0，当 $n>N_0$ 时 $\left|\dfrac{T(n)}{f(n)}-c\right|<c$，于是 $\dfrac{T(n)}{f(n)}<2c$，即 $T(n)<2c\cdot f(n)$。取 $C=2c$，大 O 的定义得到满足。 ■

反过来不成立：$T(n)=n$，$f(n)=n^2$。当 $n\ge 1$ 时 $n\le n^2$，所以按定义 $n=O(n^2)$ 成立；但 $\lim\limits_{n\to\infty}\dfrac{n}{n^2}=0$，不是非零常数。

怎么理解这两种说法　“极限是非零常数”是一个**更强**的条件：它说 $T(n)$ 与 $f(n)$ 增长得**一样快**（高等数学里叫“同阶”）。大 O 的定义只要求 $T(n)$ 增长得**不比** $f(n)$ **快**。按定义，对 $T(n)=3n+3$ 写 $O(n)$、$O(n^2)$ 甚至 $O(2^n)$ 都不算错，但后两个太松，没有信息量。**本课程的约定：写时间复杂度时，总是取最紧的那个**——也就是课件用极限描述的那个。做题时按“两步法”得到的就是它。

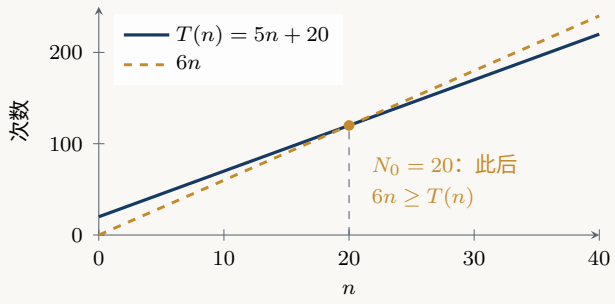
例题　8.1　写出大 O 并用定义验证

题目　写出下列时间频度的时间复杂度：（a）$T(n)=5n+20$；（b）$T(n)=\dfrac{n(n-1)}{2}$；（c）$T(n)=100$；（d）$T(n)=2n+3\log_2 n$。对（a）用定义找出一组 C、N_0。

解　（a）最高次项 $5n$，去系数，$O(n)$。验证：要 $5n+20\le 6n$，只需 $n\ge 20$，取 $C=6$，$N_0=20$（取 $C=25$，$N_0=1$ 也可以，答案不唯一）。

（b）展开为 $\dfrac{1}{2}n^2-\dfrac{1}{2}n$，最高次项 $\dfrac{1}{2}n^2$，去系数，$O(n^2)$。

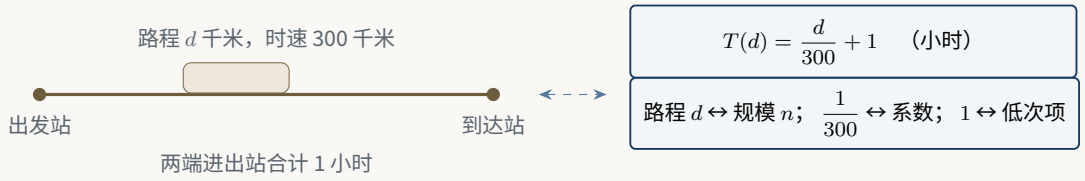
（c）常数，$O(1)$。　（d）$\log_2 n$ 比 n 增长慢（第 9 节），最高阶的项是 $2n$，$O(n)$。



本题想说明什么　找 C 和 N_0 就是在图上找“从哪里开始，虚线一直在实线上方”。

生活中的它　估算一次高铁出行要多久

生活场景　坐高铁出行，总时间 = 路上的时间 + 进站、候车、出站的固定时间。



一次小估算　$d = 300$ 时共 2 小时，固定的 1 小时占 50%；$d = 1200$ 时共 5 小时，占 20%；$d = 3000$ 时共 11 小时，只占约 9%。路程越远，总时间越接近“只由路程决定”，即 $T(d) = O(d)$。

边界说明　路程很短时固定开销反而是主角。大 O 只描述 n **很大时**的趋势，对很小的 n 不作保证。

易错点

(1) **写成** $O(3n^2)$、$O(n^2+n)$。大 O 里只保留最高次项且不带系数，应写 $O(n^2)$。

(2) **把“=”当成真正的等号**。$T(n) = O(n^2)$ 读作“$T(n)$ 是 $O(n^2)$ 的”，不能反过来写成 $O(n^2) = T(n)$，更不能由 $n = O(n^2)$ 和 $n^2 = O(n^2)$ 推出 $n = n^2$。

(3) **认为** $O(n)$ **的算法一定比** $O(n^2)$ **的快**。只在 n 足够大时成立。$100n$ 与 n^2 相比，$n < 100$ 时反而是 n^2 小（练习第 28 题）。

(4) **纠结对数的底**。由换底公式 $\log_a n = \dfrac{\log_2 n}{\log_2 a}$，不同底的对数只差一个常数倍，而常数倍被 C 吸收。所以 $O(\log_2 n)$、$O(\log_{10} n)$、$O(\log_3 n)$ 是同一个阶，课件第 49 页列出的 $O(\log_2 n)$、$O(\log n)$、$O(\mathrm{lb}\, n)$ 是同一回事。

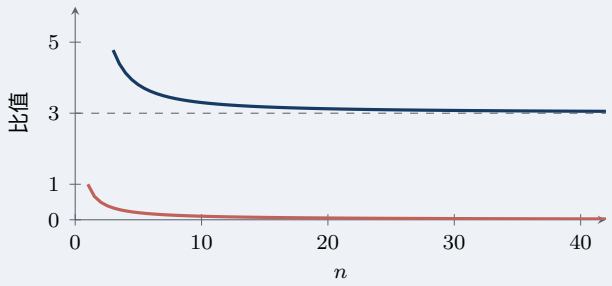
对照　大 O ↔ 高等数学“无穷大量阶的比较”（推广型）

一句话定位　时间频度 $T(n)$ 在 $n \to \infty$ 时都是无穷大量；大 O 做的事，就是高等数学里“比较两个无穷大谁增长得快”，只不过把要求放宽成了单向的上界。

比较维度	高等数学：无穷大的比较	算法分析：大 O
研究对象	$x \to \infty$ 时趋于无穷的函数 $f(x)$、$g(x)$	$n \to \infty$ 时的时间频度 $T(n)$（n 为正整数）
比较方法	求极限 $\lim f/g$	找常数 C、N_0 使 $T(n) \le Cf(n)$
结果分类	高阶（极限为 ∞）、同阶（非零常数）、等价（极限为 1）、低阶（极限为 0）	只说“不超过某个阶”
系数是否保留	等价无穷大保留系数：$3n^2 \sim 3n^2$，不等价于 n^2	不保留：$3n^2 = O(n^2)$

相同之处　都只关心自变量很大时的行为；都认为低阶项可以忽略；判断“谁增长得快”的结论完全一致。

不同之处及根源　（1）高数的“同阶”是**双向**的（比值趋于非零常数），大 O 是**单向**的上界。根源：算法分析首先要的是保证——“最多这么慢”。（2）高数的等价无穷大要保留系数，大 O 把系数丢掉。根源：系数取决于机器速度、编程语言等与算法无关的因素（图 17 中被排除的三项），保留它没有意义。



同一个小算例　比较 $3n^2+2n+10$ 与 n^2。高数的做法：$\lim_{n\to\infty}\frac{3n^2+2n+10}{n^2}=3$，是**同阶**无穷大（因为极限不是 1，所以不是等价无穷大）。大 O 的做法：取 $C=15, N_0=1$，得 $O(n^2)$。两种做法都说“它是 n^2 这个级别的”。

高等数学中的结论	迁移结果	理由或反例
求极限时低阶项可以忽略	✓ 直接成立	多项式的大 O 只由最高次项决定（已证）
两个无穷大同阶	✓ 直接成立	同阶 $\Rightarrow$ 大 O 成立（极限判别定理，已证）
$T(n)=O(f(n))$，所以 T 与 f 同阶	✗ 不成立	反例：$n=O(n^2)$，但 $n/n^2\to 0$（上图红线）
等价无穷大替换时系数不能丢，所以应写 $O(3n^2)$	✗ 不成立	大 O 中系数被常数 C 吸收，规范写法是 $O(n^2)$
用洛必达法则比较增长快慢	△ 加条件	需把 n 换成连续变量 x 再求导；得到的“谁快谁慢”可直接用于排阶，如 $\lim_{x\to\infty}\frac{\ln x}{x}=0$ 说明对数阶低于线性阶

到这里，我们已经解决了什么？　有了一个忽略系数和低次项、只比较增长阶的记号。**下一步：**常见的阶有哪些？谁快谁慢？快慢的差距到底有多大？

第 9 节　常见的时间复杂度

9.1 先看图：增长曲线

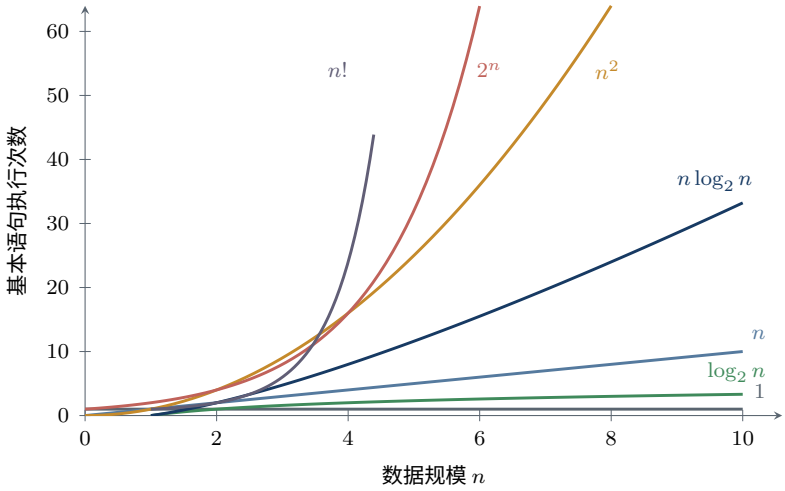


图 22　七种常见时间复杂度的增长曲线（笔记“常见时间复杂度”的图像版）

读图：横轴是数据规模 n，纵轴是执行次数，所有曲线用同一坐标。最该看出的是曲线的**走势**而不是具体数

值：$\log_2 n$ 几乎贴着横轴；n 和 $n\log_2 n$ 稳步上升；n^2 开始明显上翘；2^n 和 $n!$ 在 n 还不到 6 时就冲出了图框。n 稍大以后，各曲线自下而上的次序就固定不变了。

核心公式　常见时间复杂度的大小关系（笔记与课件第 49 页）

$$O(1) < O(\log_2 n) < O(n) < O(n\log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

这里“$<$”表示：当 n 足够大时，左边的增长得比右边慢，即时间效率更高。常见的阶不仅限于这几种。

9.2 每一个阶长什么样

下表把笔记中“常见时间复杂度”的 7 条和课件第 49 页的名称合在一起。标“后续”的例子属于后面章节，现在只需要知道名字。

阶	名称	典型的代码形态	代表（来自笔记，已补条件）
$O(1)$	常量阶	没有循环，或循环次数与 n 无关	数组随机访问；哈希表查找（平均情况，后续）
$O(\log_2 n)$	对数阶	循环变量每次乘以（或除以）一个常数	二分搜索；二叉堆的插入、AVL 树与红黑树的查找（后续）
$O(n)$	线性阶	一重循环，执行 n 次	线性搜索
$O(n\log_2 n)$	线性对数阶	一个对数循环套（或被套在）一个线性循环里	堆排序；快速排序（平均情况，后续）
$O(n^2)$	平方阶	两重循环，各执行约 n 次	冒泡、选择、插入排序
$O(n^3)$	立方阶	三重循环	两个 $n\times n$ 矩阵按定义相乘
$O(2^n)$	指数阶	每个元素“选或不选”，逐一试遍	子集树（第 11 章回溯法）
$O(n!)$	阶乘阶	把 n 个元素的所有排列逐一试遍	排列树（第 11 章回溯法）

〔教学补全〕“子集树”“排列树”是第 11 章才学的名词，但 2^n 和 $n!$ 的来历现在就能看懂：

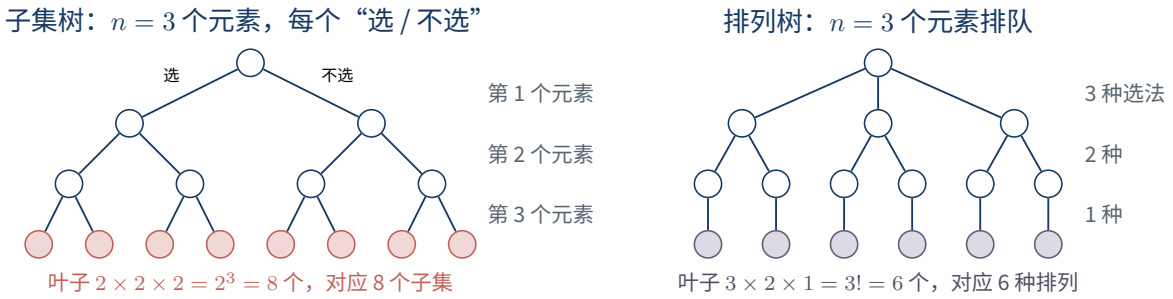


图 23　指数阶与阶乘阶的来历：把所有可能的情况都试一遍

读图：左图：每个元素有“选、不选”两种决定，n 个元素共 2^n 种组合，每片红色叶子是一个子集。右图：第 1 个位置有 n 种选法，第 2 个位置剩 $n-1$ 种……共 $n!$ 种排列。算法若要把叶子逐一检查一遍，执行次数就是 2^n 或 $n!$。

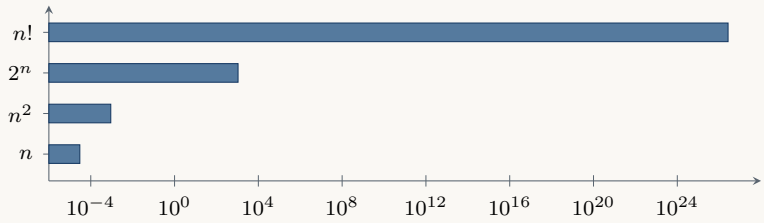
9.3 差距到底有多大

例题 9.1　同一台电脑，不同的阶

题目　假设一台计算机每秒执行 10^6 次基本操作。当 $n=30$ 时，时间频度分别为 n、n^2、2^n、$n!$ 的四个算法各需要多长时间?

解　所需时间 = 执行次数 $\div 10^6$（秒）。

时间频度	$n=30$ 时的执行次数	所需时间
n	30	0.00003 秒
n^2	900	0.0009 秒
2^n	$1\,073\,741\,824 \approx 1.07\times10^9$	约 1074 秒 $\approx$ 18 分钟
$n!$	$\approx 2.65\times10^{32}$	约 8.4×10^{18} 年



所需时间（秒，对数刻度：每一大格是 10^4 倍）

检查　$2^{10}=1024\approx10^3$，所以 $2^{30}\approx10^9$，除以 10^6 约为 10^3 秒，与表中一致。

本题想说明什么　从 n^2 到 2^n 不是“慢一点”，而是从“瞬间完成”变成“等十几分钟”；到 $n!$ 则根本等不到结果。注意横轴是对数刻度，四根柱子的真实长度相差远比看上去悬殊。

易性算法与顽性算法　课件第 49 页指出：从常量阶到平方阶，通常称为**易性算法**；到达立方阶之后，一旦数据规模大些，时间就已经不能忍受，称为**顽性算法**。笔记里也在 $O(n^2)$ 与 $O(2^n)$ 之间画了一条红线，左边标“推荐”，右边标“不推荐”，并在指数阶、阶乘阶旁写了“要优化”。

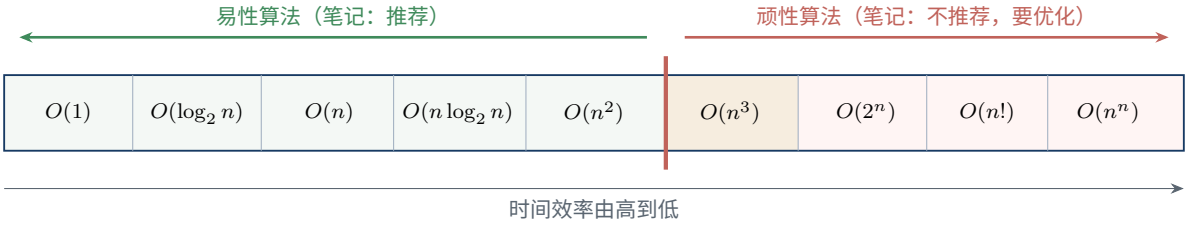


图 24　易性算法与顽性算法的分界（按课件第 49 页；红线即笔记中的红色分界线）

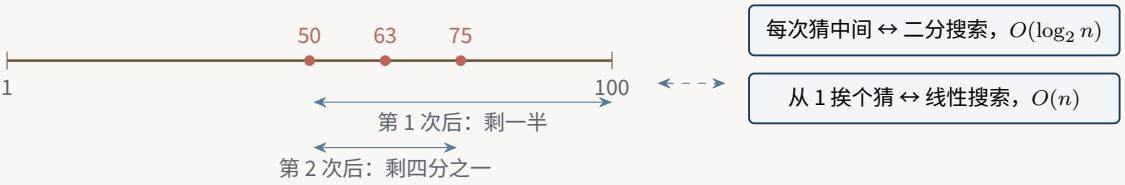
读图：绿色区域是规模增大时仍然用得起的算法，红色区域是规模稍大就无法忍受的算法。立方阶处在过渡地带：课件把它划入顽性一侧，笔记把分界线画在平方阶与指数阶之间，二者对“平方阶及以下可用、指数阶及以上不可用”的判断是一致的。

拓展联想：以下内容只用于建立知识联系，当前阶段不要求掌握。

算法理论中更常用的划分是“多项式时间”（n、n^2、n^3……）与“指数时间”（2^n、$n!$……）：前者统称有效算法，后者被认为实际不可行。这与课件的说法并不矛盾，只是分界线画得更靠右。

生活中的它　猜数字游戏

生活场景　朋友心里想一个 $1 \sim 100$ 的整数，你每猜一次，他回答“大了”“小了”或“对了”。



一次小估算　每次猜中间的数，范围减半：$100 \to 50 \to 25 \to \cdots$，最多 7 次必中（因为 $2^7 = 128 \geq 100$）。范围扩大到 $1 \sim 1\,000\,000$，也只要 20 次（$2^{20} = 1\,048\,576$）。若从 1 开始挨个猜，最坏分别要 100 次和 100 万次。

边界说明　“每次减半”依赖于数是**有序**的并且对方会告诉你大小；如果对方只回答“对”或“不对”，就只能挨个猜了。

易错点

(1) **把 n^2 和 2^n 混为一谈**。n^2 是幂函数（底数变、指数不变），2^n 是指数函数（底数不变、指数变）。$n = 10$ 时二者是 100 与 1024；$n = 30$ 时是 900 与约 10^9。

(2) **认为大小关系对任何 n 都成立**。它说的是“n 足够大”。例如 $n = 3$ 时 $2^n = 8 < n^2 = 9$；$n = 4$ 时两者相等；从 $n = 5$ 起才一直有 $2^n > n^2$。

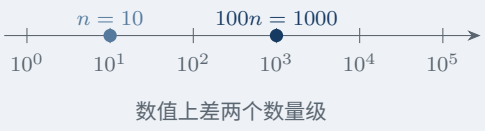
(3) **不知道 $n \log_2 n$ 放在哪**。因为 $1 < \log_2 n < n$（$n > 2$），同乘 n 得 $n < n \log_2 n < n^2$，它夹在线性阶和平方阶之间。

(4) **认为“对数阶”说明算法用了对数运算**。不是。只要每一步把问题规模缩小为原来的几分之一，步数就是对数级的（第 10 节）。

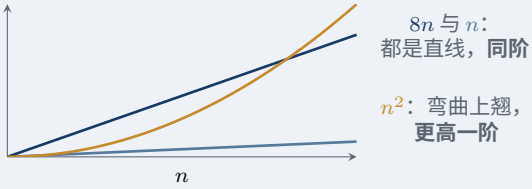
对照　复杂度的“阶” ↔ 物理估算中的“数量级”（同名异义型）

一句话定位　物理课说“差了两个数量级”，算法课说“差了一个阶”，听起来相似，指的却是两回事：数量级描述**一个具体数值**有多大，阶描述**一个函数**增长得有多快。

数量级：给一个数定位



阶：给一个函数的增长方式归类



从“数量级估算”带来的习惯	迁移结果	理由或反例
抓主要因素，忽略次要因素	✓ 直接成立	大 O 正是只保留最高次项
相差 100 倍就是相差两个数量级，所以也相差两个阶	✗ 不成立	$100n$ 与 n 永远相差 100 倍，却是**同一个阶**
		$O(n)$：常数倍不改变阶
两个量的数量级之差是固定的	△ 加条件	对**不同阶**的两个函数，差距随 n 变化：n^2 与 n 在 $n = 10$ 时差 1 个数量级，在 $n = 1000$ 时差 3 个数量级

到这里，我们已经解决了什么？　认识了常见的阶以及它们的快慢次序。**下一步：**实际的程序往往由好几段循环拼成，有的并列，有的嵌套，有的循环变量不是每次加 1。怎么快速得出整段程序的复杂度?

第 10 节　求和定理、求积定理与循环分析

10.1　两个定理

课件第 50、51 页给出了计算时间复杂度的两个简化工具。课件只写了结论和例子，这里补上证明。

定理　求和定理（顺序执行：相加，取大）

设 $T_1(n)$、$T_2(n)$ 是程序段 P_1、P_2 的运行时间，且 $T_1(n)=O(f(n))$，$T_2(n)=O(g(n))$。那么先运行 P_1 再运行 P_2 的总运行时间为

$$T_1(n)+T_2(n)=O\big(\max(f(n),g(n))\big).$$

证明　**已知：**存在正常数 C_1,N_1，使 $n>N_1$ 时 $T_1(n)\le C_1 f(n)$；存在正常数 C_2,N_2，使 $n>N_2$ 时 $T_2(n)\le C_2 g(n)$。
思路：f 和 g 都不超过它们中较大的那个。
证明：记 $h(n)=\max(f(n),g(n))$，则 $f(n)\le h(n)$，$g(n)\le h(n)$。当 $n>\max(N_1,N_2)$ 时两个不等式同时成立，于是

$$T_1(n)+T_2(n)\le C_1 f(n)+C_2 g(n)\le C_1 h(n)+C_2 h(n)=(C_1+C_2)\,h(n).$$

取 $C=C_1+C_2$，$N_0=\max(N_1,N_2)$，即得 $T_1(n)+T_2(n)=O(h(n))$。 ■

定理　求积定理（嵌套执行：相乘）

若 $T_1(n)=O(f(n))$，$T_2(n)=O(g(n))$，则 $T_1(n)\times T_2(n)=O\big(f(n)\times g(n)\big)$。

证明　沿用上面的 C_1,N_1,C_2,N_2。由于各量非负，当 $n>\max(N_1,N_2)$ 时，两个不等式可以相乘：

$$T_1(n)\,T_2(n)\le C_1 f(n)\cdot C_2 g(n)=(C_1C_2)\,f(n)\,g(n).$$

取 $C=C_1C_2$，$N_0=\max(N_1,N_2)$ 即可。 ■

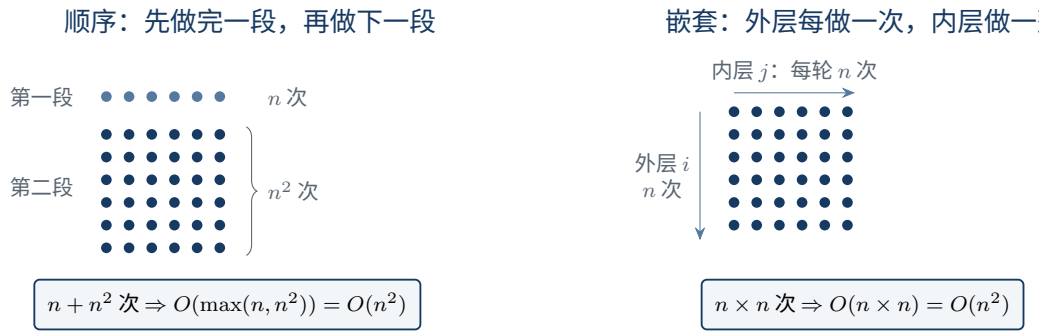


图 25　两个定理的图形含义：每个圆点表示循环体执行一次（图中取 $n=6$）

读图：左图是课件第 50 页的代码：第一段单重循环（上面一行 6 个点），第二段双重循环（下面 36 个点），总数由大的那块决定。右图是课件第 51 页的代码：外层走一步，内层走一整行，点阵排成 $n \times n$ 的方阵。

课件的两段代码

```
for (i = 0; i < n; i++)  a[i] = 0;          // 第一段：O(n)
for (i = 0; i < n; i++)                     // 第二段：O(n^2)
    for (j = 0; j < n; j++)  a[i] = i + j;
// 求和定理：O(max(n, n^2)) = O(n^2)
```

```
for (i = 0; i < n; i++)                     // 外层：O(n)
    for (j = 0; j < n; j++)                 // 内层：O(n)
        { s = s + i + j;  cout << s; }
// 求积定理：O(n * n) = O(n^2)
```

10.2 五种循环形状

把常见的循环按“循环体执行次数排成什么形状”分类，以后看到代码就能直接对号入座。

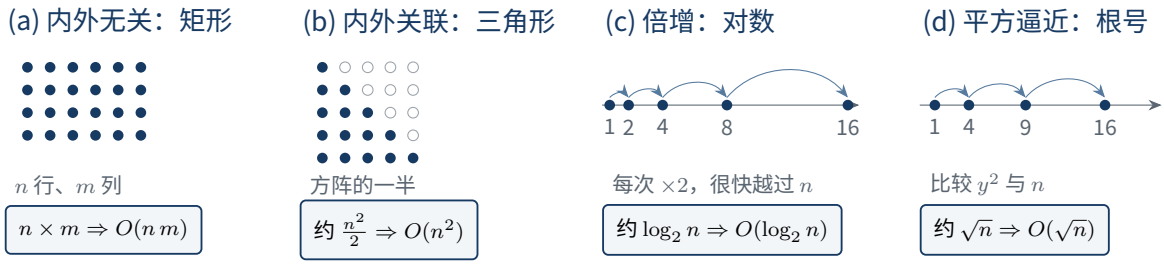


图 26　四种与 n 有关的循环形状；第五种“与 n 无关”的循环恒为 $O(1)$

读图：(a)(b) 中每个实心点是循环体执行一次，(b) 的空心点是“没有执行的另一半”；(c)(d) 画的是循环变量在数轴上跳过的位置，点的个数就是循环次数。四个图回答同一个问题：**循环体一共执行多少次。**

(a) 内外无关的嵌套　内层次数不依赖外层变量时，直接用求积定理。例：`for (i=0;i<n;i++) for (j=0;j<m;j++) a[i][j]=0;` 循环体执行 $n \times m$ 次，时间复杂度 $O(m \times n)$（课件第 58 页填空题）。规模由两个量决定时，两个字母都要保留。

(b) 内外关联的嵌套　内层次数随外层变量变化时，不能简单地写“$n \times n$”，要用求和。对课件第 47 页的代码

```
for (i = 0; i < n; i++)
    for (j = 0; j < i; j++)        // 内层执行 i 次：i 越大，这一行越长
        { s = s + i + j;  cout << s; }
```

外层 $i = 0, 1, \ldots, n-1$ 时，内层分别执行 $0, 1, \ldots, n-1$ 次。这是首项为 0、末项为 $n-1$、共 n 项的等差数列，其和为

$$\sum_{i=0}^{n-1} i = \frac{(0 + n - 1) \times n}{2} = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n \implies O(n^2).$$

(c) 倍增循环　课件第 60 页：`i=1; while (i<=n) i=i*3;`。循环变量依次取 $1, 3, 9, 27, \ldots$。设循环体执行了 k 次，此时 $i = 3^k$。循环在 i 第一次超过 n 时停止，即 k 是满足 $3^k > n$ 的最小整数：

$$3^k > n \iff k > \log_3 n \quad \Longrightarrow \quad k = \lfloor \log_3 n \rfloor + 1.$$

所以时间复杂度是 $O(\log_3 n)$，与 $O(\log_2 n)$ 同阶。（$\lfloor x \rfloor$ 表示不超过 x 的最大整数。）验证：$n=100$ 时 i 取 $1,3,9,27,81$ 共 5 次，而 $\lfloor \log_3 100 \rfloor + 1 = 4+1 = 5$。

(d) 平方逼近　课件第 62 页：`x=n; y=0; while (x>=(y+1)*(y+1)) y++;`。循环持续的条件是 $(y+1)^2 \le n$。设循环体执行了 k 次，则结束时 $y=k$，且 k 是满足 $(k+1)^2 > n$ 的最小整数，也就是 $k = \lfloor \sqrt{n} \rfloor$。时间复杂度 $O(\sqrt{n})$。验证：$n=20$ 时 y 从 0 增到 4 停止（$5^2=25>20$），执行 4 次，$\lfloor \sqrt{20} \rfloor = 4$。

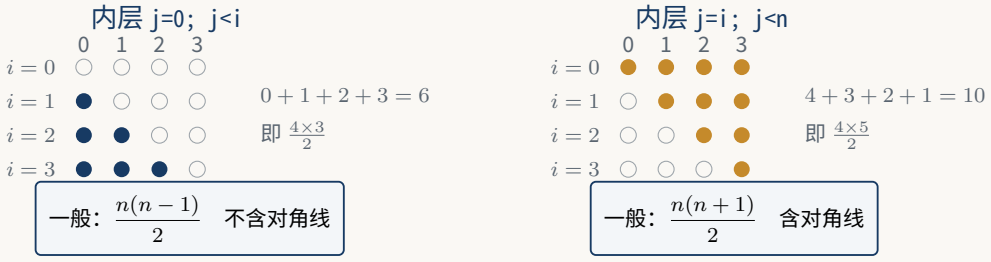
(e) 与 n 无关　课件第 57 页：

```
x = 90;  y = 100;
while (y > 0)
    if (x > 100) { x = x - 10;  y--; }
    else  x++;
```

这段程序里根本没有 n。无论外部数据规模是多少，它执行的次数都是同一个固定的数（实际跟踪可得循环体执行 1101 次），所以时间复杂度是 $O(1)$。**判断 $O(1)$ 的标准不是“次数少”，而是“次数不随 n 变化”。**

例题　10.1　两个长得很像的三角形

题目　课件第 47 页代码的内层是 `for (j=0; j<i; j++)`，而求和式对应的是 `for (j=i; j<n; j++)`。取 $n=4$，分别数出两种写法循环体的执行次数，并写出一般公式。



结果　两种写法分别执行 $\dfrac{n(n-1)}{2}$ 次和 $\dfrac{n(n+1)}{2}$ 次，相差恰好是对角线上的 n 个点。

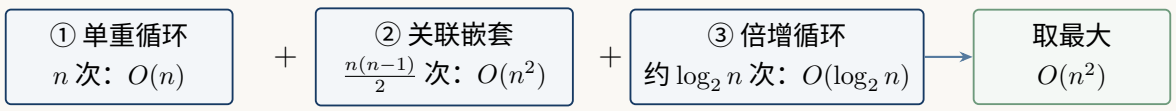
检查　两种情况正好互补：不含对角线的下三角（左图）加上含对角线的上三角（右图），拼成整个 4×4 方阵，$6+10=16=4^2$。

本题想说明什么　写求和式之前先确认“j 从哪到哪”。两种写法的**次数**不同，但最高次项都是 $\frac{1}{2}n^2$，时间复杂度都是 $O(n^2)$。

例题　10.2　一段“拼起来”的程序

题目　求下面程序段的时间复杂度。

```
for (i = 0; i < n; i++)  a[i] = 0;              // ①
for (i = 0; i < n; i++)                         // ②
    for (j = 0; j < i; j++)  b[i] = b[i] + a[j];
i = 1;  while (i < n)  i = i * 2;               // ③
```

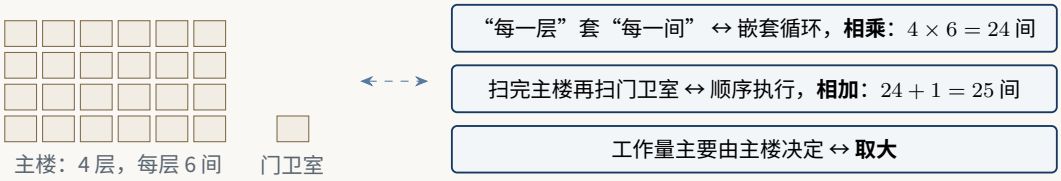


解　三段是顺序关系，用求和定理：$O(\max(n,\ n^2,\ \log_2 n)) = O(n^2)$。

本题想说明什么　一段程序的复杂度由其中**最慢的那一段**决定。想让整体变快，优化 ① 和 ③ 没有用，必须优化 ②。

生活中的它　值日生打扫教学楼

生活场景　值日生逐层逐间打扫教学楼。



一次小估算　新教学楼 20 层、每层 50 间：$20 \times 50 = 1000$ 间，再加 1 间门卫室共 1001 间——门卫室那一间几乎可以忽略。

边界说明　现实中教室有大有小，每间耗时不同；模型假设“每间教室的工作量相同”。

易错点

(1) **“看到两重循环就写 $O(n^2)$。”**要看每一重各执行多少次。外层 `i=i*2`、内层 n 次的两重循环是 $O(n \log_2 n)$（练习第 30 题）。

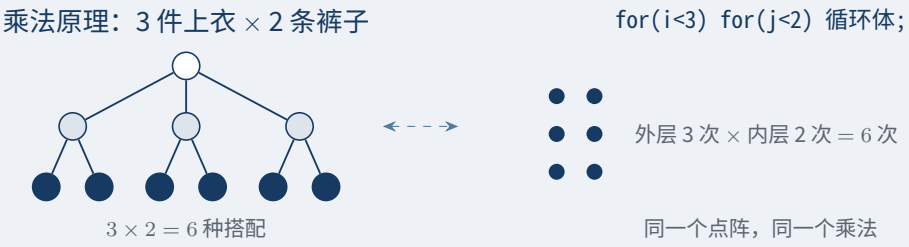
(2) **求和定理写成 $O(n+n^2)$ 就停下。**还要取较大者，化成 $O(n^2)$。

(3) **内层依赖外层时直接相乘。**这样得到的次数是错的（n^2 与 $\frac{n(n-1)}{2}$）。虽然此例的阶恰好相同，但题目若问“执行多少次”就会答错（课件作业 1.3）。

(4) **认为次数少就是 $O(1)$、次数多就不是。**课件第 57 页那段程序循环了一千多次，仍是 $O(1)$，因为次数与 n 无关。

对照　求和 / 求积定理 ↔ 计数的加法原理 / 乘法原理（思想重现型）

一句话定位　中学排列组合里的“分类相加、分步相乘”，就是循环分析里“顺序相加、嵌套相乘”的原型。



排列组合中的结论	迁移结果	理由或反例
分步完成，各步方法数相乘	△ 加条件	条件：内层次数不依赖外层变量。依赖时（如 `j<i`）要改用求和
分类完成，各类方法数相加	✓ 直接成立	顺序执行的各段，执行次数就是相加
相加之后，每一项都要保留	✗ 不成立	次数确实是 $n+n^2$，但写成大 O 时只留较大的：$O(n^2)$

到这里，我们已经解决了什么？　能分析各种形状的循环，得出时间复杂度。**下一步：**五条要求中还有一条“空间效率”没有量化。

第 11 节　空间复杂度

11.1　算法占用的空间分三块

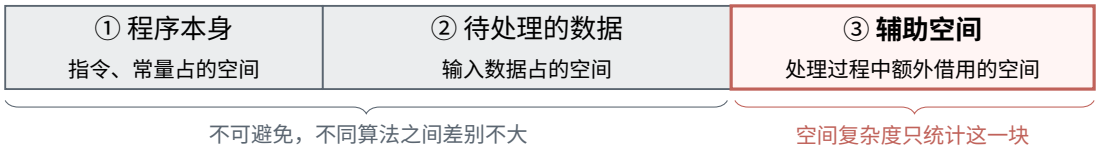


图 27　算法的空间消耗由三部分组成，设计算法时主要关注第 ③ 部分

读图：整条横条表示算法运行时占用的全部内存。前两块（灰色）是做这件事必然要占的；第三块（红色）取决于**算法怎么设计**，是比较不同算法时真正有区别的部分。

定义　空间复杂度

渐进空间复杂度（简称**空间复杂度**）是当数据规模 n 趋于无穷时，算法使用的**辅助空间**的量阶，记为 $S(n)=O(f(n))$。

它的计算方法与时间复杂度相同：先写出辅助空间大小关于 n 的函数，再取最高阶项、去掉系数。

11.2　先看图：同一件事的两种做法

课件第 53、54 页用“把数组逆置”做例子。数组为 `a[10] = {1,6,2,5,8,9,5,4,3,12}`。

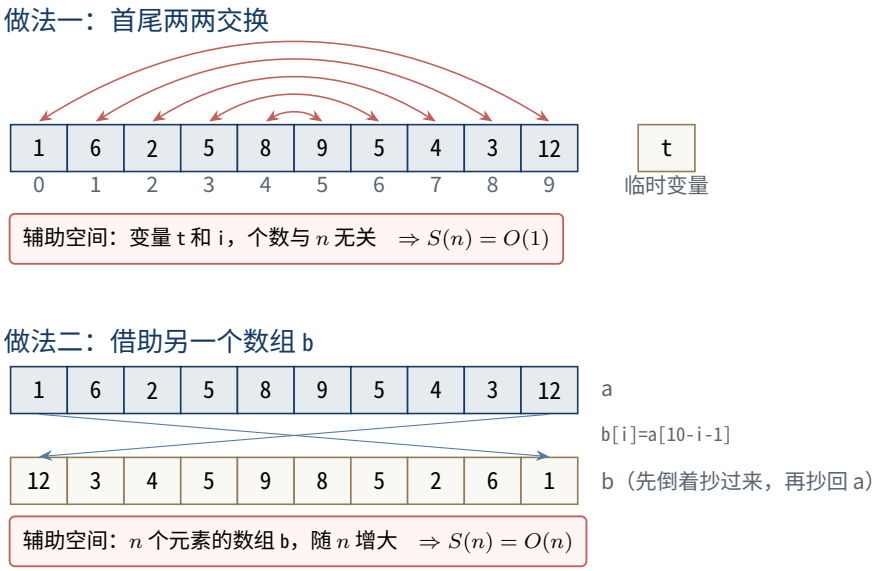


图 28　数组逆置的两种算法及其辅助空间

读图：上图：红色弧线表示一次交换，`a[0]` 与 `a[9]`、`a[1]` 与 `a[8]`……直到 `a[4]` 与 `a[5]`，一共 5 次；每次交换都借用同一个米色格子 `t`。下图：米色的一整排是额外申请的数组 `b`，数组 `a` 有多长它就得多长。

```
// 做法一：S(n) = O(1)                    // 做法二：S(n) = O(n)
int t, i;                                 int b[10], i;
```

```
for (i = 0; i < 5; i++) {
    t = a[i];
    a[i] = a[10-i-1];
    a[10-i-1] = t;
}
```

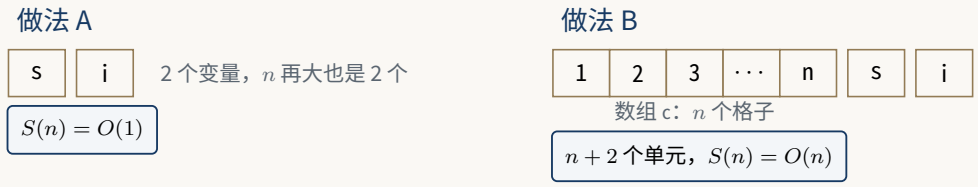
```
for (i = 0; i < 10; i++)
    b[i] = a[10-i-1];
for (i = 0; i < 10; i++)
    a[i] = b[i];
```

时间上谁快　〔教学补全〕做法一循环 $n/2$ 次（每次 3 个赋值），做法二两个循环共 $2n$ 次赋值。两者的时间复杂度都是 $O(n)$——时间同阶，这时才轮到比较空间：做法一 $O(1)$ 优于做法二 $O(n)$。

课件的说明　在内存足够大的情况下，算法更加注重时间效率，常常忽略空间复杂度的计算；或者仅当算法的时间复杂度一致时，才比较空间复杂度的优劣。

例题　11.1　判断辅助空间

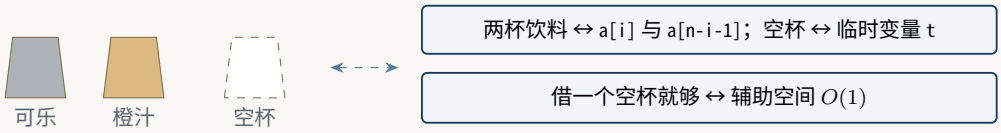
题目　求 $1+2+\cdots+n$。做法 A：用一个变量 s 边循环边累加。做法 B：先把 $1\sim n$ 存进一个新数组 c[n]，再把数组元素加起来。两种做法的空间复杂度各是多少？



本题想说明什么　判断 $S(n)$ 只问一句：**额外申请的空间会不会随着 n 变大？**不会是 $O(1)$，成正比是 $O(n)$。

生活中的它　把两杯饮料对调

生活场景　桌上一杯可乐、一杯橙汁，想让两个杯子里的饮料互换。



一次小判断　要把 10 对杯子逐对对调，需要准备几个空杯？**1 个**就够，每对调完一对，空杯又空出来了。若改为“把 20 杯饮料全部倒进另外 20 个新杯子再倒回来”，就需要 20 个空杯——对应做法二的 $O(n)$。

边界说明　程序里的“空杯”用完后可以立即重复使用；统计辅助空间时看的是**同一时刻最多**占用多少，而不是累计借用了多少次。

易错点

(1) **把输入数据也算进空间复杂度**。数组 a 本身属于第 ② 块，不计入；只算额外的 t、b 这类辅助空间。

(2) **认为 $O(1)$ 表示“不占内存”**。它表示辅助空间是与 n 无关的常数，可以是 2 个变量，也可以是 200 个。

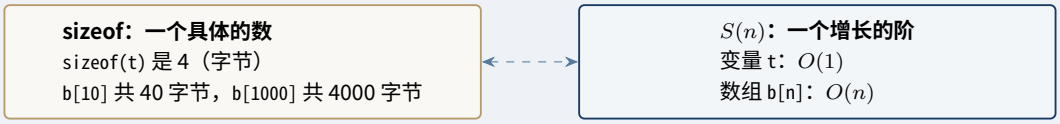
(3) **认为时间和空间能同时最优**。常常需要取舍：多用空间可以省时间，反之亦然。本课程在两者冲突时优先考虑时间。

相近概念辨析　时间复杂度与空间复杂度：

比较维度	时间复杂度 $T(n)$	空间复杂度 $S(n)$
数的是什么	基本语句的执行**次数**	辅助存储单元的**个数**
怎么累计	每执行一次就加 1，只增不减	看同一时刻的最大占用，可重复利用
逆置做法一	$O(n)$	$O(1)$
逆置做法二	$O(n)$	$O(n)$
优先级	通常优先考虑	时间同阶时再比较

对照　空间复杂度 ↔ C++ 的 sizeof（同名异义型）

一句话定位　两者都在回答“占多少内存”，但 sizeof 给的是**具体的字节数**，$S(n)$ 给的是**随 n 增长的阶**。



从 sizeof 带来的认识	迁移结果	理由或反例
数组占的空间与元素个数成正比	✓ 直接成立	n 个元素的辅助数组，$S(n)=O(n)$
int 占 4 个字节，所以变量 t 的空间复杂度是 $O(4)$	✗ 不成立	常数一律写 $O(1)$，系数不保留
sizeof 的结果依赖机器，所以 $S(n)$ 也依赖机器	✗ 不成立	换一台机器 int 可能占 8 个字节，辅助数组占 $8n$ 字节，仍是 $O(n)$；阶与机器无关

学完第二部分，你应该能够回答

(1) 算法的五个特性是什么？“有穷性”和“时间效率”有何区别？
(2) `for (i=0; i<n; i++)` 中，条件 `i<n` 执行多少次？为什么？
(3) 大 O 定义中的 C 和 N_0 各起什么作用？
(4) 为什么可以只看最高次项、不看系数？
(5) 按从快到慢排出七种常见的时间复杂度。$n\log_2 n$ 排在哪两个之间？
(6) 顺序执行的两段程序，总复杂度怎么求？嵌套呢？
(7) `while (i<=n) i=i*2;` 为什么是对数阶？
(8) 空间复杂度统计的是哪一部分空间？逆置数组的两种做法各是多少？

第三部分　写代码要用的四个 C++ 工具

第 12 节　面向对象、模板、const 与异常处理

课件最后 20 页（第 64–83 页）是“课后复习内容”，复习四个 C++ 机制。它们不是新的数据结构知识，但从第 2 章起，教材的每一个类都同时用到这四样东西。本节用同一个小任务把它们串起来。

贯穿全节的任务（课件第 66 页）　将 1 ～ 20 之间的奇数存入内存，之后在这组数中查找用户输入的任意一个整数，并报告查找结果。

12.1　面向过程与面向对象

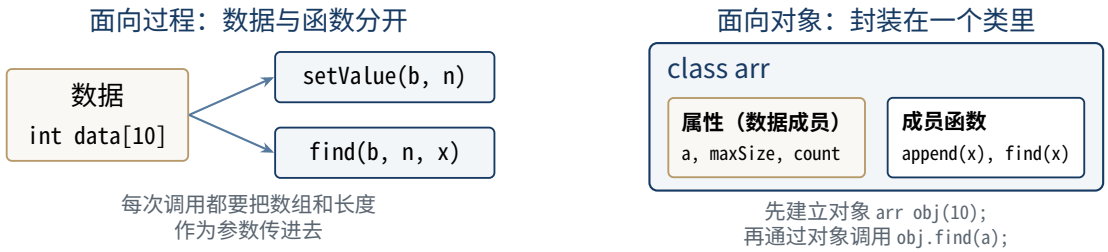


图 29　同一个任务的两种组织方式（依据课件第 67–69 页）

读图：左图：数组 data 和操作它的函数是分开的两样东西，靠参数联系。右图：数据和操作数据的函数被包进同一个类，分别成为类的**属性**和**成员函数**；整个类相当于定义了一个新的数据类型，再根据具体问题建立该类的对象来解决问题。这正是第 5 节“封装性”在 C++ 里的实现。

12.2　泛型机制：函数模板与类模板

课件第 70 页的理由是：数据结构研究的是具有一定关系且类型相同的一组元素；元素无论是整型、字符型还是复杂的结构类型，它们在关系和基本操作的处理方法上是**一样的**。既然处理方法与元素类型无关，就没有必要为每种类型各写一遍——把类型也当成一个“参数”，这就是模板。

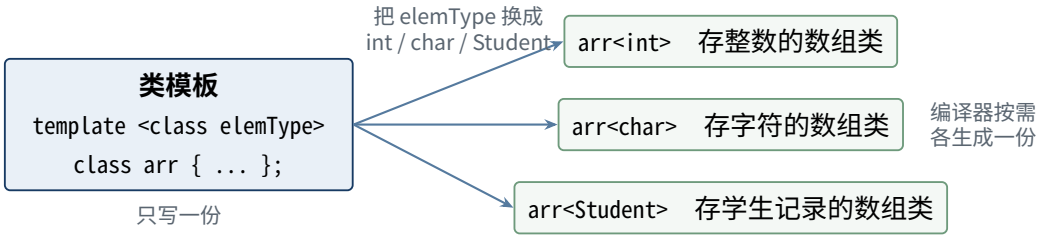


图 30　类模板的实例化：一份模板，由编译器生成多个具体的类

读图：左边是程序员写的唯一一份代码，其中元素类型用占位符 elemType 表示。写下 arr<int> obj1(10); 时，尖括号里的 int 使类模板**实例化**为一个具体的类（课件称“模板类”）。

函数模板的用法相同。课件第 71 页的例子修正后如下（函数改名为 myMax，以免与标准库已有的 max 冲突）：

```
template <class T>
T myMax(T x, T y) { if (x > y) return x; else return y; }
```

```
int main() {
    cout << myMax(3, 5) << myMax('a', 'h') << endl;     // 输出 5h
    return 0;                                           // 编译器自动检测类型，生成对应的函数
}
```

12.3 const 机制与引用

用 const 修饰变量，变量将变为常量；常量一般有初值，在其生命周期中不得改变其值，例如 const double PI = 3.14;。在类的成员函数 void find(const elemType &x) const; 中，const 出现在两个位置，保护的对象不同：

写法	保护谁	作用
参数前的 const	参数 x	函数实现中若有修改 x 的语句，编译时就报错
参数前的 &（引用）	——	形参 x 不另外分配空间，是实参的**别名**，与实参共用空间
参数表后的 const	调用该函数的对象	保证函数不修改对象的数据成员；常对象只能调用这种函数

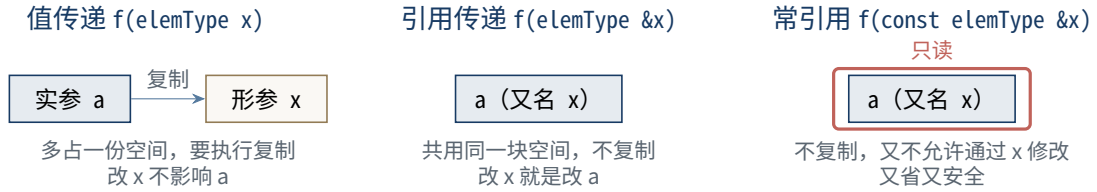


图 31　三种参数传递方式在内存中的区别

读图：蓝格是调用者的变量，米格是函数为形参另外开辟的空间。值传递有两格；引用传递只有一格，x 只是 a 的另一个名字；常引用在此基础上加了一把“只读锁”。

课件第 79 页的结论：因为参数类型 elemType 是一种泛型类型，调用时实参可能是复杂的结构类型，加了 & 就有明显的好处——（a）形参不分配空间，节省了空间消耗；（b）没有空间分配，就不涉及拷贝构造函数的执行，提高了运行效率。所以**泛型类型的参数前尽量加 &，如果函数又不需要改变这个参数，请养成 const 加 & 组合的习惯**。这句话用本章的术语说就是：值传递的辅助空间与元素大小成正比，引用传递的辅助空间是 $O(1)$。

12.4 异常处理

课件第 81 页提出的问题是：函数 fetch(i) 读取并返回下标为 i 的元素，当 i 越界时返回什么合适？直接调用 exit 结束整个程序吗？

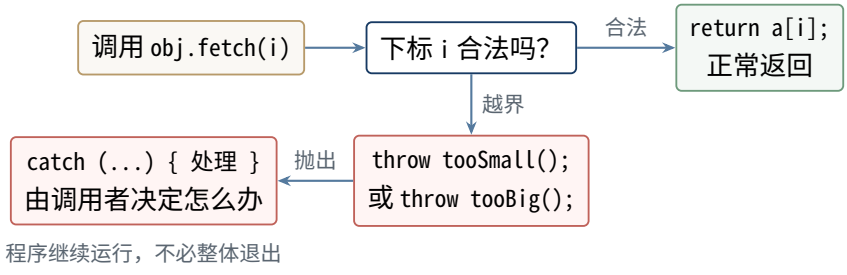


图 32　throw—try—catch 异常处理机制的流程

读图：绿框是正常路径。出现越界时，fetch 自己并不知道该怎么处理才合适，于是用 throw 把问题“抛”给调用它的人；调用者把可能出问题的语句放在 try 块中，并用 catch 接住对应类型的异常进行处理。与 exit(1) 相比，程序不会被强行终止——这对应第 6 节的**健壮性**。

12.5 把四样东西合在一起：一个完整程序

〔教学补全〕课件把这个例子分散在十几页中。下面是拼合后的完整程序，已用 g++ 实际编译运行；为便于在 main 中演示，find 改为返回 bool。

```
// 第 1 章 C++ 复习：把课件第 68 至 83 页的片段拼成一个完整、可编译的程序
#include <iostream>
using namespace std;

class tooSmall {};                      // 两个“异常类”：只用来区分出了哪种错
class tooBig {};

template <class elemType>               // 类模板：元素类型 elemType 待定
class arr {
private:                                // 内部实现：对使用者隐藏
    elemType *a;                        // 一块连续空间（顺序存储）
    int maxSize, count;                 // 容量、当前元素个数
public:                                 // 接口
    arr(int size);
    ~arr() { delete[] a; }
    void append(const elemType &x);     // 参数前加 const ：函数内不修改 x
    bool find(const elemType &x) const; // 参数表后加 const ：函数不修改调用它的对象
    elemType fetch(int i) const;        // 取下标为 i 的元素，越界时抛出异常
};

template <class elemType>
arr<elemType>::arr(int size) {
    a = new elemType[size];
    maxSize = size;
    count = 0;
}

template <class elemType>
void arr<elemType>::append(const elemType &x) {
    if (count == maxSize) return;       // 已满则不再添加
    a[count] = x;
    count++;
}

template <class elemType>
bool arr<elemType>::find(const elemType &x) const {
    int i;
    for (i = 0; i < count; i++)
        if (a[i] == x) break;
    return i != count;                  // 提前跳出说明找到了
}

template <class elemType>
elemType arr<elemType>::fetch(int i) const {
```

```
    if (i < 0)      throw tooSmall();     // 下标太小：抛出异常，交给调用者决定怎么办
    if (i >= count) throw tooBig();       // 下标太大
    return a[i];
}

int main() {
    arr<int> obj(10);                     // <int> 使类模板实例化为一个具体的类
    for (int i = 0; i < 10; i++)
        obj.append(2 * i + 1);            // 放入 1,3,5,...,19

    cout << "find 7: "  << (obj.find(7) ? "exists" : "doesn't exist") << endl;
    cout << "find 8: "  << (obj.find(8) ? "exists" : "doesn't exist") << endl;

    int idx[3] = {3, 12, -1};             // 依次读取下标 3 , 12 , -1
    for (int k = 0; k < 3; k++) {
        try {
            cout << "fetch " << idx[k] << ": " << obj.fetch(idx[k]) << endl;
        }
        catch (tooSmall) { cout << "index is too small!" << endl; }
        catch (tooBig)   { cout << "index is too big!"   << endl; }
    }
    cout << "Return to main, it is over!" << endl;
    return 0;
}
```

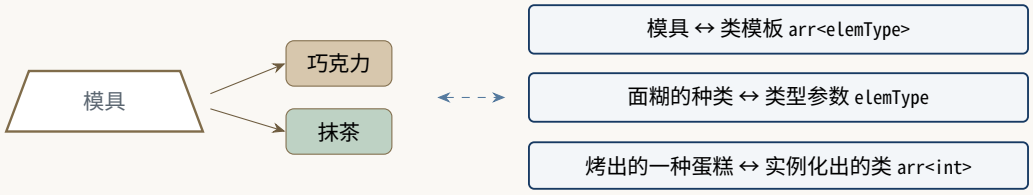
运行结果

```
find 7: exists
find 8: doesn't exist
fetch 3: 7
fetch 12: index is too big!
fetch -1: index is too small!
Return to main, it is over!
```

每一段在干什么　两个空类 tooSmall、tooBig 只起“标签”作用，用来区分是哪一种错误。类模板 arr 的 private 部分是顺序存储的三个数据成员，public 部分是接口。append 把新元素放在下标 count 处；find 是线性搜索，时间复杂度 $O(n)$；fetch 直接按下标取元素，时间复杂度 $O(1)$。main 中读取下标 3 成功，读取 12 和 -1 时分别被两个 catch 接住，程序没有中断，继续执行到最后一行。

生活中的它　蛋糕模具

生活场景　同一个蛋糕模具，倒进巧克力面糊烤出巧克力蛋糕，倒进抹茶面糊烤出抹茶蛋糕。



一次小判断　arr<int> 和 arr<double> 是同一个类吗？**不是**。它们由同一个模板生成，但是两个不同的类，就像两种口味的蛋糕。

边界说明　模具不能倒钢水。同样，模板要求传入的类型支持模板里用到的操作：find 中用了 a[i]==x，所以 elemType 必须能用 == 比较。

易错点

(1) **分不清两个 const**。参数前的保护**参数**，参数表后的保护**调用函数的对象**。

(2) **认为加了 & 函数就一定会修改实参**。不会。`const elemType &x` 既不复制，也不允许修改。

(3) **越界时用 exit 直接退出**。这会让整个程序终止，调用者没有机会补救。应抛出异常，让调用者决定。

(4) **模板写成 arr obj(10);**。类模板必须给出类型参数：`arr<int> obj(10);`。

对照　函数模板 ↔ 函数重载（方法互补型）

一句话定位　两者都能让“同一个函数名处理多种类型”：重载是程序员为每种类型**各写一份**；模板是程序员**只写一份**，由编译器按需生成。



从函数重载带来的认识	迁移结果	理由或反例
同一个函数名可以处理不同类型的参数	✓ 直接成立	模板同样做到这一点
每增加一种类型，就要再写一个函数	✗ 不成立	模板只写一份，新类型由编译器自动生成
不同类型的版本可以有不同的处理逻辑	△ 加条件	模板的各个版本逻辑**完全相同**；只有当各类型的处理方法一样时才适合用模板——这正是数据结构的情形

本章知识地图

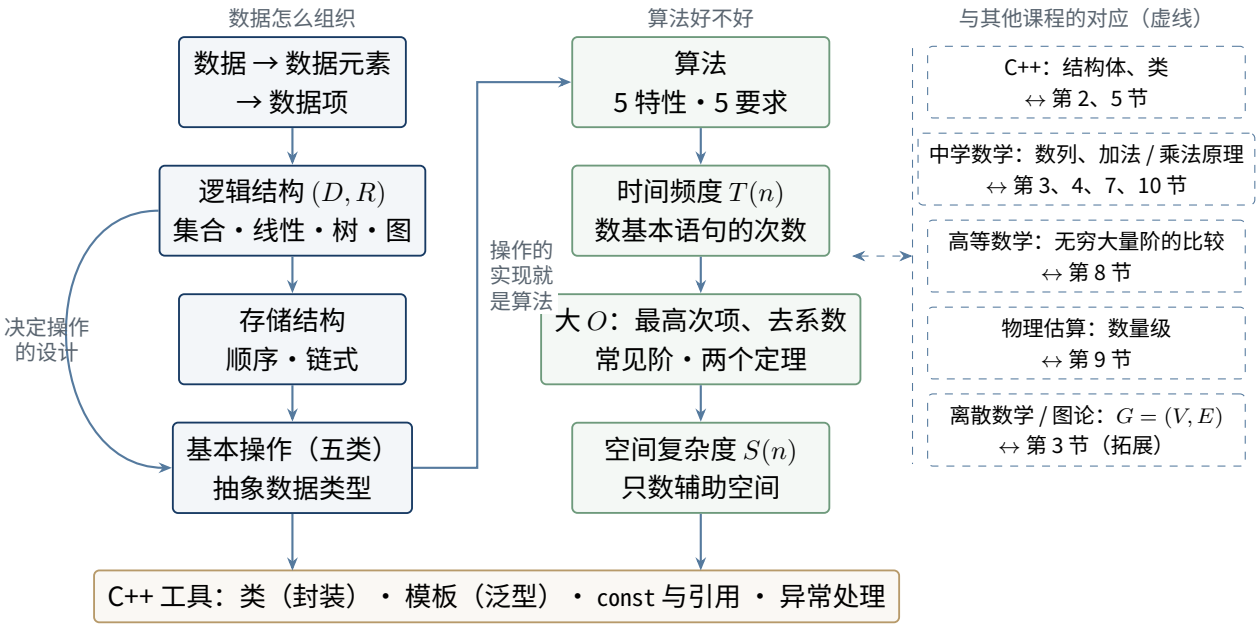


图 33　第 1 章知识地图：实线是章内的推导关系，虚线是与其他课程知识的对应

读图：左列自上而下是“数据”这条线，中列是“算法”这条线，底部米色框是把它们写成代码的工具。两列之间的连接点是：基本操作的**实现**就是算法。右侧虚线框是各节“对照”模块中出现过的其他课程知识。

第四部分　分层练习：38 道

第 13 节　练习说明与知识点—题目对应表

练习分三层：基础 25 道，学完正文即可完成；提升 10 道，需要把两三个知识点连起来用；跨学科 3 道，把本章知识搬到一个相邻学科的简单情境里，所需背景都写在题干中。其中课件第 30–32、57–63 页的 9 道课堂题和 1 道作业题已全部收入（题后注明“课件题”）。带 生活情境 的题目采用生活情境。题目区的图只画题目给出的信息；答案速查在第 17 节，逐题图解在第 18–20 节。除第 29 题专门考查定义外，所有题目中的时间复杂度都按第 8 节的约定**取最紧的阶**。

知识模块	基础题	提升题	跨学科	涉及的对照
1 为什么学数据结构	11	34		
2 数据 · 数据元素 · 数据项	4			
3 逻辑结构	1, 2, 3, 17, 22	34	37	
4 存储结构	5, 14			
5 基本操作与 ADT	6			
6 算法及其特性	12			
7 时间频度	7, 21, 23	32	38	数列求和（21, 32）
8 大 O 表示法	8, 24	28, 29		高数同阶无穷大（29）
9 常见的时间复杂度	9, 13	33, 34	36	数量级（28）
10 两个定理与循环分析	15, 18, 19, 20	26, 27, 30, 31, 35	37	乘法原理（15, 30）
11 空间复杂度	10, 25			
12 四个 C++ 工具	16			

第 14 节　A 层：基础巩固训练（25 道）

一、单项选择题（第 1–10 题）

1. 基础　（课件题）在数据结构中，从逻辑上可以把数据结构分成（　）。

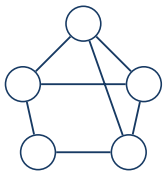
A. 动态结构和静态结构　　B. 紧凑结构和非紧凑结构

C. 线性结构和非线性结构　　D. 内部结构和外部结构

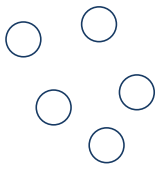
2. 基础　（课件题）与数据元素本身的形式、内容、相对位置、个数无关的是数据的（　）。

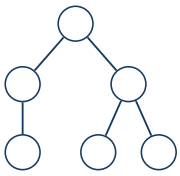
A. 存储结构　　B. 存储实现　　C. 逻辑结构　　D. 运算实现

3. 基础　下面四幅逻辑结构图中，表示**树形结构**的是（　）。

A. 

B. 

C. 

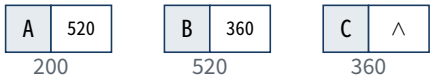
D. 

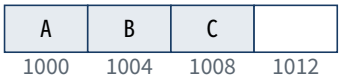
4. 基础　生活情境　班级花名册如下。图中红框圈出的“袁秋慧”这一个格子，属于（　）。

学号	姓名	性别
98165	卢声凯	男
98182	袁秋慧	女
98224	洪伟	男

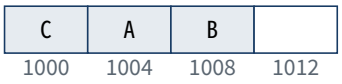
A. 数据对象　　B. 数据元素　　C. 数据项　　D. 数据类型

5. 基础　线性结构 (A, B, C) 采用**顺序存储**，每个元素占 4 个字节。下列内存示意图中正确的是（　）。（格子下方的数字是地址，$\wedge$ 表示空。）

A. 

B. 

C. 

D. 

6. 基础　“将实体的外部特性和其内部实现细节分离，并且对外部用户隐藏其内部实现细节”，这是抽象数据类型的（　）。

A. 抽象性　　B. 封装性　　C. 确定性　　D. 有穷性

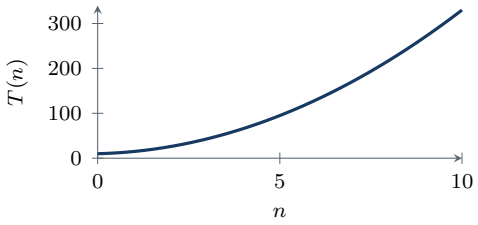
7. 基础　对程序段 `s=0; for (i=0; i<n; i++) s=s+i;`（$n > 0$），语句 `s=s+i` 的执行次数是（　）。

跟踪模板（自行填写）：

$i = 0$	$i = 1$	$\cdots$	$i = n - 1$	$i = n$
进入循环体?	进入循环体?	进入循环体?	进入循环体?	进入循环体?

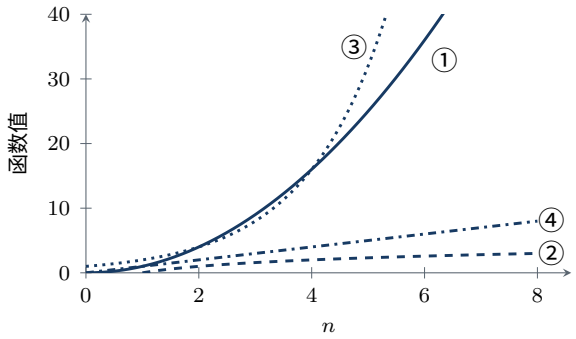
A. $n + 1$　　B. $2n$　　C. $n - 1$　　D. n

8. 基础　某算法的时间频度为 $T(n) = 3n^2 + 2n + 10$，其时间复杂度通常记为（　）。



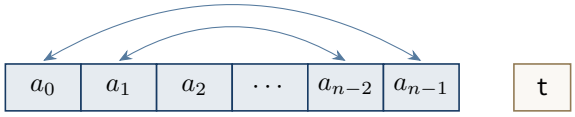
A. $O(n^2)$　　B. $O(n)$　　C. $O(3n^2)$　　D. $O(n^3)$

9. 基础　下图在同一坐标系中画出了 n、n^2、2^n、$\log_2 n$ 四个函数的图像，其中表示 $\log_2 n$ 的曲线是（　）。



A. ①　　B. ②　　C. ③　　D. ④

10. 基础　用一个临时变量 t 把数组 a[0..n-1] 的首尾元素两两交换，实现逆置（如图）。该算法的空间复杂度是（　）。



A. $O(1)$　　B. $O(\log_2 n)$　　C. $O(n)$　　D. $O(n^2)$

二、多项选择题（第 11–15 题）

11. 基础　下列问题中，属于**非数值计算**问题的有（　）。

A. 在电话号码簿中按姓名查号码
B. 井字棋人机对弈
C. 用常微分方程预测人口增长
D. 为 n 个小区选择造价最低的煤气管道铺设方案

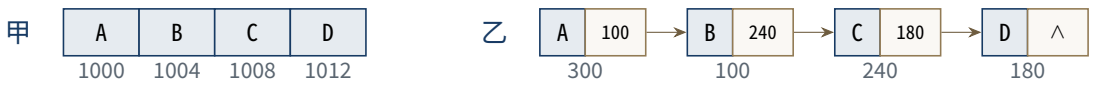
12. 基础　关于算法，下列说法正确的有（　）。

A. 算法必须在有限步之后结束
B. 算法的每一步必须有确定的含义
C. 算法可以有零个输入
D. 算法必须用 C++ 语言描述

13. 基础　当 n 足够大时，下列函数中增长得比 n^2 **更快**的有（　）。

A. n^3　　B. $n\log_2 n$　　C. 2^n　　D. $n!$

14. 基础　同一个线性结构 (A, B, C, D) 有甲、乙两种存储方式（如图）。下列说法正确的有（　）。



A. 乙方式要求各结点的存储空间必须连续
B. 甲方式用一块连续空间存放元素
C. 乙方式需要额外的空间保存元素之间的关系
D. 甲、乙表示的是同一个逻辑结构

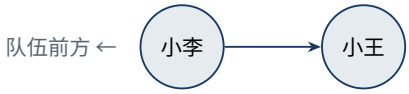
15. 基础　下列程序段中，时间复杂度为 $O(n^2)$ 的有（　）。

A. `for (i=0;i<n;i++) for (j=0;j<n;j++) x++;`
B. `for (i=1;i<=n;i=i*2) x++;`
C. `for (i=0;i<n;i++) for (j=0;j<i;j++) x++;`
D. `for (i=0;i<n;i++) x++; for (i=0;i<n;i++) for (j=0;j<n;j++) x++;`

三、填空题（第 16–20 题）

16. 基础　在类模板的成员函数声明 `void find(const elemType &x) const;` 中，参数表**后面**的 `const` 保护的是_____的值不被修改。

17. 基础　生活情境　食堂排队时，小王紧跟在小李的后面，用有序对记作 ⟨小李, 小王⟩。在这个线性结构中，小李称为小王的_____，小王称为小李的_____。



18. 基础　生活情境　（课件题）教室有 n 排、每排 m 个座位，下面的程序段把座位表逐个清零。它的时间复杂度为_____。（按课件要求填写类似 `O(1)`、`O(m*n)`、`O(log2(n))`、`O(n^3)`、`O(sqrt(n))` 的形式，下同。）

```
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        a[i][j] = 0;
```



19. 基础　（课件题）下面程序段的时间复杂度为_____。

```
i = 1;
while (i <= n)
    i = i * 3;
```



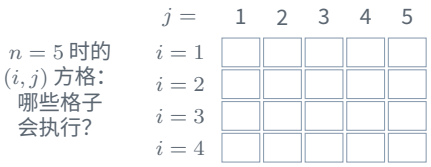
20. 基础　（课件题）下面程序段的时间复杂度为_____。

```
x = 90;  y = 100;
while (y > 0)
    if (x > 100) { x = x - 10;  y--; }
    else  x++;
```

四、解答题（第 21–25 题）

21. 基础　（课件作业 1.3）求下面程序段中语句 `x=x-1` 的执行次数（用 n 表示）。

```
for (i = 1; i <= n-1; i++)
    for (j = i+1; j <= n; j++)
        x = x - 1;
```



22. 基础　设 $D=\{a,b,c,d\}$。（1）若 $R_1=\{\langle a,b\rangle,\langle b,c\rangle,\langle c,d\rangle\}$，画出逻辑结构图，指出它属于哪种逻辑结构；（2）若 $R_2=\{\langle a,b\rangle,\langle a,c\rangle,\langle c,d\rangle\}$，画出逻辑结构图，指出它属于哪种逻辑结构，并写出每个元素的前驱个数。

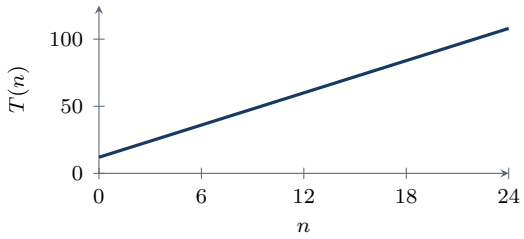
23. 基础　对下面的程序段（$n>0$），逐句统计各部分的执行次数（把次数填入图中括号），写出时间频度 $T(n)$ 和时间复杂度；并求 $n=5$ 时 $T(n)$ 的值。

```
s = 0;      ← (    ) 次

        (  ) 次   (  ) 次   (  ) 次
          ↓         ↓         ↓
for ( i = 0 ;  i < n ;  i++ )

    s = s + i;    ← (    ) 次
```

24. 基础　某算法的时间频度为 $T(n)=4n+12$（图像如下）。（1）写出它的时间复杂度；（2）按大 O 的定义，找出一组正常数 C 和 N_0，使得 $n>N_0$ 时 $T(n)\le C\cdot f(n)$。



25. 基础　把含 n 个元素的数组 `a` 逆置，有两种做法。做法一：`for (i=0; i<n/2; i++) { t=a[i]; a[i]=a[n-i-1]; a[n-i-1]=t; }`；做法二：先用 `for (i=0; i<n; i++) b[i]=a[n-i-1];` 倒着抄到数组 `b`，再用 `for (i=0; i<n; i++) a[i]=b[i];` 抄回。（1）分别写出两种做法的空间复杂度；（2）分别写出两种做法的时间复杂度；（3）哪种做法更好？说明理由。

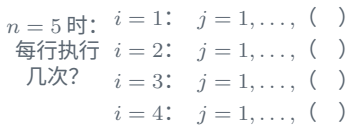


第 15 节　B 层：中等偏下提升训练（10 道）

一、单项选择题（第 26–28 题）

26. 提升　（课件题）下面程序段的时间复杂度是（　）。

```
x = 0;
for (i = 1; i < n; i++)
    for (j = 1; j <= n-i; j++)
        x++;
```



A. $O(n)$　　B. $O(n\log_2 n)$　　C. $O(n^2)$　　D. $O(n^3)$

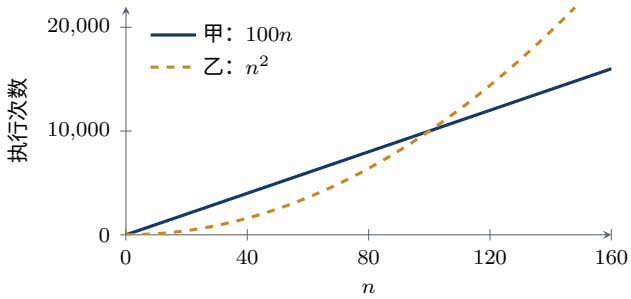
27. 提升　（课件题）下面程序段（$n>1$）的时间复杂度是（　）。

```
x = n;  y = 0;
while (x >= (y+1)*(y+1))
    y++;
```



A. $O(\log_2 n)$　　B. $O(\sqrt{n})$　　C. $O(n)$　　D. $O(n^2)$

28. 提升　解决同一个问题的两个算法：甲的时间频度 $T_{甲}(n)=100n$，乙的时间频度 $T_{乙}(n)=n^2$，图像如下。下列说法正确的是（　）。



A. 对任何 n，甲都比乙快　　B. 乙的时间复杂度比甲低

C. 甲和乙的时间复杂度同阶　　D. 当 $n>100$ 时，甲比乙快

二、多项选择题（第 29–30 题）

29. 提升　按照大 O 的**定义**（存在正常数 C、N_0，使 $n>N_0$ 时 $T(n)\le C\cdot f(n)$），下列结论成立的有（　）。

A. $2n+3=O(n)$　　B. $n=O(n^2)$

C. $n^2=O(n)$　　D. $O(\log_2 n)$ 与 $O(\log_{10} n)$ 是同一个阶

30. 提升　对下面的程序段，说法正确的有（　）。

```
for (i = 1; i <= n; i = i*2)
    for (j = 0; j < n; j++)
        x++;
```

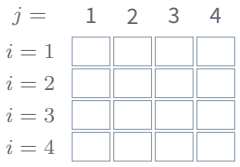
A. 外层循环体约执行 $\log_2 n$ 次
B. 语句 x++ 共执行 n^2 次
C. 外层每执行一次，内层循环体执行 n 次
D. 时间复杂度为 $O(n\log_2 n)$

三、填空题（第 31–33 题）

31. 提升　（课件题）下面程序段的时间复杂度为_____。

```
s = 0;
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        s += B[i][j];
sum = s;
```

32. 提升　对程序段 `for (i=1; i<=n; i++) for (j=1; j<=i; j++) x++;`，当 $n = 4$ 时 x++ 共执行_____次；对一般的 n，共执行_____次。



(i, j) 方格：自行涂出会执行的格子

33. 提升　某算法的时间频度为 $T(n) = 2^n$。当数据规模从 n 增加到 $n+1$ 时，执行次数变为原来的_____倍；从 n 增加到 $n+10$ 时，变为原来的_____倍。

四、综合大题（第 34–35 题）

34. 提升　生活情境　（电话号码查询，改编自课件第 15 页）一个电话号码簿用数组依次存放了 n 个人的记录（姓名，号码）。给定一个姓名，要求找出对应的号码。



(1) 这些记录之间是哪一种逻辑结构？一条记录、记录中的“姓名”分别对应第 2 节的哪个概念？

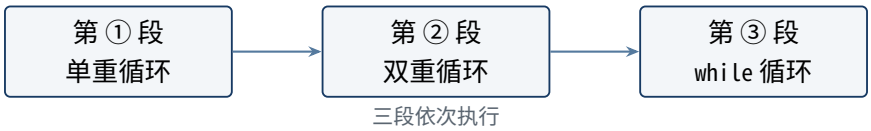
(2) **顺序查找**：从下标 0 开始逐个比较姓名，找到即停。最好和最坏情况各需比较多少次？最坏情况的时间复杂度是多少？

(3) 若记录已按姓名排好序，可用**折半查找**：每次与当前范围正中间的记录比较，相等则找到；否则根据大小舍去一半范围，在剩下的一半中继续。当 $n = 1024$ 时，最坏情况最多比较多少次？

(4) 当 n 从 1024 增大到 1024×1024（约 10^6）时，两种查找在最坏情况下的比较次数各变为多少？由此说明笔记中“$O(\log n)$ 优于 $O(n)$”的含义。

35. 提升　阅读下面的程序段（a 是含 n 个整数的数组，n 是 2 的整数次幂），回答问题。

```
s = 0;
for (i = 0; i < n; i++)                 // 第 ① 段
    s = s + a[i];
for (i = 0; i < n; i++)                 // 第 ② 段
    for (j = i; j < n; j++)
        if (a[i] > a[j]) cnt++;
k = n;
while (k > 1)                           // 第 ③ 段
    k = k / 2;
```



三段依次执行

(1) 分别写出三段中循环体（第 ② 段指 `if` 语句）的执行次数：第 ①、② 段用 n 表示；第 ③ 段先算 $n = 16$ 的情形，再写出一般结果。

(2) 整个程序段的时间复杂度是多少？用了哪个定理？

(3) 当 n 很大时，若把 n 增大到原来的 2 倍，运行时间大约变为原来的几倍？

(4) 若把第 ② 段的内层循环改为 `for (j = 0; j < 10; j++)`，整个程序段的时间复杂度变为多少？

第 16 节　C 层：跨学科迁移训练（3 道）

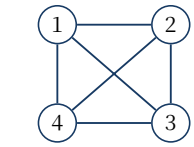
单项选择题（第 36 题）· 多项选择题（第 37 题）· 填空题（第 38 题）

36. 跨学科　（× 物理测量）一张纸厚 0.1 mm。把它对折 1 次变成 2 层，再对折 1 次变成 4 层……每对折一次，层数变为原来的 2 倍，总厚度 = 层数 × 单层厚度。假设能够连续对折 10 次，总厚度最接近（　）。

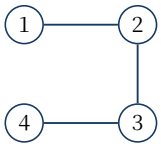


A. 1 mm　　B. 2 mm　　C. 10 cm　　D. 10 m

37. 跨学科　（× 通信网络）要在 n 个站点之间建立通信网。方案一：任意两个站点之间都直接拉一条线路（下图是 $n=4$ 的情形，共 6 条）。方案二：只要求所有站点连通，用尽可能少的线路。已知：把 n 个点连通，最少需要 $n-1$ 条线路。下列说法正确的有（　）。



方案一（$n=4$）：6 条



方案二（$n=4$）：3 条

A. 方案一中，$n=5$ 时需要 10 条线路

B. 方案一的线路数为 $\dfrac{n(n-1)}{2}$，属于平方阶

C. 方案一中，当 n 较大时，站点数增加到原来的 2 倍，线路数大约变为原来的 4 倍

D. 方案二的线路数与方案一的线路数是同阶的

38. 跨学科　（× 概率统计）在含 n 个互不相同元素的数组中做顺序查找：目标在第 k 个位置时，需要比较 k 次。假设目标一定在数组中，且出现在每个位置的可能性相同，则“平均比较次数”等于各种情况比较次数的算术平均值，即 $\dfrac{1+2+\cdots+n}{n}$。当 $n=99$ 时，平均比较次数为_____。

	位置 1	位置 2	位置 3	···	位置 n
目标在该位置时：	比较 1 次	比较 2 次	比较 3 次	···	比较 n 次

第五部分　答案速查与逐题图解

第 17 节　答案速查

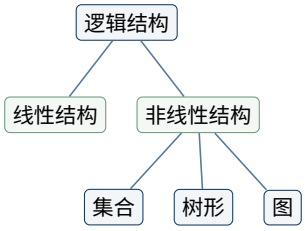
基础单选	1 C　2 C　3 D　4 C　5 B　6 B　7 D　8 A　9 B　10 A
基础多选	11 ABD　12 ABC　13 ACD　14 BCD　15 ACD
基础填空	16 调用该函数的对象　17 前驱；后继　18 `O(m*n)`　19 `O(log3(n))`（与 `O(log2(n))` 同阶）　20 `O(1)`
基础解答	21 $\dfrac{n(n-1)}{2}$ 次　22 （1）线性结构；（2）树形结构，前驱个数依次为 a:0、b:1、c:1、d:1　23 $T(n)=3n+3$，$O(n)$，$T(5)=18$　24 $O(n)$；可取 $C=5,\ N_0=12$（不唯一）　25 空间 $O(1)$、$O(n)$；时间均为 $O(n)$；做法一更好
提升单选	26 C　27 B　28 D
提升多选	29 ABD　30 ACD
提升填空	31 `O(n^2)`　32 10；$\dfrac{n(n+1)}{2}$　33 2；1024
提升大题	34 （1）线性结构；数据元素、数据项　（2）最好 1 次、最坏 n 次，$O(n)$　（3）11 次　（4）顺序 1 048 576 次，折半 21 次 35 （1）n；$\dfrac{n(n+1)}{2}$；4 次，一般为 $\log_2 n$ 次　（2）$O(n^2)$，求和定理　（3）约 4 倍　（4）$O(n)$
跨学科	36 C　37 ABC　38 50

第 18 节　A 层详细解析

基础单选（第 1–10 题）

第 1 题　答案 C　基础　考点：逻辑结构的分类（第 3 节）

依据　逻辑结构按元素间的关系分为集合、线性、树形、图四种；其中只有线性结构是“一对一”，其余三种统称非线性结构。



逐项　A“动态 / 静态”、B“紧凑 / 非紧凑”、D“内部 / 外部”都不是按**元素间的逻辑关系**划分的，与题干“从逻辑上”不符；C 正确。

常见错误　把“顺序结构和链式结构”当成逻辑上的分类——那是存储结构。

一句话 逻辑上只分两大类：线性、非线性。

第 2 题　答案 C　基础　考点：逻辑结构的含义（第 3 节）

依据 逻辑结构只描述元素之间的关系，“是事物本身的性质，和计算机没有关系”（课件第 24 页）。

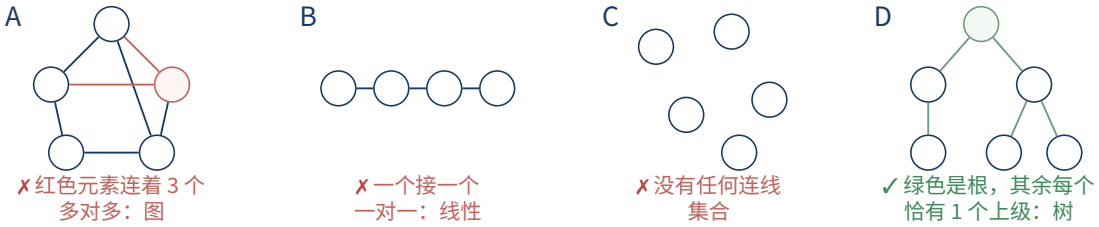


逐项 A 存储结构、B 存储实现与元素在内存中的位置、所占空间直接相关；D 运算实现依赖具体的存储方式；只有 C 逻辑结构与这些都无关。

一句话 换一批元素、换个个数，“线性”还是“线性”。

第 3 题　答案 D　基础　考点：四种逻辑结构的图形（第 3 节）

判断方法 数每个元素“上面”连着几个元素：都不超过 1 个且有唯一的顶端元素，就是树。



常见错误 看到 A 也“有层次”就选 A。A 中红色元素同时与上面一个、同层一个、下面一个相连，出现了回路，是图。

一句话 树里没有“圈”，每个元素只有一个上级。

第 4 题　答案 C　基础　考点：数据元素与数据项（第 2 节）

学号	姓名	性别
98165	卢声凯	男
98182	袁秋慧	女
98224	洪伟	男

三行合在一起 = 数据对象（A）
蓝色虚框：一整行 = 数据元素（B）
红框：一个格子 = **数据项**（C）

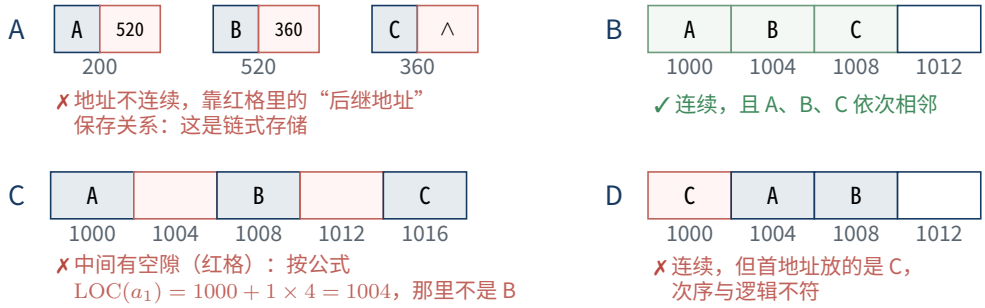
逐项 A 数据对象是全体记录的集合；B 数据元素是一整行；D 数据类型是“值的集合加操作”，如字符串类型，而不是某个具体的值；C 正确：姓名是一条记录中不可再分的一栏。

常见错误 把“一个格子”当成数据元素。

一句话 元素是行，数据项是格。

第 5 题　答案 B　基础　考点：顺序存储的特征（第 4 节）

依据　顺序存储要同时满足：（1）占用一块**连续**空间；（2）元素的物理次序与逻辑次序**一致**，因为关系就是靠位置表示的。



常见错误　只看“是否连续”就选 D，忽略了次序。

一句话　顺序存储 = 连续 + 次序一致。

第 6 题　答案 B　基础　考点：ADT 的两个特性（第 5 节）

依据　题干是课件第 34 页对**封装性**的原话。抽象性强调“本质特征、所能完成的功能以及对外接口”。



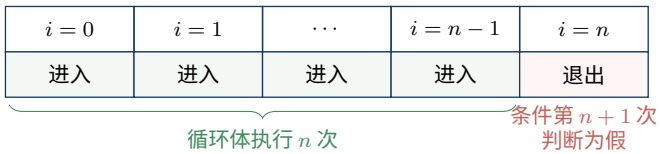
逐项　A 抽象性说的是“强调什么”，不是“隐藏什么”；C 确定性、D 有穷性是**算法**的特性，不是 ADT 的特性；B 正确。

常见错误　抽象性、封装性互相记反。

一句话　抽象：只说做什么；封装：藏起怎么做。

第 7 题　答案 D　基础　考点：循环体执行次数（第 7 节）

思路　循环变量 i 依次取 $0, 1, \ldots, n-1$ 时条件 i<n 为真，各进入一次循环体；取 n 时为假，退出。



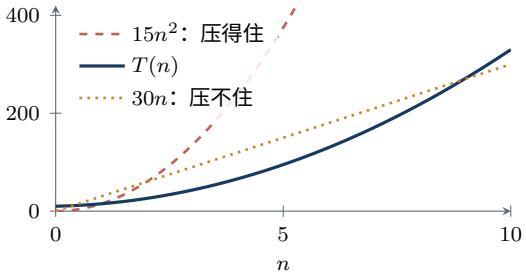
逐项　A（$n+1$）是条件 i<n 的判断次数，不是循环体的次数；B（$2n$）没有依据；C（$n-1$）漏数了 $i=0$ 那一次；D 正确。

常见错误　把“条件判断次数”与“循环体执行次数”混淆。

一句话　i 从 0 到 $n-1$，共 n 个值，循环体 n 次。

第 8 题　答案 A　基础　考点：最高次项、去系数（第 8 节）

做法　最高次项是 $3n^2$，去掉系数 3，得 $O(n^2)$。验证：$n \ge 1$ 时 $T(n) \le 3n^2 + 2n^2 + 10n^2 = 15n^2$，取 $C = 15$、$N_0 = 1$。



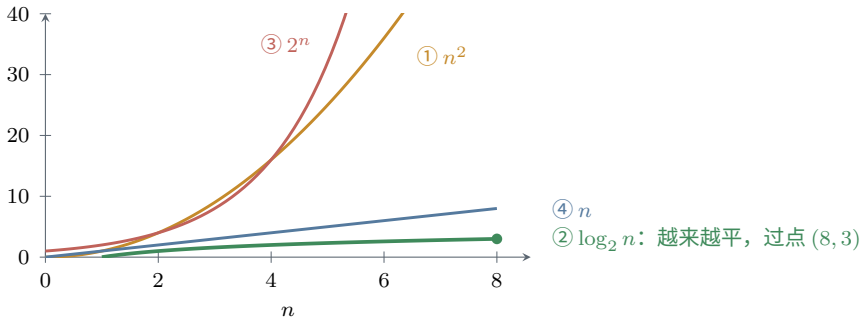
逐项　A 正确。B：任何一条直线 $C \cdot n$（图中点线取 $C = 30$）在 n 足够大后都会被 $T(n)$ 超过，所以 $T(n)$ 不是 $O(n)$。C：系数不应保留，写法不规范。D：$O(n^3)$ 作为上界虽然成立，但太松，不是“通常记为”的最紧的阶。

常见错误　保留系数写成 $O(3n^2)$。

一句话　最高次项、去系数。

第 9 题　答案 B　基础　考点：常见阶的图像（第 9 节）

判断方法　看走势：直线是 n；越来越陡的两条中更早冲出图框的是 2^n，另一条是 n^2；越来越平的是 $\log_2 n$。



图上证据　曲线 ② 经过点 $(8,3)$：$\log_2 8 = 3$；它在 $n = 1$ 处为 0，且越往右越平。

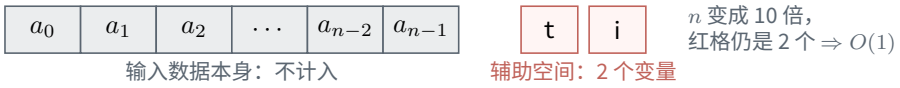
逐项　① 过 $(4,16)$、$(6,36)$，是 n^2；③ 过 $(4,16)$、$(5,32)$，是 2^n；④ 是过原点的直线 n；② 是 $\log_2 n$。

常见错误　把最平的曲线当成“常数”。它仍在缓慢上升。

一句话　对数曲线：先升后平，永不封顶。

第 10 题　答案 A　基础　考点：空间复杂度（第 11 节）

依据 空间复杂度只统计辅助空间。本算法除数组本身外，只用了临时变量 t（和循环变量 i），个数与 n 无关。



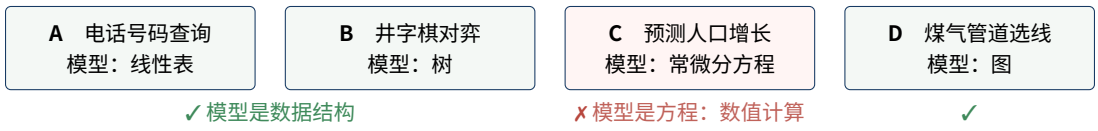
逐项 A 正确。C（$O(n)$）是把数组 a 本身也算了进去，或者想成了“借助另一个数组”的做法；B、D 没有依据。

常见错误 把时间复杂度 $O(n)$ 当成空间复杂度。

一句话 只数“额外借用”的空间。

基础多选（第 11–15 题）

第 11 题　答案 ABD　基础　考点：数值计算与非数值计算（第 1 节）



A 正确：记录一条接一条，模型是线性表（课件第 15 页）。

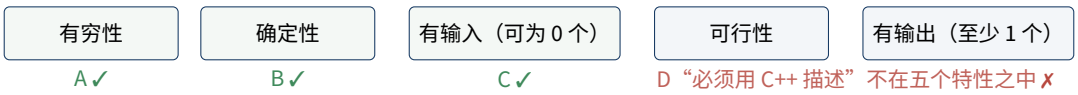
B 正确：棋局层层展开，模型是树（第 16 页）。

C 错误：课件第 14 页明确把它作为数值计算的例子，模型是常微分方程。

D 正确：小区是点、管线是带造价的边，模型是图（第 17 页）。

一句话 看模型：是方程就是数值计算，是表、树、图就是非数值计算。

第 12 题　答案 ABC　基础　考点：算法的五个特性（第 6 节）



A 正确：有穷性。

B 正确：确定性。

C 正确：输入可以是零个或多个。

D 错误：算法是解决问题的方法和步骤，可以用自然语言、流程图、伪代码或任何编程语言描述。

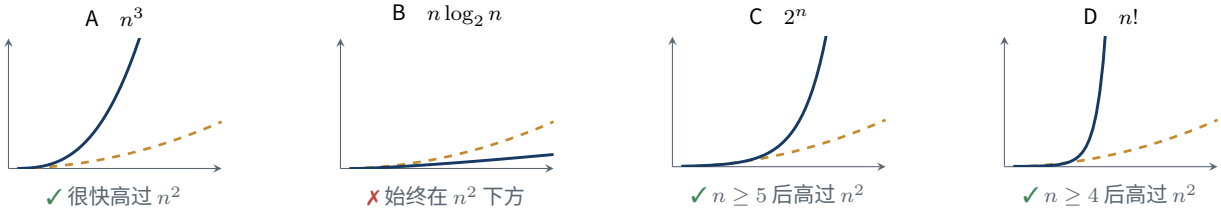
对照说明 D 来自“写程序必须选一种语言”的习惯，这个结论迁移到算法上**不成立**（第 6 节对

照表）。

一句话 算法不挑语言。

第 13 题　答案 ACD　基础　考点：常见阶的大小关系（第 9 节）

依据 $n\log_2 n < n^2 < n^3 < 2^n < n!$（$n$ 足够大时）。四幅小图使用同一坐标范围，橙色虚线都是 n^2，蓝线是选项中的函数。



A 正确：$n^3/n^2 = n \to \infty$。

B 错误：因为 $\log_2 n < n$，两边同乘 n 得 $n\log_2 n < n^2$。

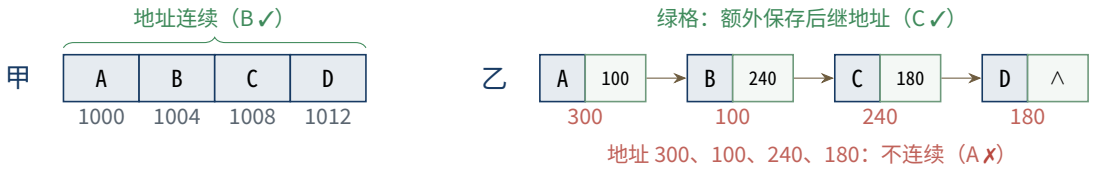
C 正确：$n = 10$ 时 $2^{10} = 1024 > 100$，且差距越来越大。

D 正确：$n = 5$ 时 $5! = 120 > 25$。

常见错误 看到 $n\log_2 n$ 里“有两个因子”就以为比 n^2 大。

一句话 $n\log_2 n$ 夹在 n 和 n^2 之间。

第 14 题　答案 BCD　基础　考点：顺序存储与链式存储（第 4 节）



A 错误：乙是链式存储，各结点地址 300、100、240、180 并不连续，也不要求连续。

B 正确：甲的四个元素占 1000–1015 这一段连续空间。

C 正确：乙的每个结点多出一格存后继地址。

D 正确：两图按逻辑次序读出来都是 $A \to B \to C \to D$，是同一个线性结构的两种存储。

一句话 一个逻辑结构可以有多种存储结构。

第 15 题　答案 ACD　基础　考点：循环的形状（第 10 节）

做法　取 $n=5$，把每个选项循环体的执行情况画成点阵（一个点 = 执行一次）。

A　独立嵌套



✓ $5\times5=25$

B　$i=1,2,4$

✗ 只有 3 次

C　关联嵌套



✓ $0+1+2+3+4=10$

D　顺序两段



✓ $5+25$：取大

A 正确：求积定理，$n\times n$。

B 错误：i 每次乘 2，约 $\log_2 n$ 次，$O(\log_2 n)$。

C 正确：$\frac{n(n-1)}{2}$ 次，最高次项 $\frac{1}{2}n^2$。

D 正确：求和定理，$O(\max(n,n^2))=O(n^2)$。

对照说明　A 是乘法原理的直接迁移（内层与外层无关，✓）；C 的内层依赖外层，乘法原理要加条件（△），改用求和。

常见错误　认为 C 的次数比 A 少一半，所以不是 $O(n^2)$——系数 $\frac{1}{2}$ 不影响阶。

一句话　方阵和三角形都是平方阶。

基础填空（第 16–20 题）

第 16 题　答案：调用该函数的对象　基础　考点：两种 const（第 12 节）

```
void find ( const elemType &x ) const ;
```

参数前的 const：保护参数 x

参数表后的 const：保护调用该函数的对象

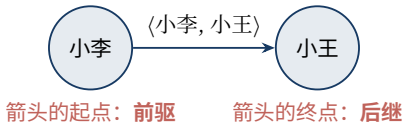
调用 obj1.find(a); 时：前者保护实参 a，后者保护 obj1

推理　课件第 80 页：参数表后的 `const` 保护调用它的对象的值不能被改变；语句 `obj1.find(a)` 中，它保护的就是对象 `obj1`。

可接受的等价答案　“调用它的对象”“对象的数据成员”“`obj1`（调用者）”。

易错点　填成“参数 x”——那是参数**前面**那个 `const` 的作用。

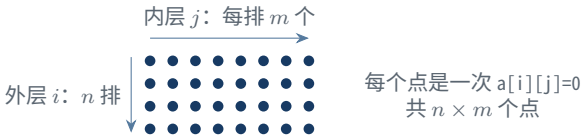
第 17 题　答案：前驱；后继　基础　考点：有序对（第 3 节）



依据　对有序对 $\langle u, v\rangle$，u 称为 v 的直接前驱，v 称为 u 的直接后继。这里 u =小李，v =小王。

易错点　按“排在后面的人是前驱”去想，把两个词填反。前驱是“排在它前面的那个”。

第 18 题　答案：O(m*n)　基础　考点：独立嵌套、求积定理（第 10 节）

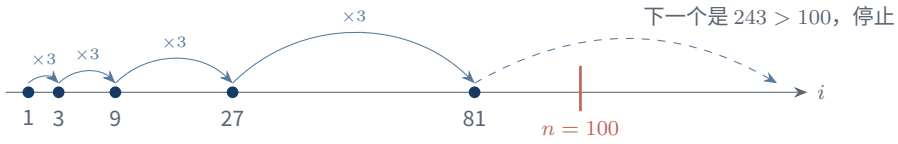


推理　外层执行 n 次，每次内层执行 m 次，且内层次数与 i 无关，循环体共执行 $n \times m$ 次，时间复杂度 $O(m \times n)$。

易错点　写成 $O(n^2)$。m 与 n 是两个独立的量，不能合并成一个字母。

等价答案　O(n*m)。

第 19 题　答案：O(log3(n))　基础　考点：倍增循环（第 10 节）

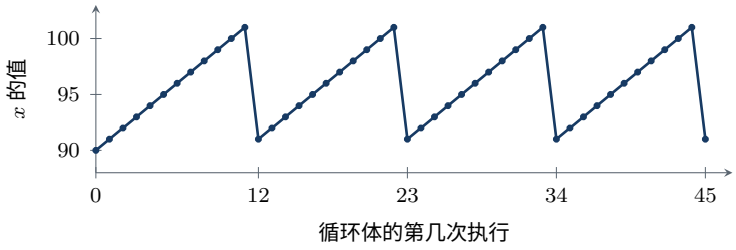


推理　设循环体执行 k 次，此时 $i = 3^k$。循环在 $3^k > n$ 时停止，$k = \lfloor \log_3 n \rfloor + 1$，是对数阶。图中 $n = 100$：i 取 1、3、9、27、81 时进入循环体，共 5 次，$\lfloor \log_3 100 \rfloor + 1 = 5$。

等价答案　$\log_3 n = \dfrac{\log_2 n}{\log_2 3}$，只差常数倍，所以 O(log2(n))、$O(\log n)$ 与它同阶，都算对；按课件的填写格式，最贴近代码的是 O(log3(n))。

易错点　看到 while(i<=n) 就填 $O(n)$——没注意 i 是乘 3 而不是加 1。

第 20 题　答案：O(1)　基础　考点：与 n 无关的循环（第 10 节）



每一“齿”：x 加到 101，然后 x 减 10、y 减 1
共 100 齿（y 从 100 减到 0）

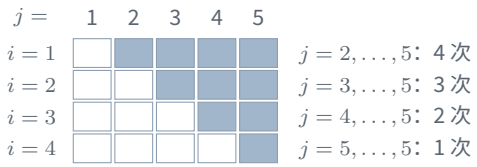
推理　程序中没有出现 n。跟踪：x 从 90 逐次加 1 到 101（11 次），然后执行一次“x 减 10、y 减 1”；此后每一轮是 10 次加 1 再加 1 次减法，共 11 次。y 要从 100 减到 0，共 100 轮，循环体总共执行 $12 + 99 \times 11 = 1101$ 次。这是一个固定的数，与数据规模无关，所以是 $O(1)$。

易错点　因为“循环了一千多次”或者“有 `while`”就填 $O(n)$。判断标准是次数**是否随 n 变化**，而不是次数多少。

基础解答（第 21–25 题）

第 21 题　答案：$\dfrac{n(n-1)}{2}$ 次　基础　考点：关联嵌套、等差数列求和（第 7、10 节）

题意分析　外层 i 从 1 到 $n-1$；对每个 i，内层 j 从 $i+1$ 到 n，共 $n-(i+1)+1 = n-i$ 次。



$n=5$：$4+3+2+1 = 10 = \dfrac{5\times 4}{2}$
涂色部分是对角线右上方的三角形

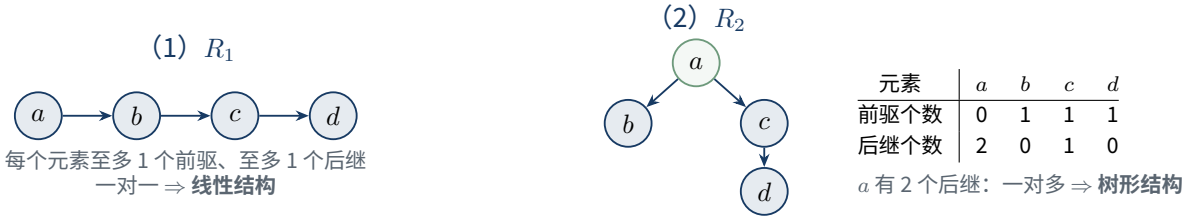
分步求解　第一步，写出每个 i 对应的次数：$i=1$ 时 $n-1$ 次，$i=2$ 时 $n-2$ 次，……，$i=n-1$ 时 1 次。第二步，求和——这是首项 $n-1$、末项 1、共 $n-1$ 项的等差数列：

$$\sum_{i=1}^{n-1}(n-i) = (n-1)+(n-2)+\cdots+1 = \frac{[(n-1)+1](n-1)}{2} = \frac{n(n-1)}{2}.$$

结果检查　$n=5$ 时公式给出 10，与图中涂色格数一致；$n=2$ 时给出 1，代码确实只执行一次（$i=1, j=2$）。

易错点　直接写 $(n-1)\times n$，把内层当成与 i 无关。
同类题　先写“第 i 轮内层几次”，再对 i 求和。

第 22 题　答案：（1）线性结构；（2）树形结构　基础　考点：二元组与逻辑结构（第 3 节）



元素	a	b	c	d
前驱个数	0	1	1	1
后继个数	2	0	1	0

a 有 2 个后继：一对多 ⇒ **树形结构**

分步求解　第一步，每个有序对画一条从前驱指向后继的箭头。第二步，数每个元素的前驱个数和后继个数。R_1 中每个元素前后各至多一个，是线性结构。R_2 中 a 没有前驱（是根），b, c, d 各恰有一个前驱，而 a 有两个后继，是树形结构。

易错点　(2) 中因为 $a \to c \to d$ 是一条“线”就判断成线性——还要看 a 另有后继 b。

同类题　先画图，再数前驱、后继。

第 23 题　答案：$T(n) = 3n + 3$，$O(n)$，$T(5) = 18$　基础　考点：逐句统计（第 7 节）

```
s = 0;      ← (1) 次

        (1) 次   (n+1) 次   (n) 次
          ↓        ↓        ↓
for ( i = 0 ;  i < n ;  i++ )

    s = s + i;    ← (n) 次
```

$n = 5$ 时	次数
s=0	1
i=0	1
i<n	6
i++	5
s=s+i	5
合计	18

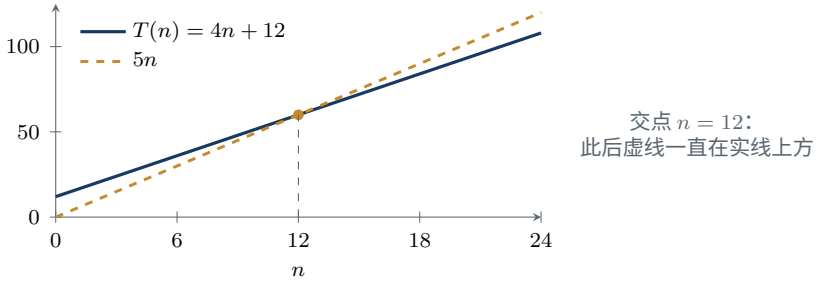
分步求解　s=0 和 i=0 各 1 次；i<n 在 $i = 0, 1, \ldots, n$ 时各判断一次，共 $n + 1$ 次；i++ 与 s=s+i 各 n 次。相加得 $T(n) = 1 + 1 + (n + 1) + n + n = 3n + 3$。最高次项 $3n$，去系数，时间复杂度 $O(n)$。代入 $n = 5$：$T(5) = 3 \times 5 + 3 = 18$。

易错点　i<n 填成 n 次，得到错误的 $3n + 2$。

同类题　循环条件永远比循环体多判断一次。

第 24 题　答案：$O(n)$；如 $C = 5,\ N_0 = 12$　基础　考点：大 O 的定义（第 8 节）

知识定位　最高次项 $4n$，去系数得 $f(n) = n$，时间复杂度 $O(n)$。下面按定义找 C、N_0。

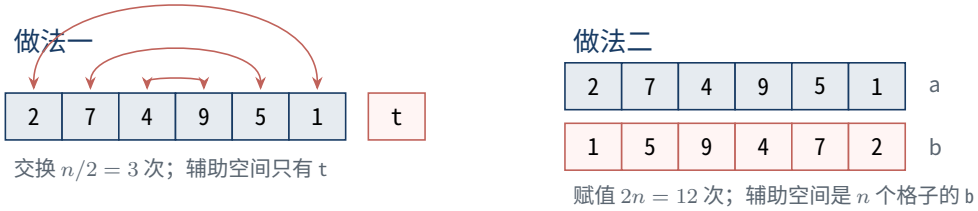


分步求解　希望 $4n + 12 \le C \cdot n$。取 $C = 5$：$4n + 12 \le 5n \iff n \ge 12$。所以取 $C = 5$、$N_0 = 12$，当 $n > N_0$ 时不等式成立。

另一组答案 用“把低次项放大”的办法：$n \geq 1$ 时 $12 \leq 12n$，于是 $4n + 12 \leq 16n$，取 $C = 16$、$N_0 = 1$ 也对。答案不唯一。

易错点 取 $C = 4$——那样 $4n + 12 \leq 4n$ 永远不成立。C 必须比最高次项系数大一点。

第 25 题　答案：见下　基础　考点：空间复杂度与时间复杂度的比较（第 11 节）



（1）空间 做法一只用临时变量，$S(n) = O(1)$；做法二用了 n 个元素的辅助数组，$S(n) = O(n)$。

（2）时间 做法一循环 $n/2$ 次，$O(n)$；做法二两个循环各 n 次，由求和定理 $O(\max(n, n)) = O(n)$。

（3）比较 两者时间复杂度同阶，再比较空间：$O(1)$ 优于 $O(n)$，所以**做法一更好**。

结果检查 图中 $n = 6$：做法一 3 次交换后得到 1,5,9,4,7,2，与题图“逆置后”一致。

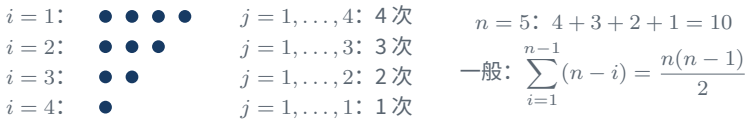
易错点 看到做法一循环次数少一半，就说它“时间复杂度更低”。$n/2$ 与 $2n$ 都是 $O(n)$。

第 19 节　B 层详细解析

提升单选（第 26–28 题）

第 26 题　答案 C　提升　考点：关联嵌套（第 10 节）

思路 内层次数 $n - i$ 随外层变量变化，要用求和。



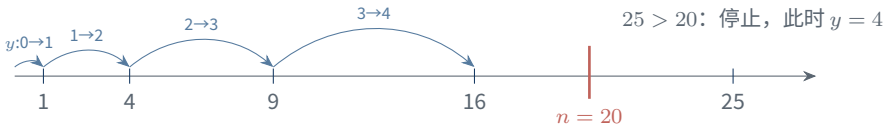
判断 执行次数 $\dfrac{n(n-1)}{2} = \dfrac{1}{2}n^2 - \dfrac{1}{2}n$，最高次项 $\dfrac{1}{2}n^2$，去系数，$O(n^2)$。

逐项 A：只看了外层的 $n - 1$ 次；B：点阵是三角形，不是“n 行、每行 $\log_2 n$ 个”；C 正确；D：只有两重循环，不会到 n^3。

常见错误 认为“内层越来越短”所以比 n^2 低一阶。三角形的面积仍是方阵的一半。
一句话 三角形点阵 $\Rightarrow$ 平方阶。

第 27 题　答案 B　提升　考点：平方逼近型循环（第 10 节）

思路 循环继续的条件是 $(y+1)^2 \le n$。每执行一次 y 加 1，直到 $(y+1)^2$ 第一次超过 n。

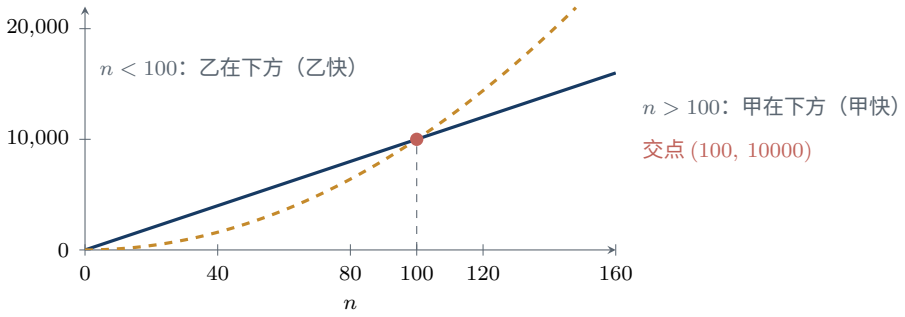


判断 结束时 $y=k$ 满足 $k^2 \le n < (k+1)^2$，即 $k=\lfloor\sqrt{n}\rfloor$。循环体执行 $\lfloor\sqrt{n}\rfloor$ 次，时间复杂度 $O(\sqrt{n})$。图中 $n=20$：依次比较 1、4、9、16（都不超过 20，各执行一次），到 25 停止，共 4 次，$\lfloor\sqrt{20}\rfloor=4$。

逐项 A：y 是每次加 1，不是成倍增长；B 正确；C：循环在 $y\approx\sqrt{n}$ 时就停了，远不到 n；D 没有依据。

常见错误 看到 y++ 就选 $O(n)$——要看的是循环**什么时候停**。
一句话 条件里是 y^2 和 n 比，次数就是 $\sqrt{n}$。

第 28 题　答案 D　提升　考点：阶的含义、系数与小规模（第 8、9 节）



解题 令 $100n=n^2$，得交点 $n=100$。$n<100$ 时 $n^2<100n$，乙的执行次数更少；$n>100$ 时 $100n<n^2$，甲更少。

逐项 A 错：图中交点左侧乙的曲线在下方，例如 $n=10$ 时甲 1000 次、乙 100 次。B 错：甲是 $O(n)$，乙是 $O(n^2)$，甲的阶更低。C 错：$\dfrac{100n}{n^2}=\dfrac{100}{n}\to 0$，不同阶。D 正确。

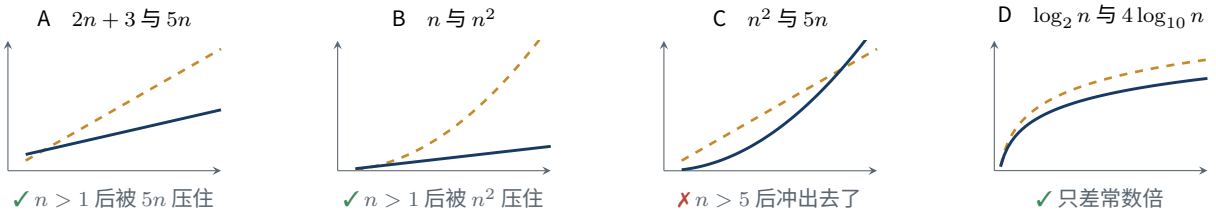
对照说明 “相差 100 倍就差两个阶”是从数量级估算带来的错误迁移（第 9 节对照表 ✗）：系数 100 不改变阶，只是把交点推到了 $n=100$。

常见错误 认为“阶低的算法永远更快”。
一句话 阶决定 n 很大时的胜负，系数决定“多大才算很大”。

提升多选（第 29–30 题）

第 29 题　答案 ABD　提升　考点：大 O 的定义与同阶（第 8 节）

做法 每个选项都问一句：能不能找到常数 C，使 n 足够大后，蓝线（左边的函数）一直在橙色虚线（C 乘以右边的函数）下方？



A 正确：$n \ge 1$ 时 $2n+3 \le 2n+3n = 5n$，取 $C=5, N_0=1$。

B 正确：$n \ge 1$ 时 $n \le n^2$，取 $C=1, N_0=1$。按定义大 O 只要求“不超过”，所以成立（只是这个上界很松）。

C 错误：若 $n^2 \le C \cdot n$ 对所有大 n 成立，则 $n \le C$，而 n 可以任意大，矛盾。图中取 $C=5$，$n>5$ 后蓝线就超过了虚线；换任何 C 都一样。

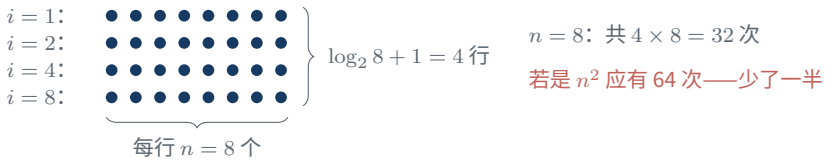
D 正确：$\log_2 n = \dfrac{\log_{10} n}{\log_{10} 2} \approx 3.32 \log_{10} n$，互为常数倍，是同一个阶。

对照说明 B 源于高数“同阶无穷大”的习惯：n 与 n^2 不同阶，于是误以为 $n = O(n^2)$ 不成立。大 O 是**单向**上界，不要求同阶（第 8 节对照表 ✗）。

一句话 小的可以被大的“O”住，大的不能被小的“O”住。

第 30 题　答案 ACD　提升　考点：倍增循环套线性循环（第 10 节）

思路 外层 i 取 $1, 2, 4, 8, \ldots$（不超过 n）；内层与 i 无关，每轮固定 n 次，可用求积定理。



A 正确：外层执行 $\lfloor \log_2 n \rfloor + 1$ 次，约 $\log_2 n$ 次。

C 正确：内层 j 从 0 到 $n-1$，与 i 无关。

B 错误：总次数是 $n(\lfloor \log_2 n \rfloor + 1)$；$n=8$ 时是 32，不是 64。

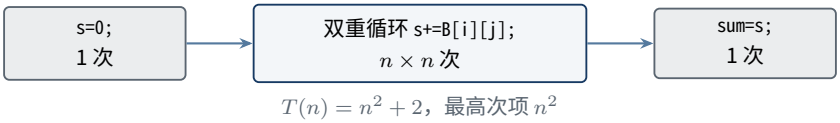
D 正确：求积定理，$O(\log_2 n \times n) = O(n \log_2 n)$。

对照说明 这里内层次数不依赖外层，乘法原理可以直接迁移（✓）；错选 B 是因为“见两重循环就乘 $n \times n$”，没有先数清每一重各几次。

一句话 先数每一重各几次，再相乘。

提升填空（第 31–33 题）

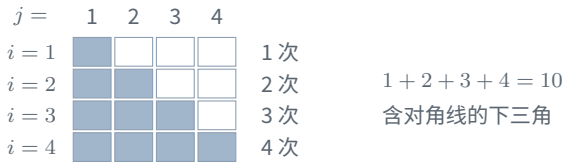
第 31 题　答案：O(n^2)　提升　考点：只看与 n 有关的循环体（第 8、10 节）



推理　s=0 与 sum=s 各执行 1 次，与 n 无关，可不计入；循环体 s+=B[i][j] 执行 $n \times n$ 次。$T(n) = n^2 + 2$，时间复杂度 $O(n^2)$。

易错点　被 sum=s; 的缩进迷惑，以为它在循环里。它没有被大括号括进任何循环，只执行一次；即使它在外层循环里（执行 n 次），由求和定理结果仍是 $O(n^2)$。

第 32 题　答案：10；$\dfrac{n(n+1)}{2}$　提升　考点：关联嵌套的精确次数（第 7、10 节）

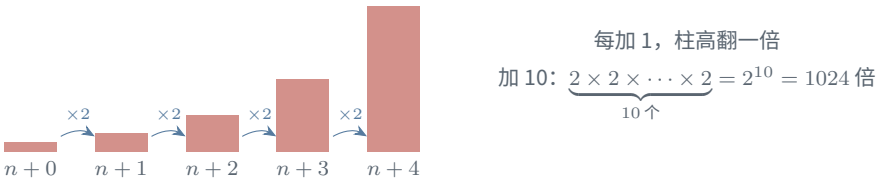


推理　外层取 i 时，内层 j 从 1 到 i，执行 i 次。总次数 $\sum_{i=1}^{n} i = 1+2+\cdots+n = \dfrac{n(n+1)}{2}$；$n=4$ 时为 $\dfrac{4\times5}{2} = 10$。

对照说明　这是等差数列求和公式的直接迁移（✓）。与第 21 题相比，本题内层是 j<=i（含对角线），所以是 $\frac{n(n+1)}{2}$ 而不是 $\frac{n(n-1)}{2}$——上下限必须对准代码（△）。

易错点　把 j<=i 看成 j<i，得到 6。

第 33 题　答案：2；1024　提升　考点：指数阶的增长方式（第 9 节）



推理　$\dfrac{T(n+1)}{T(n)} = \dfrac{2^{n+1}}{2^n} = 2$；$\dfrac{T(n+10)}{T(n)} = \dfrac{2^{n+10}}{2^n} = 2^{10} = 1024$。

意义　对比平方阶：n 从 100 增加到 110，n^2 只变为原来的 1.21 倍；而 2^n 变为 1024 倍。这就是指数阶被划入“不推荐”的原因。

易错点　第二空填 20（把“翻 10 次倍”算成 2×10）。

提升综合大题（第 34–35 题）

第 34 题　提升　考点：数据结构基本概念 + 两种查找的复杂度（第 2、3、9 节）

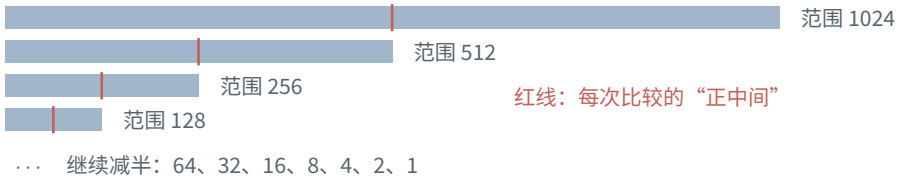
1. 题目在问什么　同一份数据、同一个任务，换一种算法，比较次数差多少。

2. 知识链　逻辑结构 → 顺序查找的 $T(n)$ → 折半查找的次数 → 两种阶的对比。

（1）　记录一条接一条排列，每条记录前后各至多一条，是**线性结构**。一条记录是一个**数据元素**；其中的“姓名”是一个**数据项**。

（2）顺序查找　最好情况：目标就在下标 0，比较 1 次。最坏情况：目标在最后或根本不在表中，比较 n 次，时间复杂度 $O(n)$。

（3）折半查找　每比较一次，待查范围至多剩下一半。



范围大小依次为 $1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1$；每种大小下最多比较一次，范围为 1 时还要比较最后一次，共 **11 次**。一般地，最坏比较次数为 $\lfloor \log_2 n \rfloor + 1$，时间复杂度 $O(\log_2 n)$。

（4）　$n = 1024 \times 1024 = 2^{20} = 1\,048\,576$ 时：顺序查找最坏 $1\,048\,576$ 次；折半查找最坏 $\lfloor \log_2 2^{20} \rfloor + 1 = 21$ 次。

规模 n	顺序查找（最坏）	折半查找（最坏）	相差倍数
1024	1024	11	约 93 倍
1 048 576	1 048 576	21	约 5 万倍

数据量变成 1024 倍，顺序查找的工作量也变成 1024 倍，折半查找只多了 10 次。这就是“$O(\log n)$ 优于 $O(n)$”：**规模越大，优势越明显**。

结果合理性检查　$2^{10} = 1024$，所以 1024 个元素减半 10 次后剩 1 个，再加最后一次比较共 11 次，吻合。

最容易卡在哪里　第（3）问把答案写成 10：忘了范围缩到 1 时还要比较一次才能确认。

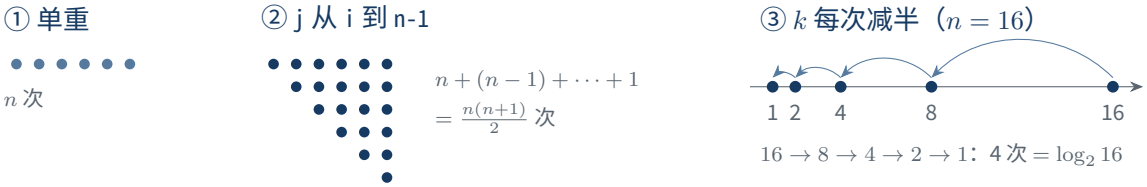
如果条件变化　若记录**没有排序**，折半查找不能用（“舍去一半”的依据是有序），只能顺序查找。

变式思考　$n = 100$ 时折半查找最坏比较几次？（7 次：$\lfloor \log_2 100 \rfloor + 1 = 6 + 1$。）

第 35 题　提升　考点：三种循环形状 + 求和定理（第 8、10 节）

1. 题目在问什么　一段由三个循环顺序拼成的程序，每段各执行多少次，整体是什么阶。

2. 知识链　单重循环 → 关联嵌套（求和）→ 减半循环（对数）→ 求和定理取最大 → 由阶推断规模翻倍的影响。



（1）　第 ① 段：循环体执行 n 次。第 ② 段：外层取 i 时内层 j 从 i 到 $n-1$，共 $n-i$ 次，`if` 语句执行 $\sum_{i=0}^{n-1}(n-i)=n+(n-1)+\cdots+1=\dfrac{n(n+1)}{2}$ 次。第 ③ 段：$n=16$ 时 k 依次为 $16,8,4,2$ 时进入循环体，变成 1 后退出，共 4 次；一般地 $n=2^m$ 时执行 $m=\log_2 n$ 次。

（2）　三段顺序执行，用**求和定理**：$O(\max(n,\ n^2,\ \log_2 n))=O(n^2)$。

（3）　n 很大时总次数约为 $\dfrac{1}{2}n^2$。规模变为 $2n$ 时约为 $\dfrac{1}{2}(2n)^2=4\times\dfrac{1}{2}n^2$，运行时间大约变为原来的 **4 倍**。

（4）　内层改为固定 10 次后，第 ② 段执行 $10n$ 次，是 $O(n)$。此时 $O(\max(n,\ n,\ \log_2 n))=O(n)$。

结果合理性检查　取 $n=4$ 验证第 ② 段：$4+3+2+1=10=\dfrac{4\times5}{2}$。

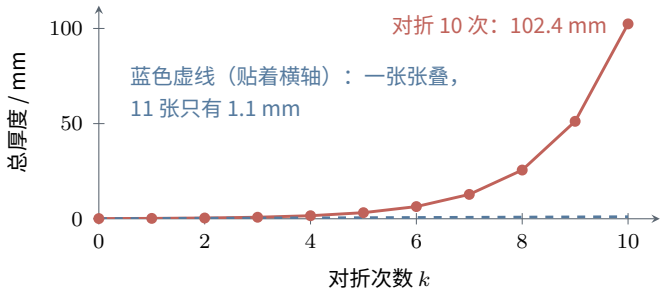
最容易卡在哪里　第（1）问第 ② 段写成 $\frac{n(n-1)}{2}$——内层从 `j=i` 开始，包含 $j=i$ 这一次。

如果条件变化　第（4）问说明：决定整体复杂度的是最慢的那一段；把它从平方阶降到线性阶，整体才跟着降。

变式思考　若把第 ③ 段改为 `while (k>1) k=k-1;`，它的执行次数是多少？整体复杂度变吗？（$n-1$ 次；整体仍是 $O(n^2)$。）

第 20 节　C 层详细解析

第 36 题　答案 C　跨学科　考点：指数增长 2^n（第 9 节）



解题　对折 k 次后层数为 2^k，总厚度 $= 0.1 \times 2^k$ mm。$k = 10$ 时：$0.1 \times 1024 = 102.4$ mm ≈ 10 cm。

逐项　A（1 mm）是按“每次加一层”的线性想法算的（11 层 ≈ 1.1 mm）；B（2 mm）是按“每次加两层”算的（20 层）；D（10 m）多算了约 100 倍，对折 17 次左右才到这个量级；C 正确。

当前专题知识　指数阶 2^n：每一步翻倍。

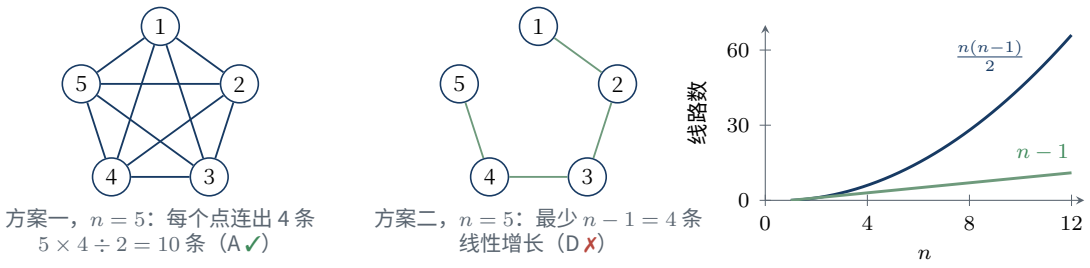
借用的外部情境　物理中的长度测量与单位换算。

为什么不算超纲　题干给出了“每对折一次层数翻倍”“总厚度 = 层数 × 单层厚度”，只需计算 2^{10} 并把毫米换成厘米。

迁移点　算法中“每多一个元素，情况数翻倍”的 2^n，与“每多折一次，层数翻倍”是同一个函数。

一句话　翻倍 10 次约等于乘以 1000。

第 37 题　答案 ABC　跨学科　考点：图结构中的计数与平方阶（第 3、10 节）



推导　方案一：每个站点要与其余 $n-1$ 个站点相连，共 $n(n-1)$ 个“端点”，每条线路有两个端点，所以线路数为 $\dfrac{n(n-1)}{2}$。

A 正确：$\dfrac{5 \times 4}{2} = 10$（左图）。

B 正确：$\dfrac{n(n-1)}{2}=\dfrac{1}{2}n^2-\dfrac{1}{2}n$，最高次项是 n^2。

C 正确：n 较大时线路数约为 $\dfrac{1}{2}n^2$，n 变为 $2n$ 则约为 $\dfrac{1}{2}(2n)^2$，是原来的 4 倍。

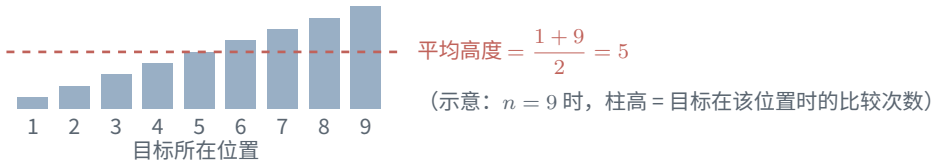
D 错误：方案二需要 $n-1$ 条，是线性阶；$\dfrac{n-1}{n(n-1)/2}=\dfrac{2}{n}\to 0$，与方案一不同阶（右图两条线越离越远）。

当前专题知识 图结构（多对多）；关联嵌套的计数 $\frac{n(n-1)}{2}$；阶的比较。
借用的外部情境 通信网络的布线。

为什么不算超纲 “两两直连”“最少 $n-1$ 条”的含义都已在题干说明，不需要任何网络专业知识。

迁移点 “n 个对象两两配对”的个数 $\frac{n(n-1)}{2}$，与双重循环 `for(i) for(j>i)` 的执行次数（第 21 题）是同一个式子。

第 38 题　答案：50　跨学科　考点：时间频度的平均情况、等差数列求和（第 7 节）



推理 平均比较次数 $=\dfrac{1+2+\cdots+n}{n}=\dfrac{n(n+1)/2}{n}=\dfrac{n+1}{2}$。$n=99$ 时为 $\dfrac{100}{2}=50$。

图上验证 各柱高成等差数列，平均高度等于首末两项的平均值；$n=99$ 时为 $\dfrac{1+99}{2}=50$。

当前专题知识 数基本操作的次数；等差数列求和。
借用的外部情境 概率统计中“等可能情况下的平均值”。

为什么不算超纲 题干已给出平均值的计算式，只需代入求和公式，不需要学过数学期望。

迁移点 正文讨论的都是最坏情况；这里换一种统计口径（平均），得到 $\dfrac{n+1}{2}$，最高次项仍是 n 的一次方，时间复杂度仍为 $O(n)$。

易错点 答 49.5（用了 $n/2$）或 99（当成最坏情况）。

第六部分　总表：复习时看这几页

第 21 节　易错点、解题步骤与概念对比总表

21.1　高频易错点总表

序	错误想法	正确认识	节
1	数据元素是最小单位	数据元素是基本单位；最小单位是数据项	2
2	顺序、链式是逻辑结构	它们是存储结构；逻辑结构只有集合、线性、树形、图	3、4
3	树是多对多	树是一对多（每个元素只有一个前驱）；多对多是图	3
4	逻辑相邻则物理一定相邻	只对顺序存储成立	4
5	ADT 就是 C++ 的类	ADT 是规格说明，类是实现手段；一个 ADT 可有多种实现	5
6	算法必须有输入；算法就是程序	输入可为零个，输出至少一个；程序可以不满足有穷性	6
7	循环条件 `i<n` 执行 n 次	执行 $n+1$ 次，循环体才是 n 次	7
8	写成 $O(3n^2)$、$O(n^2+n)$	只留最高次项、去系数：$O(n^2)$	8
9	阶低的算法任何时候都更快	只在 n 足够大时成立	8、9
10	n^2 与 2^n 差不多	一个是幂函数，一个是指数函数，相差悬殊	9
11	两重循环一定是 $O(n^2)$	要数每一重各执行几次（可能是 $n\log_2 n$、$n\,m$）	10
12	循环次数多就不是 $O(1)$	只要次数与 n 无关就是 $O(1)$	10
13	空间复杂度要算上输入数据	只统计辅助空间	11
14	两个 `const` 作用相同	参数前的保护参数；参数表后的保护调用函数的对象	12

21.2　基础解题步骤总结

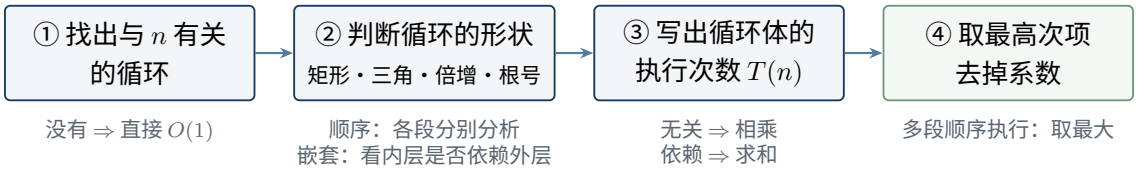


图 34　分析一段程序时间复杂度的四个步骤

代码特征	循环体执行次数	时间复杂度
没有与 n 有关的循环	常数	$O(1)$
`for (i=0;i<n;i++)`	n	$O(n)$
`i=i*2`（或 `*3`、`/2`）	$\lfloor \log_2 n \rfloor + 1$（或以 3 为底）	$O(\log_2 n)$
条件是 `y*y<=n` 一类	$\lfloor \sqrt{n} \rfloor$	$O(\sqrt{n})$
两重，内层 m 次，与外层无关	$n \times m$	$O(m\,n)$；$m = n$ 时 $O(n^2)$
两重，内层 `j<i`	$n(n-1)/2$	$O(n^2)$
两重，内层 `j<=i` 或 `j` 从 `i` 到 `n-1`	$n(n+1)/2$	$O(n^2)$
外层倍增，内层 n 次	$n(\lfloor \log_2 n \rfloor + 1)$	$O(n \log_2 n)$

21.3 相近概念总对比

容易混的一组	一句话区分
数据元素 / 数据项	一条记录 / 记录里的一栏
逻辑结构 / 存储结构	元素间本来的关系（与计算机无关）/ 关系在内存中的表示
线性 / 顺序	逻辑上的“一对一” / 存储上的“连续存放”
顺序存储 / 链式存储	靠位置相邻表示关系 / 靠保存后继地址表示关系
数据类型 / 抽象数据类型	语言已实现的值集加操作 / 只定义不实现的模型加操作
抽象性 / 封装性	只说能做什么 / 把怎么做藏起来
属性类 / 数据操纵类操作	不改变元素和关系 / 改变元素或关系
算法的特性 / 算法的要求	是不是算法 / 好不好
有穷性 / 时间效率	能停下来 / 多久停下来
时间频度 $T(n)$ / 时间复杂度 $O(f(n))$	具体的执行次数 / 只保留增长的阶
时间复杂度 / 空间复杂度	数执行次数 / 数辅助空间

21.4 跨学科 / 相似知识总对比表

节	本章知识	相似知识	✓ 可以直接搬	✗ / △ 不能照搬
1	数据结构模型	方程模型（数学、物理）	先建模再求解	✗ 模型不一定是方程；△ 好坏不看误差而看复杂度
2	数据元素、数据项	C++ 结构体变量、成员	一个结构体变量是一个数据元素	△ 成员可能还能再分；✗ 同一元素内各数据项类型不必相同
3	线性结构	中学数列	每一项有唯一的前一项	✗ 数列可以无穷，数据结构必须有限
4	顺序 / 链式访问	通项公式 / 递推公式	有通项就能直接算	△ 下标从 0 开始；✗ 链表化不出“通项”
5	抽象数据类型	C++ 的类	public 函数是基本操作	✗ 一个 ADT 不止一种实现
6	算法	程序	每一步含义确定	✗ 程序可以不终止、算法不挑语言
7	数循环次数	数列求和	等差数列求和公式	△ 上下限要与循环对准
8	大 O	高数无穷大阶的比较	低阶项可忽略；同阶则大 O 成立	✗ 大 O 不要求同阶，也不保留系数
9	复杂度的阶	物理估算的数量级	抓主要因素	✗ 相差常数倍仍是同阶
10	求和 / 求积定理	加法 / 乘法原理	顺序相加	△ 内层依赖外层时不能直接乘；✗ 相加后只留最大项
11	空间复杂度	C++ 的 sizeof	数组空间与元素个数成正比	✗ 不写 $O(4)$；阶与机器无关
12	函数模板	函数重载	同名函数处理多种类型	✗ 模板只写一份；△ 各类型逻辑必须相同

第 22 节 后续课程连接与参考资料

22.1 这一章的内容以后在哪里再出现

本课程内 **第 2–7 章**每学一种结构（线性表、栈和队列、数组和广义表、树和二叉树、图），都严格按照第 3 节“四个方面”的顺序：先讲逻辑结构和基本操作（给出 ADT），再讲顺序和链式两种存储，然后实现各个操作并分析时间复杂度，最后讲典型应用。**第 8、9 章**的查找和排序几乎全靠时间复杂度来比较算法：笔记里列出的二分搜索、哈希表、AVL 树、堆排序、快速排序、冒泡排序等，都会在那里得到完整的分析。**第 11 章**算法设计中的回溯法，会用到“子集树”“排列树”，并解释怎样避免真正跑满 2^n 或 $n!$。

其他课程 **数据库**：一张数据表就是数据元素的集合，“索引”是为了把查找从 $O(n)$ 降到 $O(\log_2 n)$ 而建立的树形结构。**操作系统**：文件目录是树形结构，进程排队用到队列。**嵌入式与控制类课程**：单片机的内存和主频都有限，写程序时“这段循环是什么阶、要不要额外开数组”是必须回答的实际问题。

拓展联想：以下内容只用于建立知识联系，当前阶段不要求掌握。

本章只用到了表示“上界”的大 O。算法分析中还有表示“下界”的 Ω 和表示“同阶”的 Θ；课件用“极限为非零常数”描述的其实就是 Θ。在算法课中会正式学习。

22.2 参考资料

原始教学依据

(1) 课堂手写笔记《数据结构算法 第一章 绪论 §1 算法复杂度》，1 页，记于 2026 年 10 月 8 日。

(2) 《数据结构与算法》课程课件“第 1 章：绪论”，共 84 页，某高校相关学院 。

(3) 课件指定教材：游洪跃、唐宁九 主编，《数据结构与算法（C++ 版）》（第 2 版）。

核心定义与公式的通行出处 （编写时依据这些教材的标准表述，未逐页对照纸质原书；措辞请以你手头的教材为准）

(4) 严蔚敏、吴伟民，《数据结构（C 语言版）》，清华大学出版社。（数据、数据元素、数据项、数据对象、数据类型、抽象数据类型的定义；算法五个特性中“一个或多个输出”的表述。）

(5) T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, MIT Press.（大 O 记号的定义；O、Ω、Θ 的区别。）

(6) D. E. Knuth, The Art of Computer Programming, Vol. 1: Fundamental Algorithms, Addison-Wesley.（算法的五个特性。）

学科史

(7) “Big O notation”, Wikipedia, https://en.wikipedia.org/wiki/Big_O_notation。（记号 O 由 P. Bachmann 于 1894 年引入、E. Landau 沿用并于 1909 年引入小 o、D. E. Knuth 于 1976 年讨论 Ω 与 Θ。）

(8) B. Liskov, S. Zilles, “Programming with abstract data types”, ACM SIGPLAN Notices, 1974; N. Wirth, Algorithms + Data Structures = Programs, Prentice-Hall, 1976; J. Hartmanis, R. E. Stearns, “On the computational complexity of algorithms”, Transactions of the AMS, 1965。

说明 讲义中的全部数值（如循环执行 1101 次、$30! \approx 2.65 \times 10^{32}$、折半查找最坏比较 11 次等）均用程序实际计算核对；第 12 节的完整 C++ 程序经 g++ 编译运行。生活类比仅用于帮助理解，不替代定义。